國立陽明交通大學

資訊科學與工程研究所

碩士論文

Institute of Computer Science and Engineering

National Yang Ming Chiao Tung University

Master's Thesis

Compile Once, Execute Many：重複性模板實例化任務的可驗證規格編譯方法

Compile Once, Execute Many: Verifiable Specification Compilation for Recurrent Template-Instantiation Tasks

研 究 生： 林冠丞 (Lin, Kuan-Cheng)

指導教授： 陳添福 (Chen, Tien-Fu)

中華民國 一一五年十一月

November 2026

Compile Once, Execute Many：重複性模板實例化任務的可驗證規格編譯方法

Compile Once, Execute Many: Verifiable Specification Compilation for Recurrent Template-Instantiation Tasks

研 究 生： 林冠丞　　Student： Kuan-Cheng Lin

指導教授： 陳添福 博士　　Advisor： Dr. Tien-Fu Chen

國立陽明交通大學

資訊科學與工程研究所

碩士論文

A Thesis

Submitted to Institute of Computer Science and Engineering

College of Computer Science

National Yang Ming Chiao Tung University

in Partial Fulfillment of the Requirements

for the Degree of

Master of Science

in

Computer Science

November 2026

Taiwan, Republic of China

中華民國 一一五年十一月

誌　謝

（致謝待撰寫。）

林冠丞 於

國立陽明交通大學 資訊科學與工程研究所

中華民國 一一五年十一月

Compile Once, Execute Many：重複性模板實例化任務的可驗證規格編譯方法

學生：林冠丞　　指導教授：陳添福 博士

國立陽明交通大學
資訊科學與工程研究所

摘　要

（中文摘要待撰寫。）

關鍵字：（待定，5–7 個）

Compile Once, Execute Many: Verifiable Specification Compilation for Recurrent Template-Instantiation Tasks

Student : Kuan-Cheng Lin Advisor: Dr. Tien-Fu Chen

Institute of Computer Science and Engineering
National Yang Ming Chiao Tung University

Abstract

(Abstract to be written.)

Keywords: (TBD, five to seven).

Contents

Chinese Abstract i

English Abstract ii

Contents iii

List of Figures ix

List of Tables x

1 Introduction 1

1.1 Motivation 1

1.2 A Second Motivation, Which Does Not Depend on Volume 4

1.3 Two Existing Approaches, Both Wrong 6

1.3.1 Approach One: Let an Agent Do It Every Time 7

1.3.2 Approach Two: Freeze It as a Skill or an SOP 8

1.3.3 The Positioning, in One Sentence 9

1.4 Our Position 11

1.4.1 One Question 11

1.4.2 Where the Inference Goes 12

1.4.3 The Conditions Under Which This Is Allowed 12

1.4.4 What the Position Costs 13

1.5 Contributions 14

1.6 Claims and Where They Are Tested 18

1.7 Thesis Organization 20

2 Related Work 23

2.1 Template-Shaped Tasks 24

2.2 Document Formats and Representation Mismatch 27

2.3 Failure Modes 30

2.4 Governance Requirements 33

2.5 LLMs for Document and Engineering-Drawing Understanding 34

2.6 Document Automation, RPA, and Template Filling 37

2.7 Programming by Example and Program Synthesis 39

2.8 Tool-Making, Skill Libraries, and Amortized LLM Computation 42

2.9 Execution-Feedback Program Repair 45

2.10 Verification and Guardrails for LLM Output 47

2.11 Cost-Aware Agent Evaluation 49

2.12 The Families Side by Side 51

2.13 Quantifying the Uncertainty of One Derivation 55

3 System Model 59

3.1 Problem Formulation 59

3.1.1 A Stateful, Two-Stage Input Model 59

3.1.2 Compile-Once, Execute-Many 62

3.1.3 Required Properties of the Specification IR 64

3.1.4 Measured Boundaries of the Specification IR 67

3.1.5 Design Principle: Fail Loud, Not Silent 70

3.1.6 Scope and Applicability Conditions 73

3.2 Domain-Dependent and Domain-Independent Layers 75

3.3 Specification IR 78
3.4 Structured Dump as Grounding 82
3.5 Locator Mechanisms 84
3.5.1 Semantic Locators 84
3.5.2 Range Locators and Source-Side Aggregation 86
3.5.3 Failure Modes and What Each Mechanism Blocks 89
3.6 Target Reference and Write-Back 91
3.7 Deterministic Compiler and Zero-LLM Runtime 95
3.8 Lifecycle Management 98
3.8.1 Lineage and Versioning 98
3.8.2 Regression Suite 99
3.8.3 Drift Detection 99
3.8.4 Auditability 102
3.9 Expressiveness Boundary 104
4 Methodology 107
4.1 State Machine 107
4.2 Structural Gates 110
4.2.1 What a Structural Gate Catches That an Instance Run Cannot 112
4.3 Two Complementary Oracles 113
4.3.1 The Author-Time Expected-Text Oracle 114
4.3.2 The Case-Level Oracle 116
4.3.3 Why Not an LLM Judge 116
4.4 Case-Repair Loop 118
4.4.1 Two Loops, Deliberately Distinct 119

4.4.2 Four Invariants 119
4.4.3 A Refusal That Was Correct 120
4.5 Human in the Loop: Evidence Review 121
4.5.1 What Is Recorded 121
4.5.2 What Publication Refuses 122
4.6 Coverage Is the Real Bottleneck 123
4.6.1 Why the First Denominator Did Not Work 124
4.6.2 The External Denominator 124
4.6.3 Metric Plumbing Is Methodology 125
4.6.4 What Coverage Does and Does Not Measure 126
4.7 Boundaries of Author-Time Verification 127
5 Performance Evaluation 132
5.1 Research Questions 132
5.2 Study Design 134
5.2.1 Why Not a Few-Shot Held-Out Split 134
5.2.2 Two-Level Split 135
5.2.3 Dataset Construction 142
5.3 Baselines and Fairness 155
5.4 Metrics 159
5.4.1 Correctness 159
5.4.2 Reliability, and the Instrument That Measures It 161
5.4.3 Cost 162
5.4.4 Rule Coverage Is an Arm C Diagnostic 166
5.4.5 Asymmetric Rerun Budget 167

5.5 The Evaluator 171
5.5.1 Separate Channels, Never One Average 171
5.5.2 Two Self-Validations, Neither Sufficient Alone 172
5.5.3 A Negative Control for Every Zero 173
5.5.4 The Extent Diagnostic Is Not Symmetric Across Arms 174
5.6 Template Drift 175
5.6.1 What Exists, and at Which Moment It Runs 175
5.6.2 The Variant Corpus, and What It Cannot Reach 178
5.6.3 Metrics for This Axis 181
5.7 Ablations 183
5.7.1 Two Kinds of Ablation, and Which One Is Cheap 183
5.7.2 The Four Planned Ablations 184
5.8 Results 186
5.8.1 Where the Randomness Went 200
5.9 Case Study: Engineering Drawings 205
5.9.1 What the Case Study Is 206
5.9.2 What It Contributes Anyway 206
5.10 Threats to Validity 208
5.10.1 Threats Biased Towards the Proposed Method 209
5.10.2 Threats Biased Against the Proposed Method 215
5.10.3 Threats of Undetermined or Mixed Direction 221
6 Conclusions 228
6.1 Conclusion 228
6.2 Limitations and When This Method Does Not Apply 228

6.3 Future Work 228

References.. 229

List of Figures

List of Tables

Table 1 The three claims, the questions that operationalize them, and what currently stands behind each. No cell in the rightmost column is an experimental result. . 19

Table 2 The five originally claimed gaps, the work that occupies each, and where this chapter discharges it. 23

Table 3 Where each family puts the model, and what follows from that. Every non-empty cell is read from a work whose abstract or full text was fetched for this survey; *not stated* marks a cell the surveyed material does not settle, and is not a claim that the question is unanswered elsewhere. 53

Table 4 Where format knowledge is allowed to live. 76

Table 5 Failure modes and the mechanism that is supposed to make each one loud. . 90

Table 6 Structural fingerprint on the four evaluation templates, two per carrier. . . . 100

Table 7 Two identical writes, three seconds apart, on the real evaluation templates. . 162

Table 8 Derivation-time dispersion, $D = 3$ isolated replicates per evaluation template. The columns are fixed in advance; no replicate has been derived under the protocol and no cell carries a number. 170

Chapter 1. Introduction

1.1 Motivation

A shipping department issues a commercial invoice and a packing list for every outbound consignment. The two documents are the same two Word files every time; what changes is which products went out, in what quantity, in how many cartons. Unit prices come from a product master workbook, quantities from a shipment detail workbook, and the totals, conversions and roundings that tie them together are written down in an operating procedure. In a different sector entirely, a highways district office produces one monthly site-audit report by aggregating however many individual audit sheets its inspectors filed that month — again the same spreadsheet every month, again a written procedure saying which column of which sheet feeds which cell.

Both are instances of the same task shape. A fixed *template* is filled from structured *source data* according to *explicit rules*, over and over, with only the values changing. The work is mechanical, it is high volume, and it is on the critical path of something consequential: the first pair of documents is presented at customs, and the second is a governmental audit record. Both are also real; the two evaluation datasets of this thesis are built from artifacts of exactly these two kinds, downloaded rather than drawn, and Section 5.2.2 describes them.

What makes the task worth automating is not what makes it hard. That a person filling in the same form for the four hundredth time is slow and will eventually mistype a figure is

the obvious argument, and it is the weaker one. The difficult property is the shape of the failure. When this task goes wrong, nothing crashes. There is no exception, no malformed file, no red mark. What is produced is a document that opens correctly, is laid out correctly, is complete, is plausible in every respect — and carries one wrong number. This thesis calls that a *silent failure*, and treats it as the primary risk to which every design decision reported here is answerable. Section 3.1.5 states the principle that follows from it, and Section 5.5.1 states how the evaluation scores it as a distinct outcome rather than folding it into an accuracy figure.

Silent failure is not a hypothetical hazard invoked to motivate a system. Two measurements in this thesis are specimens of it, and one of them was produced by the proposed method itself. A rounding-mode defect in the host language turned a volumetric weight of 0.053 into 0.052 on three of the DOCX dataset's seventy-nine detail rows, one of them inside a sealed evaluation instance; every gate in the system passed both values, because both are entirely ordinary-looking figures with the right unit and the right precision (Section 3.7). Separately, executing the best available compiled specification for the XLSX dataset over five instances produced 347 correct semantic values out of 350, and all three failures were a remarks column that the specification did not write at all — a column that stayed blank, in a file that was otherwise perfect. The system's own publish gate had flagged the underlying gap, and the classification the evaluator assigned those three cells was `silent_failure`. A system whose motivating risk is one it can be shown to have produced stands on firmer ground than one whose motivating risk is only asserted.

Carriers. The main evaluation carriers of this thesis are DOCX and XLSX. Engineering drawings (DWG) remain in the repository and are reported as a case study, and the reason they are excluded from the main result table is neither their difficulty nor their prestige: the entire system was originally built on a drawing template, so every prompt, gate and representation

choice was tuned against it, and any figure it produced would be contaminated by construction. That is a validity precaution, not a showcase, and Section 5.9 reports the case study on those terms. The secondary observation — that a drawing is the carrier on which serializing a document into tokens destroys the most meaning, since its semantics live in spatial layout, layers and block attributes — belongs to Section 2.2, where it is deliberately treated as an observation about representation rather than as an argument for the method, because an argument of the form "models cannot read this format" is answered by the next model.

Status: the risk characterization is implementation-backed; the volume premise is normative. The two specimens of silent failure above are measurements taken in this repository: the rounding sweep over the DOCX dataset's detail rows, and the offline execution probe over five XLSX calibration and development instances that scored 347 of 350 semantic checks with three `silent_failure` outcomes. Neither is a formal experimental result; both were produced outside the evaluation harness, and Section 5.8 is where results belong. The premise that a template is instantiated tens to thousands of times is *not* measured anywhere in this thesis: no survey of real instantiation frequency was conducted, and the two datasets carry ten instances each because that is how many were built, not because that reflects field practice. The bias direction is favourable to the method, since recurrence is precisely the condition under which a one-off derivation cost amortizes; it is therefore stated as an admission condition rather than as a finding in Section 3.1.6, and Section 5.4 reports a break-even instance count rather than assuming one.

1.2 A Second Motivation, Which Does Not Depend on Volume

The motivation of Section 1.1 is an argument about volume: the same form is filled four hundred times, so a one-off derivation amortizes. That argument has an honest weakness, disclosed in its own Status paragraph and again in Section 5.8.1 — below some break-even instance count it fails, and this thesis has not measured where that count lies. A second motivation exists which does not depend on N at all, and it is stated separately here for exactly that reason.

The constraint is that the data cannot leave. An organization holding personal, commercial or regulated records is frequently in a position where the capable models are reachable only as a network service and the records may not be sent to one. The usual response is to run a smaller model on the organization's own hardware and accept a weaker one. This thesis does not measure any local model and makes no claim about how much weaker such a model is; the constraint is treated as a deployment condition reported by practitioners rather than as a performance finding, and nothing below rests on it, for a reason that is worth stating early: the arrangement proposed here needs *no* local model, so how good a local model would have been is not a premise of the argument but only of the alternative it displaces.

The arrangement, and why this thesis has already performed the hard half of it. Compile-once, execute-many splits the work at exactly the boundary this constraint needs. Derivation consumes a template, a rules document and a small number of calibration instances, and it emits a specification. Execution consumes the specification and the real source data and emits documents, with no model call at all (Section 1.4.2). Placing the derivation in the cloud and the execution on the organization's own machines therefore sends the template, the rules and the

calibration instances outward, brings a specification back, and never sends a real record anywhere. What makes this more than a proposal is that the calibration instances of this thesis are *already synthetic*: all three datasets are populated by hand-written generators over templates that are genuine circulating forms, so every derivation reported here has been a derivation from synthetic instances of a real template. The half that would need demonstrating is the half that has been demonstrated throughout; what has not been demonstrated is the deployment.

The claim is also stronger than the more obvious version of it, which is to ship a compiled tool for a small local model to call. Nothing local is needed. What crosses back is a specification that a person can read to the end — its fields, its locators, its transformations, its extents and the locations it is forbidden to write — rather than a model whose behaviour nobody can read to the end. That is a direct consequence of C4 and of the same negative-control test that pins it, and of the measurement in which the shipping production path produced byte-identical output on five instances across 380 compared cells (Section 5.10).

Four boundaries, stated here rather than later, because without them this reads as a privacy guarantee. First, three things cross outward and only one of them is obviously innocuous: the template, usually a blank form; the *rules document*, which may itself be confidential, since an operating procedure can encode pricing policy, thresholds or internal structure; and the synthetic calibration instances. Only the specification crosses back, and no real instance data crosses in either direction. Second, a template revision invalidates the specification and sends the organization back to the cloud for another derivation, so the frequency of that round trip is the revision rate — which is the axis of RQ4 and of Section 5.6, not a detail. Third, the synthetic calibration instances must be faithful enough; where they are not, the derived specification will be wrong on real data *deterministically and silently*, which is this thesis's headline failure mode arriving through a new door, and the existing disclosure of it is the shared-misreading class of

Section 5.10 and the single-example primitives recorded there alongside it. Fourth, and most importantly, **this is not a security argument.** No threat model is constructed anywhere in this thesis, and no claim is made that a specification is free of information that could be traced back to source data — an anchor string in a locator may well be a real field name lifted from a real document. Readability is not confidentiality. What is claimed is architectural: the category of data that must cross the boundary is different, and it is smaller.

Status: normative throughout, and no enterprise deployment has been performed. This section takes the third of the three admissible responses to a gap: the claim is made, labelled as a design argument rather than a finding, and disclosed as a threat in Section 5.10 rather than left in the introduction alone. No on-premises installation exists, no organization has used this system under such a constraint, and no experiment in this thesis varies the deployment topology. Two of its components are nonetheless implementation-backed rather than asserted, and they are the two that would be hardest to retrofit: that execution performs no model inference is pinned by a negative-control test (Section 1.4.4), and that calibration instances are synthetic while templates are real is a property of the datasets as built (Section 5.2.3). The bias direction is favourable to this thesis, because a motivation that requires no experiment is a motivation nobody has yet had the opportunity to falsify; Section 5.10 records it under that heading. Finally, this section makes no claim that any regulation *requires* deterministic output, which Section 2.4 looked for in the literature and did not find.

1.3 Two Existing Approaches, Both Wrong

Two ways of automating this task are already available, and this thesis argues that both are placed wrongly rather than that either is incompetent. The distinction matters, because a

contribution that depends on its comparators being bad is worth as much as its comparators.

1.3.1 Approach One: Let an Agent Do It Every Time

The immediately available approach is to hand each instance to a language-model agent along with the template, the rules and the source files, and take the document it returns. Nothing persists between instances: the four hundredth consignment starts from the same prompt as the first, with no artifact carried over.

The usual objections — that this is slow, and that it costs a model call per instance — are true and are the least interesting things wrong with it. Latency and price are quantities, and quantities improve. The structural objection is that correctness is *re-gambled on every instance*. Each document is an independent draw; a run that is right tells you nothing about the next run, because nothing about the first run is carried into the second. And since the failure mode of this task is silent (Section 1.1), an independent draw whose bad outcomes are invisible is a poor thing to place on the critical path of a customs declaration.

A pilot measurement reported in Section 5.1 sharpens this into a claim that can be defended rather than assumed, and it did so by refuting the easier version of it. Running a current frontier agent three times over one instance of the DOCX dataset produced semantically accurate output on all three runs — the cross-file lookup, the rounding, the gram conversion and the aggregation extent were correct every time — and three different files, with cell-level agreement between runs of 69.2%. Every divergence was a formatting decision retaken from scratch on each run. So the defensible statement of the objection is not that a competent agent gets this task wrong. It is that a competent agent gets it *differently right* each time, that no artifact exists which could be reviewed, versioned, diffed or signed off before the four hundred documents are produced, and that the cost of the correctness it does achieve is paid four hundred times. That pilot was

run outside the evaluation harness and is labelled a pilot throughout this thesis for exactly that reason; it is not an Arm A result.

1.3.2 Approach Two: Freeze It as a Skill or an SOP

The second approach is the one a practitioner reaches for after paying for the first: have the agent generalize from a few worked examples into a reusable artifact — a skill file, a checklist, a written procedure, a script — freeze it, and have an agent follow it on every subsequent instance. This is the correct instinct. It is the same instinct this thesis has, and the disagreement is about where the artifact sits relative to the execution path rather than about whether there should be one.

An earlier draft of this chapter dismissed the approach on the grounds that such an artifact's correctness is never tested. That statement is not available to this thesis, because this thesis builds a tested version of it. Arm B of the evaluation (Section 5.3) constructs exactly this: an agent generalizes freely from the calibration instances into an artifact of its own design, the artifact is hashed and frozen, its provenance is bound to the build envelope it came from, and every evaluation instance is then executed by an agent that may read it and may not modify it. The freeze is enforced by the harness rather than by instruction — an attempt to write to the frozen artifact is recorded as a protocol violation — and the resulting documents are scored by the same deterministic evaluator as every other arm.

The real deficiency is therefore narrower, and it survives the strengthening. Whatever is frozen, *execution still routes through a model*. The artifact is read, interpreted and applied by an inference step, so the output remains stochastic, remains priced per instance, and remains capable of being right in a different way each time. And the artifact carries no executable contract: nothing binds it to the rule document it was derived from, nothing enumerates which

rules it covers, and nothing fails when the template underneath it changes. Its correctness, if established at all, is established the same way the agent's output was — by looking at what came out. What this thesis proposes is not a better artifact but a different *kind* of artifact: one that a deterministic program can execute without interpretation, and against which those questions can be asked mechanically.

Refusing the straw man has a price, and it is worth naming, because it is the reason Arm B exists at all. If only the per-instance agent and the proposed method were compared, any advantage of the latter could be attributed to the mere existence of a persistent artifact rather than to anything about how that artifact is produced and verified. Arm B holds persistence fixed and varies only the production procedure, which makes it the strongest available objection to this thesis's result, and it is entitled to win.

1.3.3 The Positioning, in One Sentence

Placing the two approaches beside the older automation tradition yields the shortest statement of where this work sits:

- **Robotic process automation and template-filling tooling** are *deterministic but manual*. Execution is a program, so it repeats exactly, can be versioned and can be audited — but a person authored the mapping, cell address by cell address, and a person must reauthor it when the form changes.

- **Per-instance agents** are *automatic but stochastic*. Nobody writes the mapping, and nothing about the execution repeats exactly.

- **This work** is *automatic derivation plus deterministic execution*. A model authors the mapping once, under verification; a program executes it thereafter.

The two existing approaches are not opposite errors to be split between. They occupy opposite corners of one table, and the corner this work is placed in is the one in which the authoring is automatic and the execution is not. Section 2.6 develops the three-way division against the literature.

That corner is not empty, and saying it was would repeat the error this chapter has already been corrected for four times. Programming by example has occupied it since 2011 [1] and occupies it today inside a shipping spreadsheet product [2]: a program is synthesized automatically and then executed deterministically, which is this work's arrangement exactly. The difference is not the corner but what has to be inferred to enter it, and Section 2.7 states it precisely. The three bullets above are the abbreviated form of a comparison that Table 3 makes across nine families and four decidable coordinates; where the two disagree, the table is the authority.

Status: the taxonomy is normative; the criticism of approach one is pilot-supported; the criticism of approach two is implementation-backed. The three-way division is a positioning argument, not a measurement: no experiment in this thesis varies automation paradigm, and no citation is attached to the characterization of robotic process automation until Chapter 2 supplies one. It is also a division over three traditions rather than a partition of the design space, which the paragraph above concedes explicitly after Table 3 showed that a fourth tradition shares this work's corner. The claim about approach one rests on a three-run pilot on a single instance, which is enough to establish that run-to-run divergence exists and nowhere near enough to quantify it; the formal instrument is the rerun budget of Section 5.4.5, which has not been executed. The claim about approach two is implementation-backed in the specific sense that matters — the frozen-skill contract exists, is tested, and enforces freezing, provenance and read-only execution in `benchmark/harness/contracts.py` and

`harness.runner.build_arm_b_skill` — but the live adapter that would drive Arm B against a model has not been built, so no comparative figure for it exists.

1.4 Our Position

1.4.1 One Question

The related-work survey conducted for this thesis returned an uncomfortable finding: of the five gaps the work initially claimed, every one had a neighbour that already occupied it. Amortizing model computation into reusable tools, repairing programs from execution feedback, verifying generated output against oracles, versioning and auditing generated artifacts, and evaluating agents for cost rather than accuracy alone are each somebody's contribution already. A novelty claim of the form "others have these five things separately and we have them together" is a conjunction, and a conjunction is the weakest shape a contribution can take, because it invites the reader to remove one conjunct at a time.

This thesis therefore argues one question:

> Under which *executable conditions* may a language model be removed from the recurring execution path permanently, without losing correctness, traceability, or fail-loud behaviour?

The word carrying the weight is *permanently*. The nearest neighbour the survey identified — an industrial system that observes the same problem, compiles recurring procedural steps once into tested tools before deployment, and carries labelled-case repair, versioning and audit logging — nonetheless retains a model agent at run time, with code generation available as a fallback. A fallback is not a small residue of the same design; it is the difference between a guarantee and a tendency, because the properties one wants from determinism are exactly the

properties a fallback suspends, and it suspends them precisely on the inputs that are unusual. Section 2.8 places this work against that neighbour in detail.

1.4.2 Where the Inference Goes

The move is to relocate inference rather than to eliminate it. A model is used once, to derive a *specification* from the template, the rules and a small number of calibration instances. That specification is an explicit intermediate representation — not a prompt, not a script, not natural language — and it is subjected to executable verification before it may be used: structural gates that reject malformed or unexecutable specifications, a semantic oracle that compares what the specification would write against an independently produced expected document, and a bounded repair loop whose invariants forbid a repair from becoming a rewrite. Every instance thereafter is produced by a deterministic compiler and a format-specific writer, with no model call at all.

In cost terms the arrangement replaces $O(N)$ model invocations with $O(1)$ model invocations plus $O(N)$ deterministic executions, where N counts instances of one template revision. Two qualifications belong in the same breath as that expression, since both are places where it is routinely overstated. The constant is per template *revision*, not per template for all time: a template that changes requires a new derivation, and detecting that it changed is itself part of the design (Section 3.8.3). And the $O(1)$ is not free of the repair loop — every retry, every rejected candidate and every failed attempt is derivation cost and is counted as such, which is the rule Section 5.4 imposes on the cost table.

1.4.3 The Conditions Under Which This Is Allowed

The question above asks for conditions, and the answer is three of them, stated as an admission test in Section 3.1.6 and referred to throughout as C-I, C-II and C-III: the task must *recur*;

its outputs must be *deterministically derivable* from structured source data without judgement; and its mapping must be written down as *explicit operational rules*. They are independent, and each carries an operational test — a question one could actually ask before adopting the method — which is not restated here. The scope of this thesis is precisely the region in which all three hold. A task containing one field requiring professional discretion fails C-II for that field, and the honest response is to carve the field out rather than to widen the method; a task performed once fails C-I and should be done by hand or by an agent, since nothing about this method is cheap at $N = 1$.

1.4.4 What the Position Costs

Relocating inference does not eliminate randomness. It moves it, and this thesis is obliged to report where it went. The derivation is stochastic, and the same prompt over the same bytes does not produce the same specification. Two consecutive derivations of the XLSX dataset with byte-identical inputs differed in ways that matter: one declared three detail slots and the other seven, one emitted a source-artifact collection and the other emitted none, one covered fifteen of twenty-seven rules and the other twelve, and one raised four hard structural errors while the other raised none. That is not a footnote — it is the direct subject of RQ2 in Section 5.1, and reporting this method as having "zero variance" while omitting it would be a misstatement, since the variance is not absent but relocated into the one step whose output a human is expected to review.

What is bought in exchange is that the variance is now attached to an artifact that exists before any document is produced. A specification can be gated, oracle-checked, diffed against its predecessor, hashed, and refused; a distribution over four hundred future documents cannot. Section 4.5 makes the corresponding argument about human effort, which does not disappear

but changes units.

Status: the relocation is implementation-backed; the amortization is normative. That execution contains no model call is the narrowest and best-evidenced claim in this thesis, and it is pinned by a negative control rather than by inspection: `test_successful_production_operation_records_zero_llm_calls` replaces both agent repair entry points with a recorder that returns a failure, runs a complete production operation, and asserts both that the recorder was never invoked and that the operation's persisted model-call count is zero (Section 3.1.2). Reading the source would establish only that no call site exists today; the test also fails if one is reintroduced. The derivation-variance figures above are measured, from two paid derivations recorded in the repository. The amortization claim is *not* established: no break-even instance count has been measured, because the formal cost experiment has not been run. What exists is one derivation's cost on the XLSX dataset — 553.4 seconds of wall clock, one model call, 128,299 input and 20,068 output tokens, with no repair round triggered — taken by an offline probe rather than by the harness, which fixes the order of magnitude of the numerator and says nothing about the denominator. The bias direction is favourable to the method: an unmeasured break-even point is easy to assume is small.

1.5 Contributions

The five contributions of this work are conventionally listed in parallel. They are not parallel, and presenting them so would misrepresent which of them is load-bearing. The related-work survey's verdict was that C4 — zero-model execution — is the only claim without a close neighbour, and this chapter takes that verdict literally: C4 is the claim, and the other four are what make C4 worth having rather than merely true.

The distinction is not rhetorical. Removing a model from an execution path is trivially achievable by a system that deterministically writes the wrong thing, or that can express only tasks so simple nobody needed a model for them, or whose one derivation nobody can check, or whose advantage nobody can measure. Each of the remaining contributions closes one of those escapes.

C4 — zero-model execution. Once a specification is published, producing an instance involves no model call: the deterministic compiler emits a carrier-neutral write plan and a format-specific adapter applies it. Cost, latency and run-to-run variance collapse together, and lineage, versioning and regression become available because there is a versioned program to trace. Developed in Section 3.1.2 and Section 3.7; its consequences for consistency and cost are RQ2 and RQ3.

C1 — the formalization, and what the representation must satisfy. Template instantiation is formalized as compile-once/execute-many over a stateful two-stage input model, and five properties are stated that any specification representation must satisfy for the split to be sound — separation of structure from instance values chief among them. This is what stops C4 from being a claim about one program: without it, "we removed the model" describes an implementation. Developed in Section 3.1 and Section 3.1.3; its measured limits are B1–B7 in Section 3.1.4.

C2 — the specification IR and its locator mechanisms. A carrier-neutral intermediate representation with semantic source locators, range extents with declared inclusion boundaries, grouped multi-row records, and equality merges between resolved row sets. This is what stops C4 from being a claim about tasks too easy to have needed a model: it determines which real forms are expressible at all. Developed in Section 3.3 and Section 3.5.

C3 — the derivation closed loop. The one expensive step is made reviewable: an agent proposes, executable structural gates and two independent oracles judge, a bounded repair loop with anti-overfitting invariants revises, and a human reviews evidence rather than output. This is what stops C4 from being a claim nobody can check. Developed in Chapter 4; its limits are D1–D6 in Section 4.7.

C5 — the cost-aware evaluation method and datasets. A three-arm protocol with material and tooling parity, a deterministic evaluator with separate channels and no model in the scoring loop, silent failure as an explicit outcome class, an externally supplied coverage denominator, and three datasets built from genuine third-party forms. This is what stops C4 from being a claim nobody can measure. Developed in Chapter 5.

A correction to the contribution table this thesis inherited. The planning document that defines C1–C5 maps them onto an eleven-chapter structure (C1→3, C2→4, C3→5, C4→6, C5→7). This thesis has six chapters, and the mapping above supersedes it. Any reading that expects C5 in a seventh chapter is reading the superseded plan.

Status: C4 implementation-backed; C1 and C5 mixed; C2 and C3 each carry one named component that is not evidenced. C4's evidence is the negative-control test named in Section 1.4.4, and it is deliberately the narrowest claim here: it covers execution, not derivation, and not the derivation-time repair loop, which is by construction inside the step whose cost repeated execution amortizes.

C1's formalization and its five properties are stated against the implementation, and each property's status is given individually in Section 3.1.3. The IR has a published contract, and the honest measurement is that 24 of the 35 specifications stored in this repository satisfy it, so the contract is described as published and partially enforced rather than as an invariant.

C2 is implementation-backed for the source-side locators, extents, records and merges, each exercised on a main evaluation dataset. One named component is not: label-proximity resolution of *write* targets has never executed on either main carrier — of thirty-three stored specifications, the thirteen geometrically matched fields all belong to three drawing specifications from an earlier period, and the two office-document datasets contribute zero. The response taken is the third admissible one: the mechanism is retained and described, and this thesis does not claim it works on DOCX or XLSX (Section 4.7, D3). The original C2 statement also claims resilience to layout change; the drift-*detection* half of that is measured (Section 3.8.3), while the re-derivation-under-drift half is RQ4 and has not been run.

C3 is implementation-backed for the derivation-time loop, its four invariants, and the author-time oracle, all of which run on every derivation and are pinned by tests. The case-level repair loop named in the original C3 statement is implemented and tested as pure functions but *has no caller on the automated path* — only an offline operator tool and tests reach it. It is therefore presented as a designed and tested mechanism whose automated integration is future work, and no measured result may be attributed to it (Section 4.4).

C5's protocol, evaluator, scoring policy, coverage denominator and all three datasets exist and are frozen, their content sealed and their reference outputs reviewed by a person to the limited extent T12 records. The methodology freeze completed on 30 August 2026 and the formal run began the same day: nine scored-eligible attempts exist across three templates and the two baseline arms, and six produced documents have been scored. *No template has a complete arm and the compiled arm has not been run under the protocol at all*, so no proportion in Chapter 5 is yet a result. C5 is thus a contributed *method* with a partial run behind it, not yet a contributed result.

1.6 Claims and Where They Are Tested

Three claims follow from the position above. Each is stated here in the form it will be tested in, with the research question that operationalizes it and the section that reports it. None of the three has a result yet; Table 1 records what currently stands behind each, so that a reader can tell an intention from a finding at a glance.

Claim 1 — correctness is comparable, and execution-time variance is zero while derivation-time variance is not. Field-level semantic accuracy of a compiled specification is at least as good as a per-instance agent's on the same instances under the same evaluator, and repeated execution of a published specification is bit-identical. The second half of the claim must be stated with its complement or it is false as an account of the method: the variance has moved to the derivation, so the claim is reported as two measurements — run-to-run variation of the baselines, and derivation-to-derivation variation of the specifications. An earlier formulation of this claim said simply "variance is zero"; that formulation is withdrawn. Operationalized by RQ1 and RQ2, measured by the metrics of Section 5.4 under the rerun budget of Section 5.4.5, and reported in Section 5.8.

Claim 2 — amortized cost is lower by one to several orders of magnitude, and there is an identifiable break-even instance count. The comparison is between a per-instance model cost and a one-off derivation cost plus a near-zero marginal execution cost; the quantity of interest is where the two cross, and how sensitive that crossing is to derivation retries. Repair and regeneration are derivation cost and are not excluded from the denominator. Operationalized by RQ3, defined in Section 5.4, reported in Section 5.8.

Claim 3 — failure is loud rather than silent. When the template changes underneath a frozen artifact, the compiled method fails at a gate with an auditable error that locates the change, at a bounded re-derivation cost, whereas an arm that interprets the template afresh produces a plausible wrong document. This is the direct test of the design principle of Section 3.1.5, and the reason the evaluation scores silent failure as its own outcome class. Operationalized by RQ4 and reported in Section 5.6.

Table 1: The three claims, the questions that operationalize them, and what currently stands behind each. No cell in the rightmost column is an experimental result.

Claim	RQ	Reported in	Current evidence
1. Comparable correctness; zero execution variance, non-zero derivation variance	RQ1, RQ2	Sections 5.4, 5.4.5, 5.8	Execution determinism follows from C4's negative-control test. Derivation variance is measured, on two paid derivations with identical inputs. Comparative accuracy: not measured; a three-run pilot outside the harness found the baseline semantically accurate.
2. Order-of-magnitude amortized cost; identifiable break-even N	RQ3	Sections 5.4, 5.8	One derivation's cost is recorded by an offline probe. Per-instance baseline cost, marginal execution cost, and the crossing point: not measured.
3. Loud failure under template drift	RQ4	Section 5.6	Fingerprint sensitivity measured on both evaluation templates, with a library round trip as the control; gate refusals are implemented and tested; twenty-eight single-variable variants are built and validated, twelve of them classified as outside what the detector can see. Comparative behaviour of the baselines under drift: not measured; the experiment has not been run.

Status: all three claims are normative as results; their mechanisms are implementation-backed. No arm has been executed under the evaluation protocol, so no figure in this thesis answers any of the three claims. What is implementation-backed is the machinery each claim will be tested with, together with those components of each claim that are not compara-

tive: execution determinism, derivation-time variance, gate refusals and fingerprint sensitivity are measurements, whereas parity, cost ratio and the drift behaviour of the baselines are not. The bias direction of stating the claims ahead of the results is favourable to the method, and the mitigation adopted is the one used throughout this thesis — every section ends with a paragraph like this one. Section 5.10 enumerates the threats to validity individually, with those favouring the proposed method listed first.

1.7 Thesis Organization

Chapter 2 characterizes the task and its failure modes, describes the representation mismatch between document formats and token sequences, and places this work against its neighbours — tool-making and skill libraries, programming by example, execution-feedback repair, output verification, and cost-aware agent evaluation — with the three-way positioning of Section 1.3.3 developed against the literature.

Chapter 3 and Chapter 4 divide the system along a deliberate seam. Chapter 3 is what exists statically: the formalization, the properties a specification representation must satisfy, the admission conditions, the carrier-dependent and carrier-neutral layers, the representation itself, the locator mechanisms, and how a specification is executed and governed over its lifetime. Chapter 4 is how a specification comes to exist: the derivation state machine, the structural gates, the two oracles, the repair loop and its invariants, the evidence a human reviews, and how rule coverage is computed against an external denominator.

Chapter 5 gives the evaluation: the research questions, why a conventional few-shot held-out split is the wrong instrument here, the two-level split and the datasets, the three arms and the two layers of fairness between them, the evaluator and its separate scoring channels, the drift and ablation designs, the results, the drawing case study, and the threats to validity. Chapter 6

answers only what the evidence supports, and develops the limitations as future work.

Two conventions the reader should know in advance. First, **every section of this chapter and of Chapters 3 through 5 ends with a paragraph beginning "Status:"**, which states whether that section's claims are implementation-backed — naming a file, a function, a test or a measurement — or normative, in which case the direction of the resulting bias is disclosed. There is no unmarked middle state. The convention exists because it was earned. Successive audits of this document's own outline, conducted while writing Chapters 3 to 5, found ten passages in which the plan had drifted away from the implementation — every one of them unmarked, and every one of them in the direction of *understating* the system or misidentifying which side a risk favoured. One described C4, the central claim of this thesis, as not yet supportable, months after the execution path it doubted had been removed. Auditing the present chapter's notes before writing it found four more, and it is worth recording that three of those four lean the opposite way: they overstated the method, by dismissing a comparator that this thesis in fact implements and tests, by presenting a validity precaution about the drawing carrier as a showcase, and by claiming a variance of zero that holds only for half of the pipeline. An introduction is the chapter most likely to drift towards its own argument, which is the reason this one carries the same convention as the rest.

Second, the boundaries of the method are stated in three separate lists that are deliberately not merged: Section 3.1.4 (B1–B7) is what the representation can express, Section 4.7 (D1–D6) is what author-time verification can see, and Section 3.9 (E1–E3) is what the running system still cannot promise. "A merge does not perform outer joins" and "the oracle cannot see a cell that is blank on both sides" are different kinds of statement, and a combined list would invite a reader to treat them as interchangeable.

Status: descriptive. This section makes no claim requiring evidence. The audit it refers to is recorded in the project's completed-work log, and each of its seven findings is reflected in the section it corrected.

Chapter 2. Related Work

The survey behind this chapter produced one uncomfortable result, and the honest thing to do is to put it first. Of the five gaps this work originally claimed — amortizing model computation into reusable artifacts, repairing generated programs from execution feedback, verifying generated output against oracles, versioning and auditing what a model produced, and evaluating agents for cost rather than accuracy alone — every one already has an occupant, and in four of the five cases the occupant is stronger than the version this thesis would have claimed. A related-work chapter whose function is to report that nobody has done any of these things would be false. This chapter therefore does something else: it locates each component in its own literature, states as narrowly as possible what remains, and marks each remaining claim by whether the negative half of it was actually checked.

Table 2: The five originally claimed gaps, the work that occupies each, and where this chapter discharges it.

Originally claimed	**Already occupied by**	**Discharged in**
Amortizing model computation into reusable artifacts	Tool making and library learning [3, 4, 5, 6, 7], and an industrial deployment [8]	Section 2.8
Repairing generated programs from execution feedback	Execution-guided synthesis and self-debugging [9, 10, 11, 12]	Section 2.9
Verifying generated output before reuse	Validation before abstraction [6, 5]; automated oracle construction [13, 14]	Sections 2.9 and 2.10
Versioning and auditing generated artifacts	Versioned tools with invocation logs and drift monitoring [8]	Sections 2.4 and 2.8
Cost- and variance-aware agent evaluation	A fast-growing evaluation literature [15, 16, 17, 18, 19]	Section 2.11

Scope of the survey, and what "verified" means here. The survey records 64 works in `docs/thesis/refs/manifest.md`, of which 62 are cited here, with, for each, the metadata claimed at the time the citation was proposed, the metadata fetched from the publisher or preprint page, and the venue as DBLP records it. Three facts about that reconciliation matter for how this chapter should be read. First, the survey draft labelled 26 of the 48 primary works "arXiv preprint" where DBLP records a peer-reviewed venue, and named a title that the cited identifier no longer carries in two further cases; the bibliography follows DBLP rather than the draft. Second, six works dated 2026 bear a disproportionate share of the argument, and for those six the full text was retrieved and the load-bearing sentences copied verbatim into `docs/thesis/refs/abs/`, because they are the works most likely to be misremembered in the direction that flatters this thesis. Third, eight of the 64 belong to two subject areas the survey draft did not cover at all — program-aided language models and uncertainty quantification — and were searched and fetched from scratch for Sections 2.12 and 2.13; one of the eight turned out to carry a published title different from the one claimed for it, which is the same class of error the original reconciliation found twice. An unreliable source list and an incomplete one fail differently: the first produces a wrong sentence that can be grepped for, and the second produces no sentence at all. Where this chapter asserts that some line of work does *not* do something, that assertion was checked against that set and no further; the wording says so each time, and a gap that was not checked is labelled as unchecked rather than quietly asserted.

2.1 Template-Shaped Tasks

The task class this thesis addresses can be stated in one line. A *template* T is a native office artifact — a `.docx`, `.xlsx` or `.dwg` file — whose layout, static text and structural anchors are fixed. An *instance* is produced by combining T with a set of values V drawn from structured

source data, yielding $D = f(T, V)$. Chapter 3 gives the formal version; what matters here is where the difficulty lives, because the literature reviewed below is organized by which part of f each line of work attacks.

The arithmetic in f is rarely hard. In the two evaluation datasets of this thesis it is multiplication, division by a unit constant, rounding, summation over a row range, and equality joins between two tables. What is hard is the other half of f: deciding *which* structural location in T receives *which* semantic field. A spreadsheet cell has no name; a paragraph in a form has no identifier; a blank underline after a Chinese label is, in the file's own data model, indistinguishable from any other run of whitespace. The binding between "port of loading" and a particular paragraph, or between "this month's audit count" and column F of row 8, is nowhere in the file. It has to be inferred, and once inferred it has to survive the next revision of the form. This is the reason the thesis calls its central mechanism a *locator* rather than a mapping, and the reason Section 2.3 lists locator drift as a first-class failure mode rather than as an implementation detail.

Three further properties delimit the class, and they are stated as an admission test in Section 3.1.6 rather than as a description: the task must *recur* over many instances of one template revision; its outputs must be *deterministically derivable* from structured source data without professional judgement; and its mapping must already be written down as *explicit operational rules*. The third property is what distinguishes this class from document generation in general, and it is also what makes the class auditable: if a rules document exists, then coverage of that document is a measurable quantity, which Section 4.6 exploits and which, as far as this survey checked, no reviewed system treats as an obligation.

The closest work in subject matter is a multi-agent system that generates semi-structured public-administration documents from semantic templates, combining retrieval, prompting and

several cooperating agents [20]. Its material is very close to the government-form benchmark of this thesis, which is a reason to cite it early rather than defensively. Its "semantic template" is nonetheless an input to prompting, and the document is produced by the agents at the moment it is requested; nothing persists afterwards that could produce the next instance without them.

None of the reviewed literature adopts the template family as its unit of evaluation. The spreadsheet and office benchmarks evaluate one instruction against one workbook, or one instruction against several workbooks [21, 22, 23]; the computer-use benchmarks evaluate one task in one environment [24, 25]; the programming-by-example literature evaluates one transformation learned from its own examples [1, 2]. This is an observation about evaluation units, not a deficiency claim: nothing prevents those benchmarks from being extended, and the reason this thesis needs a different unit is that the quantity it wants to report — one derivation cost amortized over N deterministic executions — is undefined unless the family, rather than the instance, is the unit.

Status: the task definition is normative; its instantiation is implementation-backed. The three admission conditions are a design decision of this thesis, not a finding, and no experiment here varies them; Section 3.1.6 states them with an operational test each so that a reader can at least apply them. That the class is non-empty and contains real forms is implementation-backed: the DOCX dataset is built from a commercial invoice and packing list published by a freight forwarder, and the XLSX dataset from a Directorate-General of Highways monthly site-audit report, each with ten instances, both under `benchmark/`. The premise that such templates are instantiated tens to thousands of times in practice is *not* measured anywhere in this thesis — the datasets carry ten instances each because that is how many were constructed — and the bias direction is favourable to the method, since recurrence is precisely the condition under which a one-off derivation amortizes. Section 1.1 discloses the same premise on the same terms.

2.2 Document Formats and Representation Mismatch

The carriers of this thesis have data models with far more structure than their rendered appearance suggests. An Office Open XML word-processing document carries bookmarks, content controls with stable tags, table grids, and paragraph and run boundaries that split a visually continuous sentence into arbitrarily many fragments. A spreadsheet carries defined names, merged ranges whose value lives only in the anchor cell, number formats that are presentation rather than content, and formulas alongside cached values that may disagree. A drawing carries an entity graph with layers, block references with attributes, and extension data attached to entities. In all three, a substantial part of the meaning is positional: this cell is the current-month column because it sits under a header two rows up; that paragraph is the consignee because it follows a label.

Serializing such an artifact into a token sequence destroys exactly that positional information, and a body of work exists precisely to mitigate the loss. SpreadsheetLLM compresses large sheets while preserving cell addresses, values, formats and structural anchors, and evaluates table detection and spreadsheet question answering on the result [26]. LayoutLM jointly models text and two-dimensional layout for form understanding [27]; Donut removes the external optical-character-recognition stage and decodes structured information directly from a document image [28]; and DocLLM folds bounding-box layout into the attention mechanism itself [29].

This thesis therefore *demotes* representation mismatch from an argument to an observation, deliberately and at some cost to the rhetorical force of the motivation. An argument of the form "models cannot read this format" is answered by the next model, and the works just cited are the evidence that it is being answered steadily. The inference this thesis draws instead concerns

where the reading happens: supply the structure through tools that produce a faithful dump of the artifact, and read that dump *once*, at compile time, rather than once per instance. Section 3.4 describes the dump; the design consequence is that improvements in a model's native ability to read a spreadsheet make the derivation step cheaper without changing the architecture, whereas they do nothing at all for an architecture that re-reads the artifact on every instance.

A separate line of work has begun to treat the intermediate representation itself as the object of study, and it is the most direct threat to any novelty claim this thesis might make about representing documents. Knowledge-centric templatic views treat a slide deck, a newsletter, a report and a poster as different templatic views of one underlying body of knowledge, and find that inserting a structured intermediate representation improves generation [30]. BabelDOC goes further for a concrete transformation class: it decouples visual layout metadata from semantic content in an explicit intermediate representation, performs document-level translation operations against the semantic side, and then re-anchors the translated content onto the original layout through an adaptive typesetting engine [31]. Proposing a document intermediate representation in order to preserve layout is therefore not novel, and this thesis does not claim it is.

The difference is what the representation is *for*. BabelDOC's representation describes the document, so that the document can be reconstructed after its content is replaced; its transformation class is fixed in advance and its semantic content is still model-generated at the time of use. The representation here describes *behaviour*: for each output field, which source value feeds it, which transformation is applied, how far a range extends, what the rendered format must be, which locations must be left untouched, and which validation obligations were discharged before the representation was allowed to execute. It is closer to a compiled program with a provenance record than to a document model, and Section 3.3 is where that distinction

is made precise. Stated negatively: if this thesis's representation were only a coordinate plus a value, the comparison with a layout intermediate representation would be unfavourable to it.

One part of the mismatch is not a comprehension problem and is worth separating, because no model improvement touches it. Writing into a native artifact through a general-purpose library can silently discard parts of the package that the library does not model. Measured on the XLSX evaluation template, a single load-and-save round trip through the spreadsheet library used here drops `xl/drawings` and `xl/printerSettings` — the drawing objects and the print configuration — and the file that results opens without complaint. That is a property of the writer, not of the reader, and it is the reason the production write path was changed to package-preserving editing (Section 3.6). A better model does not recover a part that the writer never wrote.

Status: the format characterization is implementation-backed; the demotion is a positioning decision. The structural claims about the carriers are backed by the dumps and fingerprints measured on the two real evaluation templates and reported in Table 6 — 27 content-bearing references for the XLSX template and 92 for the DOCX template — rather than on constructed fixtures. The preservation measurement is implementation-backed by the package-preserving writer and its post-condition test, which replaces the writer with a plain library save and asserts that parts are then lost. The demotion of representation mismatch to an observation is normative: this thesis runs no experiment comparing model comprehension of raw versus dumped artifacts, and Section 1.1 makes the same demotion for the same reason. The bias direction of the demotion is *unfavourable* to the method, since it gives up the easiest available motivation.

2.3 Failure Modes

This section fixes the vocabulary that the rest of the thesis argues over. Table 5 maps these six modes onto the mechanisms intended to make each one loud, and Section 3.1.5 states the creed over them. The ordering is by how hard the mode is to observe, not by how often it occurs.

1. **Wrong value, right place.** The document is structurally perfect, every field is populated, the layout is untouched, and one number is wrong. This is the thesis's primary risk measure, and it is the mode that defeats inspection, because nothing in the delivery path distinguishes such a document from a correct one. The nearest measurement in the literature is DELEGATE-52, which simulates long delegated editing workflows across 52 professional domains and reports that frontier models lose an average of 25% of document content over 20 delegated interactions, with errors characterized as sparse but severe and as silently corrupting [32]. That figure is a degradation of a domain-specific reconstruction-similarity score, not a count of corrupted documents or edits, and is cited here for the shape of the failure rather than for its magnitude.

2. **Locator drift.** A template is revised, a row is inserted, and a locator that was correct now addresses a neighbouring cell. Every value it writes is well-formed. The closest analogue in the reviewed literature is WorldGUI, which tests desktop automation from varying initial states and finds planning brittle under that variation [25]. The architectural response differs: WorldGUI's agent reasons around the variation at run time, whereas this work compiles the tolerable variation into the locator and requires the intolerable variation to fail loudly (Section 3.8.3).

3. **Coverage shortfall.** A field that no rule reaches is never written, so the template's own default survives into the delivered document. This is harder to see than a wrong value,

because there is no wrong value: there is an empty cell that looks like an empty cell. Detecting it requires knowing what the rules *obliged* the specification to produce, which requires an external denominator (Section 4.6). Across the surveyed set, none treats the coverage of an authoritative rules document as an obligation to be measured; the nearest is qualification against labelled historical cases [8], which measures agreement on the cases that exist rather than coverage of the rules that apply.

4. **Format and unit error.** The value is right and its rendering is wrong: a thousands separator in a customs field, a spreadsheet number format leaking into printed text, grams reported where kilograms were required, a rounding mode that differs from the one the rules specify. This mode is emphatically not a gap in the literature. Mind the Gap grades 200 Office tasks against 7,118 machine-gradable criteria and shows that fine-grained checking of Word, Excel and PowerPoint artifacts is entirely feasible [22], so "office artifacts admit no machine-checkable oracle" is not available as a claim.

5. **Negative-obligation violation.** A location that the rules require to remain blank is written to. The obligation is real and is stated in the rules of the XLSX dataset in four places — unused detail slots must stay empty, the remarks column must be left blank unless a condition holds, prior-period figures must not be carried forward, and the signature block must not be pre-filled — and it is structurally invisible to any check that compares only the values that were written. Nothing in the surveyed literature treats "I am responsible for this location and I discharge that responsibility by writing nothing" as an expressible obligation.

6. **Extent error.** An aggregate covers one row too many or too few. The result is a number with a plausible derivation and a wrong value. This mode supplies the strongest single

argument in the thesis for oracles over inspection: of the six wrong-extent hypotheses instantiated on the DOCX dataset, four produce six identical totals, because the rows they wrongly include carry an empty quantity column. The only observable difference is that one more detail slot has been consumed — which no total, and no reconciliation of totals, can see.

Three of the six are not locator problems at all, which is the reason Chapter 3 does not present locators as the whole method.

Status: five of six modes have a measured instance in this repository; two gap claims are scoped to the surveyed set. Mode 1 has two measured specimens: three of 79 detail rows in the DOCX dataset changed value when decimal rounding replaced binary rounding, one of them inside an evaluation instance; and an offline execution of a real specification over five XLSX instances scored 347 of 350 semantic checks with three failures classified `silent_failure`. Mode 2 is measured as fingerprint sensitivity on both real templates (Table 6). Mode 3 is measured as the coverage denominator over the two rule documents. Mode 5 has a measured instance in the same execution probe, where the three failures were the remarks column. Mode 6 is measured as the six wrong-extent hypotheses in the dataset's own annotation. Mode 4 is *not* independently measured: only the rounding function has been audited for decimal semantics, and the rest of the format channel is asserted from the scoring policy rather than demonstrated. The assertions that modes 3 and 5 have no analogue are checked against the 62 surveyed works only, and are stated as such rather than as claims about the literature at large.

2.4 Governance Requirements

The governance argument of this thesis is stated in exactly three words, and the reason for that restraint is a negative result of the survey. A tempting formulation — that regulated or audited workflows *require* deterministic output — was looked for and not found: no academic source in the surveyed set supports a generalization of that shape, and this thesis therefore does not make it. Section 3.8.4 states the same restriction from the implementation side. What is claimed instead is that such workflows value three properties:

- **Traceability.** For any delivered value, one can say which rule produced it and which source datum it came from.
- **Reproducibility.** The same inputs yield the same artifact, bit for bit, on a later date and a different machine.
- **Change control.** A change in behaviour is a reviewable diff against a previous, immutable version.

That these three have operational value is supported by industrial rather than regulatory evidence, and the strongest available witness is also this thesis's closest neighbour. In the deployment reported by Kujanpää et al., every tool invocation logs the tool version, its inputs and its outputs; a monitoring agent reviews those logs in batch; and the authors report that versioning improves auditability and exposes both specification gaps and upstream data drift [8]. That is evidence for the value of versioning in production, and it is explicitly not a regulatory basis.

The structural difficulty for a per-instance agent is that it supplies none of the three, and not because it is careless. There is no versioned program to trace, since the behaviour existed only inside one inference; no compiled artifact to hash, since the next run may legitimately

differ; and no diff to review, since there is nothing that persists between runs to diff against. A transcript is a record of what happened, which is a weaker object than a specification of what will happen. This is the sense in which the governance argument is architectural rather than a criticism of model quality, and it is unaffected by improvements in model quality.

Status: the requirement is normative in three words; the architecture's provision of them is implementation-backed; the comparison is unmeasured. No claim about standards or regulation is made anywhere in this thesis, and no such source is cited. That this architecture supplies the three properties is implementation-backed: published versions are immutable at the model layer, the forceable set and gate ordering are a pure function tested in isolation, forced publications are recorded in a persisted audit trail, and the structural fingerprint is measured on the real templates. That a per-instance agent supplies none of them is an argument from the absence of a persisted artifact, not a measurement — no experiment in this thesis attempts to extract traceability from an agent transcript — and the bias direction is favourable to this thesis, since a determined engineer could log a great deal around an agent. What such logging cannot produce is the object the other two properties need, which is a program that will behave the same way next time.

2.5 LLMs for Document and Engineering-Drawing Understanding

Document intelligence has historically been a read-side field: given an existing artifact, produce a semantic representation of it. LayoutLM [27], Donut [28] and DocLLM [29] are representative, and they share a property that matters here — the input document is not modified, so no obligation exists concerning the parts of it that were not the target. The write side imposes

exactly that obligation, and until recently it would have been fair to say the field did not address it. It is no longer fair. DocEdit-v2 grounds a user's edit request in a region of the document and then executes textual, visual and layout edits on it [33], which makes localized document mutation an explicit research problem rather than an unexamined one. The difference this thesis can defend is not that nobody writes, but that DocEdit-v2 re-localizes per request, whereas a locator here is a persistent part of the specification that must remain stable across instances or fail loudly when it does not.

The spreadsheet line has moved further, and it is worth being precise about how far, because the temptation to describe these systems as unvalidated is strong and would be wrong. Sheet-Copilot abstracts spreadsheet functionality into atomic actions, plans over them with a state machine, and builds an execution-based evaluation pipeline [23]. TableLLM trains for office table manipulation and uses cross-way validation to improve automatically generated training data [34]. SheetMind constrains its action agent's output with a grammar and adds a reflection agent that checks actions against the original intent [35]. NL2Formula generates an executable spreadsheet formula from a natural-language query [36], which makes its output the closest thing in this literature to a persistent deterministic artifact — a small program the spreadsheet engine will re-evaluate without any model. And SpreadsheetBench builds multiple workbook files per instruction, so that a solution is scored on whether it generalizes rather than on whether it fits one workbook [21].

So the gap here is architectural, not evidential. Validation exists; structured actions exist; executable artifacts exist; multi-file robustness evaluation exists. What differs is where the inference sits and what the cases are used for. NL2Formula's artifact is one formula, not a specification covering every target region, its formats, its merged cells, its cross-sheet references and the requirement that non-target content be untouched. SpreadsheetBench's cases score a

generated solution; here, cases decide whether a specification may be promoted to a deployment artifact, after which instances are produced with no model at all. Both differences are real and both are narrow, and stating them narrowly is the point of this section.

For engineering drawings the survey found no comparable line of work: nothing in the surveyed set addresses writing into a drawing's entity graph under a preservation obligation. That absence is not used as an argument here, for two reasons. It is an absence in a set of 64 works chosen for the office carriers, so it is weak evidence about the drawing literature; and the drawing carrier is in any case reported only as a case study in this thesis, because the entire system was originally built against a drawing template and any figure it produces is contaminated by construction (Section 5.9).

Status: the survey of this literature is citation-backed; the architectural difference is implementation-backed; the drawing claim is explicitly weak. Every characterization above is taken from the fetched abstract of the work in question, and the venue of each follows DBLP rather than the survey draft — seven of the works named in this section were among the 26 whose venue the draft recorded incorrectly. That this thesis's artifact is a specification rather than a single formula, and that cases here act as a promotion criterion rather than as a score, is implementation-backed by the specification schema and the publish gate. What is *not* available is any comparative measurement against any system named in this section: no experiment in this thesis runs SheetCopilot, SheetMind or NL2Formula, the baselines are the three arms defined in Section 5.3, and no claim of superiority over these particular systems is made or implied. The drawing-literature absence is labelled unchecked beyond the surveyed set.

2.6 Document Automation, RPA, and Template Filling

This is where the positioning sentence of Section 1.3.3 is developed against the literature, and where the citation that section deliberately withheld is supplied.

Robotic process automation is the deterministic-but-manual corner. The systematic review by Wewerka and Reichert covers 63 studies and defines the field's object as the automation of rule-based routine processes for cost and efficiency [37]. Its execution model is exactly what this thesis wants at deployment: a program, repeatable, versionable, auditable, with no model anywhere near it. Its cost is equally clear, and it is not non-determinism — it is the derivation bottleneck. A person must formalize the domain knowledge and the artifact's structure into a workflow, cell address by cell address, and a person must redo that work when the form changes.

The agent corner is automatic and stochastic. OSWorld provides 369 real computer tasks with execution-based evaluation scripts across three operating systems [24]; WorldGUI stresses planning from varying initial states over real desktop applications [25]; and Mind the Gap measures Office proficiency specifically, finding single-turn frontier models at a maximum score rate of 36.6% and an agentic system with execution feedback, iterative repair and broader automation access at 68.8%, against a community-reference score of 95.5% [22]. In all three, each task instance is perceived, planned and executed afresh; the recurrence of a template family is not exploited, because these benchmarks are not about recurrence.

Between the two corners sits a bridging community that this thesis must not pretend away. ProAgent has language-model agents *construct* workflows from human instruction and also make dynamic decisions during execution [38]; Prompt2Task generates and then executes smartphone automation from textual prompts, incorporating user feedback into later automations [39]. Automatically deriving automation logic from natural language is therefore not this the-

sis's distinguishing move, and any framing that presents "RPA versus agents" as an exhaustive dichotomy is obsolete.

The fork is narrower and sharper than that dichotomy. ProAgent gives both construction and execution to the model. This work gives construction to the model and takes execution back, and — this is the part that is not merely a preference — it makes an executable test decide whether the boundary between the two phases may be crossed at all. A specification that fails a structural gate, or whose semantic oracle disagrees with an independently produced expected document, is not published, and no instance is produced from it. The three-way statement of Section 1.3.3 thus reads, with citations: robotic process automation is deterministic but manual [37]; per-instance agents are automatic but stochastic [24, 22]; the bridging systems automate the authoring but keep the model at execution [38, 39]; and this work automates the authoring and removes the model from execution, with an executable admission test at the boundary.

The fourth clause is not, however, a description of an unoccupied position, and Table 3 is where that became visible. Automatic authoring followed by deterministic execution is also what programming by example does, and has done since 2011 [1], in a shipping spreadsheet product [2]. What distinguishes this work inside that corner is not that it is alone there but what Section 2.7 sets out: the site of each transformation is not given, the coverage of a rules document is an obligation, and the regions that must not be written are part of the specification. The corner is shared; the admission test at its boundary is what this thesis adds to it.

Status: the taxonomy is normative; each cell is now citation-backed; the claimed advantage is unmeasured. No experiment in this thesis varies the automation paradigm, so the three-way division remains a positioning argument, exactly as Section 1.3.3 states; the two Status paragraphs must be read as one claim. What this section adds is that each cell of the division now rests on a cited work rather than on assertion, which closes the gap that Section 1.3.3 ex-

plicitly left open. It also subtracts something: the division is three-way over *these* traditions and is not a partition of the design space, since programming by example sits in the same corner as this work and was omitted from the original formulation, as Table 3 shows. That the admission test exists and rejects specifications is implementation-backed — it has rejected real derived specifications, including one whose totals had no source and one whose repeat block was offset by a row. That the resulting arrangement is *better* than either corner is not measured here: no comparative correctness, cost or drift figure against a robotic-process-automation baseline or an agent baseline exists in this thesis, and Section 5.3 defines the arms that would produce one.

2.7 Programming by Example and Program Synthesis

The intellectual ancestry of this work is programming by example, and the chapter is stronger for admitting it early rather than defending against it late.

FlashFill synthesizes a string-transformation expression from spreadsheet input-output examples, ranks candidate programs, detects noisy examples and asks for disambiguating ones [1]. Its methodology is this thesis's methodology one level down: derivation may be expensive and interactive, the result is a deterministic executable program, and that program is then applied repeatedly. FlashMeta generalizes the construction of such synthesizers into a framework [40]; RobustFill shows that example-to-program synthesis survives noisy examples without any language model [41]; and FlashFill++ scales the approach in expressiveness, performance and program quality in a production spreadsheet context [2]. Two convenient distinctions are therefore unavailable. Neural Program Search already combines a natural-language description with a handful of examples [42], so "prior example-driven synthesis has no natural language" is false; and FlashFill++ is neither small nor toy, so "programming by example only handles simple single-column string edits" is false as an absolute statement.

The modern half of this literature is equally relevant and is where the survey draft had a hole. Li and Ellis ask directly whether large language models have solved programming by example, and find pretrained models ineffective but fine-tuned models strong when test problems are in distribution [43]. Compositional approaches recover from failure by decomposing the example-driven task into subtasks rather than re-prompting on the same task [44]. CodeARC makes the setting interactive, letting an agent query a hidden target function and refine against a differential-testing oracle over 1,114 functions, with the best of 18 models reaching 52.7% [45]. An evaluation of iterative example-based code generation finds that describing requirements with examples rather than natural language costs over 60% of the score and that over 95% of successes occur in the first round [46]. And a hybrid that calls a classical solver first and falls back to a model handles tabular transformations better than either alone [47]. A related idea appears in code rewriting: synthesize the transform from a few examples rather than performing the rewrite, because a synthesized transform is inspectable, debuggable and cheap to run [48].

The difference that survives all of this is the *unknown binding problem*. A programming-by-example system is normally handed the site of the transformation: here is the input column, here is the desired output column, learn the function; the domain-specific language and the scope are fixed in advance. This work is handed an entire template, an entire rules document, and a few source-and-artifact pairs, and must infer which transformations exist at all, which structural location in a heterogeneous native artifact each one writes to, how far a range extends, and which locations must not be written. Locator inference, rule coverage and preservation are therefore not post-processing around a synthesis problem; they are part of it.

Two concrete cases in this thesis do not reduce to a per-column example-driven transformation, and it is worth naming them precisely because a vaguer claim here would be the easiest

thing in the chapter to refute. The first is the detail table of the DOCX dataset, where one column's value requires a join to a product master while another requires a join to a shipment manifest, seven columns are computed across fourteen slots, and six totals aggregate over a row set that must be identical in two separate output documents — so a per-column program that happened to agree would be a coincidence rather than a guarantee. The second is the negative obligations of the XLSX dataset, where the correct behaviour at four locations is to write nothing, which no input-output example exhibits, because an example of writing nothing is indistinguishable from an example that was never generated.

Status: the lineage is citation-backed; the difference is implementation-backed; the irreducibility claim is an argument over two datasets. The ancestry claims and the three unavailable distinctions are taken from the fetched abstracts of the works cited. That locator inference, extent semantics and negative obligations are part of the synthesis problem here is implementation-backed by the specification's locator forms (Section 3.5) and the derived implied-blank mechanism, both of which exist and are exercised on real files. The claim that the two named cases cannot be reduced to per-column example-driven synthesis is an argument, not a proof: it is made over two datasets that exist in this repository, and a sufficiently expressive domain-specific language with a join operator and an explicit blank action could in principle express them. What the claim does establish is the weaker and sufficient point that they are not expressible in the setting those systems actually address. The bias direction is favourable to this thesis and is disclosed as such in Section 5.10.

2.8 Tool-Making, Skill Libraries, and Amortized LLM Computation

This is the most dangerous neighbourhood for this thesis, and one work in it is the most dangerous single paper. This section is where the forward reference of Section 1.4.1 lands.

The cost intuition behind compile-once, execute-many is not new and must not be claimed as new. Large Language Models as Tool Makers has an expensive model construct a reusable tool once, caches it, and has a cheaper model use it repeatedly, amortizing the one-off cost across instances [3]. Voyager stores successful behaviours as executable skills in a retrievable library, using environment feedback, execution errors and self-verification while generating them [4]. TroVE induces a compact reusable toolbox while solving tasks and takes *verifiability* as an explicit objective, measuring the human time needed to verify the result [5]; CRAFT generates code solutions on training examples, validates their correctness, and only then abstracts them into deduplicated reusable snippets [6]; and ReGAL refactors existing programs into shared abstractions, verifying through execution that the abstraction does not change program output [7]. Four claims are therefore unavailable to this thesis: that amortization is novel, that reusable-artifact literature lacks verification, that validation before reuse is novel, and that regression thinking is absent.

The paper that removes the fifth is Kujanpää et al., which observes the same problem this thesis observes — that production agents regenerate code for the same procedural steps on every request, wasting latency and reliability — and responds by compiling recurring standard-operating-procedure steps into validated, versioned tools *before* deployment [8]. The tool maker grounds synthesis in the live environment, collects execution traces, observes backend schemas and values, generates candidate tools, and repairs them against labelled cases. Deployed on a

fulfilment-centre alarm-triage system whose agent diagnoses alarms against a 44-node procedure, tool calls reduce median latency by 42% and reduce end-to-end error rate by up to 53% on 1,500 historical alarms, explicitly by suppressing run-to-run variance in repeated steps; versioned tools are used for auditability and for surfacing specification gaps and upstream data drift. Labelled-case repair, versioning, audit logging and drift monitoring are all present. This is simultaneously the strongest threat to this thesis's originality and its best witness, since it is industrial evidence that moving repeated procedural inference from run time to deployment time genuinely reduces error, variance and latency.

Four differences remain, and only the first carries real weight:

1. **The model stays on the execution path.** At run time the production agent calls the compiled tools directly *and falls back to code generation when needed* [8]. A fallback is not a small residue of the same design. It is the difference between a guarantee and a tendency, because the properties one wants from determinism — reproducibility, a reviewable diff, a bounded set of behaviours — are exactly the properties a fallback suspends, and it suspends them on precisely the unusual inputs where they were wanted most. This thesis's boundary admits no fallback at all (Section 3.7).

2. **The unit of compilation.** What is compiled there is a node-level procedural tool within a larger agent-driven flow. What is compiled here is the whole template-instantiation mapping, so that no orchestration decision remains for a model to make.

3. **The document contract is absent.** Nothing in that deployment must locate a write target inside a native office artifact, keep that location stable across template revisions, or leave every non-target region byte-identical.

4. **Cases qualify tools; rules do not.** Tools are qualified against labelled historical cases,

which is a different obligation from asking whether each normative rule in an authoritative document is implemented by some clause of the specification.

The gap statement of this section is therefore deliberately narrower than the one this thesis began with. It is not that tool-making systems store an artifact and stop; they validate, refactor, version and monitor it. It is that their reusable artifact is a function or skill embedded inside a model-driven run time, that none of them treats a rules document as a specification whose obligations must be covered, and that none compiles an entire template-instantiation task — locators and preservation constraints included — into a regression-gated artifact that executes with no model inference whatsoever.

Status: the gap is citation-backed and was narrowed under pressure from one paper; the zero-inference property is implementation-backed; the comparative advantage is unmeasured. The narrowing is itself a result of this survey and is recorded as such: the earlier draft of this gap asserted that tool-making lacks executable oracles, coverage and version governance, and Kujanpää et al. refutes all three, so the assertion was withdrawn rather than softened. The four differences above are read from the fetched abstract and from verbatim sentences copied out of the full text into `docs/thesis/refs/abs/b1-toolmaking.md`, including the fallback sentence on which difference 1 rests. That this system performs no model inference at execution is implementation-backed in a specific and deliberately falsifiable way: a test replaces both repair entry points with recorders that fail if called, runs the complete production path, and asserts both that no recorder fired and that the recorded model-call count is zero. Reading the source would show only that no call site exists today; that test turns red if one is added later. What is *not* measured is any comparison against Kujanpää et al. or against a fallback-bearing design — no experiment here quantifies what a fallback costs, and the argument in difference 1 is analytical. The bias direction is favourable to this thesis, and Section 5.10 lists it.

2.9 Execution-Feedback Program Repair

The repair loop of Chapter 4 descends from a well-developed line of work, and the gap this thesis can defend within that line is not the one it originally claimed.

Reflexion converts environment, compiler and task-score feedback into verbal reflections held in episodic memory and retried, without updating weights [9]. Self-Debug has a model inspect execution results and explain its own code to find its own errors, and — importantly for this thesis — shows improvement on a task where no unit tests exist, using code explanation alone [10]. So the claim that execution-feedback methods presuppose ready-made unit tests is false, and was withdrawn. LDB segments a generated program into basic blocks and inspects intermediate variables after each, moving the question from "did it crash" to "does the execution state match the intent" [11]. AlphaCodium arranges specification analysis, test reasoning, an initial solution, test execution and iterative repair into a flow [12], which is structurally the same loop used here.

Two more recent works close off the remaining easy claim. A study of self-debugging with self-generated tests examines what happens when high-quality tests do not exist, has the model generate its own, and identifies self-generated test bias as a real effect [14]. UTGen and UTDebug go further and learn to generate error-revealing inputs *together with* expected outputs, using multiple generated tests, test-time scaling and backtracking to limit the damage from a noisy oracle [13]. Automated oracle construction is therefore an active research problem with real results, not an unoccupied space.

What remains is not feedback availability but **oracle construction and independence**, and the reason it remains is specific to this failure mode. When the failure is a crash, any oracle that notices the crash suffices. When the failure is a syntactically valid document containing one

wrong number, the oracle must know what the right number was — and if the expected output is produced by the same generative process that produced the candidate, the same misconception is encoded into the test, so the test passes and the document is wrong. The response in this thesis is a red line rather than a technique: the expected artifact for every case is computed from the rules by a path that does not include the system under test, and the derivation-time oracle compares what the specification would write against that artifact (Section 4.3.1). The independence is structural, and it is the reason the oracle can catch a wrong value at all.

The mechanism that this section must not oversell is the case-level oracle. It exists, it is tested, and it carries four anti-overfitting invariants, but on the automatic derivation path it has no caller: derivation closes on the structural gates and the derivation-time oracle, and case-level repair is reachable only from an offline command-line entry point and from tests. Section 4.3.2 states this in the same words, and no measurement anywhere in this thesis is attributed to it.

Status: the gap is citation-backed; oracle independence is implementation-backed at the dataset level; one named mechanism has no evidence and is disclosed. The three withdrawn claims — that execution feedback needs pre-existing tests, that oracle generation is unexplored, and that iterative repair is novel — are withdrawn on the strength of the cited works, each read from its fetched abstract. Oracle independence is implementation-backed by the dataset construction rule that expected artifacts are never produced by the system under test, and by the derivation-time oracle's mismatch code, which has fired on real derived specifications and has a negative control that turns a passing comparison into a failing one by inverting a single arithmetic operator. The case-level repair mechanism is handled under the third route of this thesis's gap policy: it is described as designed and tested, its absence from the automatic path is stated, integration is future work, and Chapter 5 attributes nothing to it. The bias direction of omitting it is unfavourable to this thesis, since a working case-level loop would

strengthen the derivation argument.

2.10 Verification and Guardrails for LLM Output

This section does not argue a gap. Its function is to ground a design decision that would otherwise look like a preference, namely that the acceptance authority in this system is an executable predicate and never a model's judgement.

Two traditions supply the positive grounding. Constrained decoding rejects token sequences that a parser can already prove ill-formed: PICARD attaches an incremental parser to text-to-SQL decoding and makes lexically, grammatically or schema-invalid output difficult to generate at all [49]. The lesson transfers directly — whatever a grammar or schema can decide should never be delegated to a judge — and so does its limit, because a valid SQL query can still return the wrong rows, exactly as a valid `.xlsx` can still contain the wrong amount. Structural validity and semantic correctness must be separate channels, which is why they are separate channels in Section 5.5.1. The second tradition addresses what to do when the exact expected output is hard to enumerate. QuickCheck has the programmer declare properties that should hold and generates inputs to test them, rather than writing an expected output per input [50]; metamorphic testing attacks the test- oracle problem by specifying relations that must hold between the outputs of related inputs [51]. Several such relations are natural here — scaling an amount, shifting a date, permuting source rows, and the invariance of every non-target region — and none requires a model.

The negative grounding concerns judges. A systematic robustness assessment of language-model judges finds them susceptible under both pointwise and pairwise protocols, with robustness highly sensitive to the prompt template and to the choice of judge model, and with no defence dominating across robustness, utility and cost [52]. Self-preference bias is measured

directly: a strong model prefers its own outputs, and models favour text of lower perplexity to themselves more than humans do [53]. And JudgeBench constructs response pairs with objectively decidable correctness and finds that strong judges, including a frontier model, perform only slightly better than random guessing on the hard pairs [54].

Care is needed in how much is drawn from the middle result. What is demonstrated is a systematic preference dependence, not a theorem that every generator error correlates with a judge error; this thesis states the weaker claim, which is sufficient for its purpose. Combined with JudgeBench, the position is: when an independent executable criterion for correctness exists, substituting a model's preference for it is a methodological error, not a convenience. For template instantiation — where a visually flawless document may contain one consequential wrong value — the acceptance authority is therefore an executable structural, semantic or preservation predicate. Section 4.3.3 states the same rule for derivation, and Section 5.5 enforces it for measurement.

Status: the grounding is citation-backed; the no-judge property is implementation-backed; the strong reading is explicitly not claimed. That no model is consulted at acceptance or at scoring is implementation-backed: the shared evaluator is a deterministic command-line program with a frozen scoring policy, it can lint and self-test itself, and it reports a version and a source hash with every score. The claim that a judge *cannot* substitute for an oracle is not made; what is claimed is the narrower statement that where an executable criterion exists it should be preferred, supported by the three cited results. The stronger reading of self-preference bias — that generator and judge errors are necessarily correlated — is available in the rhetoric of this area and is deliberately not asserted, because the cited work does not establish it. Two of this section's grounds predate language models entirely, which is a point in their favour rather than against.

2.11 Cost-Aware Agent Evaluation

The final gap is the one that changed most under scrutiny, and it changed in the direction of narrowing.

The claim that agent evaluation reports only accuracy was true recently and is not true now. Cost-of-Pass defines the expected monetary cost of obtaining a correct answer and finds that the marginal accuracy gained from majority voting or self-refinement frequently fails to justify its cost [15]. An infrastructure-level study measures the latency, resource use and energy of agentic multi-turn reasoning and reports diminishing returns from additional inference compute alongside growing latency variance [16]. τ-bench compares final database state rather than surface text and introduces pass^k to measure whether an agent succeeds consistently across repeated runs [17]. MAESTRO runs 12 multi-agent systems under controlled repetition and finds executions that are structurally stable yet temporally variable, with substantial run-to-run variance, and reports that the system's architecture dominates its reproducibility and its cost-latency-accuracy trade-off more than the backend model does [18]. And a reliability study decomposes agent behaviour into twelve metrics across consistency, robustness, predictability and safety, evaluates 15 models, and finds that recent capability gains have yielded only small reliability gains [19].

These results do more than close a gap; they supply the precise form of this thesis's answer to the strongest available objection, which is that a better model will make per-instance execution reliable enough. The argument has four layers, and the sentence it must not collapse into is "no future model can make per-instance agents reliable", which is unprovable. Layer one: capability and reliability are different axes, and improvement on the first has bought little on the second [19]. Layer two: run-to-run variance is measurable and is driven substantially by archi-

tecture rather than by the backend model [17, 18], so replacing the model does not address it. Layer three: in document workflows the errors are silent corruption rather than visible failure, they are sparse but severe, and agentic tool use did not remove them [32]. Layer four: every instance carries a recurring inference cost whose marginal value is often negative [15, 16]. The correct conclusion is architectural rather than pessimistic: improving a model lowers the probability of an inference error, but it does not change the fact that per-instance execution incurs stochastic inference N times, and a recurring task that can instead be compiled into a validated deterministic program eliminates those execution-time properties by construction rather than improving them probabilistically.

What remains missing is an **amortization-aware evaluation for a recurring template family**. The quantity a deployment decision needs is not the cost per successful agent run; it is a one-time derivation cost — including every failed attempt and every repair — plus a near-constant deterministic execution cost across N instances, together with the break-even instance count against per-instance execution, and the cost of detecting and repairing drift when the template changes. Section 5.4 defines those quantities, and Section 5.4.5 defines the asymmetric rerun budget that keeps the variance comparison fair rather than flattering.

Honesty requires one more thing of this section. The variance that this thesis removes is execution-time variance, and it did not remove variance from the method as a whole; it moved it into derivation, where it has been measured. Two consecutive derivations over identical prompts and identical payloads produced specifications differing in slot count, in the number of repeat blocks, in rule coverage and in hard-error count; four such derivations exist in all, and Section 5.4.5 is the single place that reports their spread, so that this chapter and Chapter 5 do not end up quoting two sample sizes for one quantity. That measurement is this thesis's own, it is reported rather than omitted, and it is the reason research question two asks about consistency

separately for the two phases. Section 2.13 supplies the vocabulary in which such a spread should be stated, which this section deliberately does not attempt in passing.

Status: the gap is citation-backed and narrowed; execution determinism is implementation-backed; every comparative cost figure is unmeasured. The withdrawal of "agent evaluation reports only accuracy" rests on the five works cited, all five of which postdate the framing it replaces. Execution-time determinism is implementation-backed by the zero-inference production test described in Section 2.8, and derivation-time variance is implementation-backed by the two-derivation comparison just described, both of them figures from this repository rather than estimates. Everything comparative is missing, and the list is short enough to state in full: no per-instance baseline cost, no break-even instance count, no comparative correctness figure, and no measurement of baseline behaviour under template drift. The monetary column in the evidence store is zero throughout, because the derivations were run under a subscription rather than metered billing, and whether the cost table will finally be reported in tokens or in currency is undecided and must be fixed before the formal experiment. Section 5.10 carries the corresponding threats; the bias direction of an unmeasured break-even point is favourable to this thesis, since a break-even count large enough would undermine the amortization argument entirely.

2.12 The Families Side by Side

The preceding eleven sections each argue one relationship, which is the right shape for an argument and the wrong shape for a reader who wants to know where this work sits. The question a reviewer actually asks — given a recurring task of this kind, what are the available architectures, and on which axis does this one differ — is not answered anywhere above, be-

cause the answer is distributed across eleven separate discussions. This section puts the families in one coordinate system.

Four coordinates are used, chosen because each is decidable from a work's own description rather than being a judgement about quality: whether a model is invoked at execution time, what the approach guarantees, how the marginal cost grows with the instance count N, and what happens when the output is wrong. The fourth is this thesis's headline measure (Section 5.5.1), which makes the table a precondition for Section 5.10 as much as a summary of this chapter.

The blanks are the most useful part of the table, and they cluster in one column. Four families have no entry in the last column and a fifth has only half of one. That column asks what a reader most needs to know before putting an architecture on the critical path of a customs declaration, and for most of these families the surveyed material simply does not answer it. The blanks are marked rather than filled, because filling them from general impression is precisely the failure this table is most exposed to. Two qualifications keep them honest. First, *not stated* means the fetched abstract does not settle the question; the two works whose full text was retrieved and read for this column are Kujanpää et al. and Mind the Gap, so the blanks elsewhere rest on a weaker check than the entries do. Second, a blank is not a deficiency claim about the family: a system's failure shape can be well understood by its authors and simply not be the subject of the paper that this survey fetched.

There are two dangerous neighbours, not one, and they are dangerous in different columns. Reading the table by column rather than by row makes visible something that eleven sections of prose could not. Program-aided language models are the dangerous neighbour in the second column: PAL takes the arithmetic away from the model and gives it to an interpreter [56], and PoT states the same separation as its title [57], which is the same instinct this thesis has about

Table 3: Where each family puts the model, and what follows from that. Every non-empty cell is read from a work whose abstract or full text was fetched for this survey; *not stated* marks a cell the surveyed material does not settle, and is not a claim that the question is unanswered elsewhere.

Family	**Model at execution**	**What is guaranteed**	**Marginal cost in N**	**When it is wrong**
Per-instance agent (Arm A)	Every instance	Nothing beyond what an execution-based grader checks afterwards [24, 22]	$O(N)$ inferences	Silent corruption of documents, sparse but severe [32]
Self-consistency, majority voting	Every instance, k samples	Agreement among the k samples; nothing external to them [55]	$O(kN)$ inferences; the marginal accuracy often fails to justify the cost [15]	*not stated*
Constrained or structured decoding	Every instance (the constraint acts during decoding)	Output is well formed for the grammar or schema [49]	$O(N)$ inferences	Ill-formed output is refused as it is decoded; a well-formed output carrying a wrong value is not (Section 2.10)
LLM-as-judge	Every instance, plus a second inference to score it	Nothing decidable: strong judges are close to random on objectively hard pairs [54]	$O(N)$ generation plus $O(N)$ judging	Accepts silently; robustness swings with the prompt template and the judge model [52]
Program-aided LM (PAL, PoT)	Every instance: the model emits the program, an interpreter runs it [56, 57]	The arithmetic is the interpreter's rather than the model's [56]	$O(N)$ inferences plus $O(N)$ short program executions	*not stated*
Tool making, skill libraries (Arm B)	Every instance: the tool user is a model, with code generation as fallback [8]	Tools are validated before reuse [5, 6] and qualified against labelled historical cases [8]	$O(1)$ tool construction plus $O(N)$ agent inference [3]	End-to-end error rate reduced by up to 53%, not removed [8]; the shape of the residue is *not stated*
Agentic workflow construction	Every instance: the model builds the workflow *and* decides during execution [38]	*not stated* as a decidable property in either work	$O(1)$ construction plus $O(N)$ inference	*not stated*
Programming by example	**None**: a synthesized program runs [1]	Consistency with the supplied examples; candidates are ranked and disambiguating examples are requested [1]	$O(1)$ synthesis plus $O(N)$ deterministic executions	A wrongly generalized program writes a wrong value without complaint; strong in distribution and weak out of it [43]
RPA, template-filling tooling	**None**	That the program does what a person authored [37]	$O(1)$ *human* authoring plus $O(N)$ deterministic executions	*not stated*: the systematic review is an assessment framework, not a failure taxonomy [37]
This work	**None**, pinned by a negative control that fails if a call site is added (Section 2.8)	Structural gates, a semantic oracle against an independently produced expected artifact, and coverage of a rules document, all discharged before publication (Chapter 4)	$O(1)$ derivation including every repair, plus $O(N)$ deterministic executions	Silent failure is a scored outcome with its own denominator; observed at 3 of 350 semantic checks on a real specification (Section 5.8.1)

where computation belongs. They are not neighbours in the third column, because the program is regenerated for every question, so the inference count is still $O(N)$ and nothing persists to be reviewed, hashed or diffed. Programming by example is the opposite case. It is the dangerous neighbour in the *first and third* columns — no model runs at execution, and a one-time synthesis amortizes over N deterministic executions, which is exactly this thesis's cost shape and has been that shape since 2011 [1] — while differing in the second, for the reason Section 2.7 gives at length: a programming-by-example system is handed the site of the transformation, and this one has to infer where in a native artifact each value goes and which locations must be left alone.

A claim this table withdraws. An earlier formulation of the positioning, in Section 1.3.3 and at the close of Section 2.6, described the corner in which authoring is automatic and execution is deterministic as empty. Laid out in columns, it is not: programming by example has occupied it for over a decade, and FlashFill++ occupies it in a shipping spreadsheet product [2]. Both sections have been corrected to say so. What survives is narrower and is the thing this thesis actually defends — not that the corner is empty, but that no occupant of it addresses the unknown-binding problem, the coverage of a rules document, or the obligation to leave a native artifact's non-target regions untouched. Nothing in the argument depended on the corner being empty; the sentence was simply stronger than the evidence, in the direction that flattered this thesis, which is the direction Chapter 1 has already been corrected in once.

Status: the table is citation-backed cell by cell; the blanks are scoped to the surveyed material; the arrangement is normative. Every non-empty cell above is read from a work recorded in `docs/thesis/refs/manifest.md` with its fetched metadata and its DBLP venue, and the per-work verbatim abstracts are in `docs/thesis/refs/abs/`. The two

families added for this table — program-aided language models and, in the next section, uncertainty quantification — were absent from the survey draft entirely, so their eight works were searched, fetched and reconciled from scratch rather than transcribed; one of the eight was found to carry a different published title than the one claimed for it, and the bibliography follows the published one. That this particular set of four coordinates is the right one is a choice of this thesis and not a finding: a different reader could reasonably want a column for latency, for human effort, or for how much of the task each family covers, and none of those is here. No cell states a comparative result, because none exists: no experiment in this thesis runs any system named in the table, and the only rows with measured numbers are the ones citing measurements those authors reported or measurements taken in this repository.

2.13 Quantifying the Uncertainty of One Derivation

The previous section's second column asks what an approach guarantees, and for every family whose execution routes through a model the honest answer is a probability rather than a property. There is a mature vocabulary for stating such answers precisely, this thesis has so far used none of it, and that omission is worth repairing carefully, because the standard measures turn out to be unable to say anything about this method at execution time — which is the point rather than an inconvenience.

What the field measures, and at which layer. Five layers can be distinguished, and which layer a measure operates at determines what it can see. At the *token and sequence* layer, uncertainty is read from the model's own output distribution; Malinin and Gales set out token-level and sequence-level estimators for autoregressive structured prediction within one ensemble-based framework and give baselines for token- and sequence-level error detection [58]. At the

semantic layer, the object of measurement is meaning rather than surface form, because different sentences can mean the same thing: semantic entropy clusters generations by meaning before computing an entropy over them, and is more predictive of accuracy than surface-level baselines [59], a line strong enough to have been developed into a hallucination detector reported in a general-science venue [60]. At the *sampling* layer, dispersion over repeated samples of the same problem becomes the signal: self-consistency samples several reasoning paths and marginalizes over them [55], and τ-bench's pass^k asks whether an agent succeeds on all k of its runs rather than on one [17]. At the *calibration* layer, the question is whether stated confidence tracks correctness, and verbalized confidences emitted as output tokens turn out to be better calibrated than the model's own conditional probabilities for models fine-tuned with human feedback [61]. And at the *distribution-free* layer, conformal methods wrap the generator: conformal language modelling calibrates a stopping rule for sampling outputs until the returned set provably contains an acceptable answer with high probability [62].

Every one of them becomes silent at this thesis's execution step, and that is the argument. Four of the five layers require access to a model's output distribution or to repeated samples of the same input, and the fifth requires a stated confidence. At execution time this method offers none of these, for the reason that is its entire claim: there is no model, so there is no distribution to take an entropy over, no set of samples to disagree, and nobody to ask for a confidence. Applied to the execution step, every measure above returns the same reading it would return for a hand-written program. That is a strong statement and it must be stated in the direction that costs something: *these instruments cannot distinguish this method from a hand-written program at execution time, and they equally cannot certify that either one is correct.* A dispersion of zero is a fact about repeatability, not about truth — a specification that is deterministically wrong is deterministically wrong every time, and Section 5.5.1 scores that outcome under its own name

rather than letting a consistency figure absorb it.

The question therefore has to be restated rather than answered in the field's usual terms. The quantity of interest is not how uncertain each of N outputs is; it is how uncertain the *one derivation* was. Written as a factorization, the total dispersion of a run of N instances is a derivation-time term multiplied by an execution-time term that is zero by construction, against a baseline whose per-instance term is nonzero and is incurred N times. The consequence for measurement is that the sampling layer is the only one of the five that transfers, and it transfers to a different object: instead of k samples of one answer, the unit becomes D independent derivations of one template, compared as a distribution over specifications. Section 5.4.5 defines that measurement, its isolation conditions and the table it will fill; Section 5.8.1 argues separately about what the relocation bought and where the bargain fails. Neither is restated here.

What is already in hand, what is missing, and what none of it reaches. Two dispersion measurements exist and both are reported elsewhere in this thesis rather than introduced here: three runs of a frontier agent over one instance produced three distinct file digests at 69.2% cell-level agreement, and four derivations of the spreadsheet template with identical prompt, payload and input bytes produced rule coverage of 15, 12, 19 and 23 out of 27 (Section 5.4.5). The second figure is worth pausing on, because the temptation is to present the vocabulary of this section as an improvement. It is not. Naming the derivation-time term does not shrink it, and a spread of 15 to 23 on a 27-item denominator is large; the vocabulary makes it *describable* and therefore reportable, which is the whole of the gain. Two quantities are missing and are named as such: the cold-start distribution over D independent derivations under the isolation conditions of Section 5.4.5, and a specification-level analogue of the silent-failure rate — how often a specification passes every gate and is still wrong. One methodological limit bears on both

and is the reason no calibration curve will appear in this thesis: the calibration and conformal layers assume the same problem can be resampled cheaply, and one derivation here costs 550 to 930 seconds of wall clock and real money, so $D = 3$ is a budget ceiling rather than a design preference. What this thesis can report is a dispersion, not a calibration. Finally, none of this touches whether the reference outputs themselves are right; that is a separate threat with a separate countermeasure (Section 5.5.2), and conflating a consistency measure with a correctness measure is exactly the error Section 5.5.1 forbids.

Status: the taxonomy is citation-backed; the zero-dispersion property is implementation-backed; the reframing is normative and one of its two quantities is unmeasured. Each of the five layers is characterized from the fetched abstract of the work cited for it, and all six works of this section were added to `docs/thesis/refs/manifest.md` in this round with their DBLP venues, because the survey draft contained no bucket for them at all. That execution-time dispersion is zero is implementation-backed by the negative-control test named in Section 2.8 together with the byte-identity measurement of Section 5.4, and the qualification that the container digest of a Word package is *not* identical across writes is recorded there rather than smoothed over here. The factorization is a description of this architecture rather than a result, and the claim this section deliberately does not make is that relocating the uncertainty reduced it: the measured derivation-time spread is wide, it is reported in full, and it is unfavourable to this thesis. The bias direction of the missing cold-start distribution runs the other way, towards this thesis, since the four derivations that exist were run for debugging across changing versions of the system and a reader has no way to know whether an isolated replicate would disperse more or less.

Chapter 3. System Model

3.1 Problem Formulation

3.1.1 A Stateful, Two-Stage Input Model

The abbreviation $\mathcal{A} : T \to S$ is potentially misleading because the system does not consume all experimental material in one call, and derivation is neither pure nor deterministic. Let d denote one derivation dataset in template lineage ℓ. We split the available material into a derivation layer and a later regression layer,

$$T = \left(T_{\mathrm{der},d}^{(n)}, T_{\mathrm{reg},\ell}\right), \tag{3.1}$$

$$T_{\mathrm{der},d}^{(n)} = \left(d, \ell, B_d, Y_d^?, \{X_{dj}\}_{j=1}^{N_d}, R_d^?, U_d^?, Q_d, P, \Gamma, \Theta, H_d^{(<n)}\right), \tag{3.2}$$

$$T_{\mathrm{reg},\ell} = \mathcal{R}_\ell = \left\{c_i = \left(I_i, \{X_{ij}\}_{j=1}^{N_i}, u_i, \pi_i, Y_i^?\right)\right\}_{i=1}^{K_\ell}. \tag{3.3}$$

Here, B_d is exactly one baseline template; $Y_d^?$ is an optional expected output; $\{X_{dj}\}_{j=1}^{N_d}$ is a possibly empty collection of source artifacts; $R_d^?$ is an optional operational-rule artifact; and $U_d^?$ is an optional inline source-data override. When present, the override bypasses the locally parsed artifact dictionary and, on the agent path, takes precedence over extracted keys. Q_d groups the inspection level, force-refresh choice, content-addressed dump, cache state, CAD preflight and health observations, locator verification, and deterministic validation/repair feedback. The derivation environment is also an input: P denotes the prompt files, Γ the agent's

tool and skill definitions, and Θ the model, gateway, and MCP configuration. Thus, changing a prompt, an allowed tool, or the selected model changes T_{der} even when all customer artifacts are byte-identical.

The term $H_d^{(<n)}$ is the pre-existing state before the nth derive. It contains the field names and rule inventories accumulated from all existing non-degraded specs derived from the same dataset. The implementation reads this state through `derivation_service.DerivationService._field_hints`, and `_previous_rule_inventory`; the degraded flag used by that selection is produced by `cad_health.degradation`. The hints are fed to the agent and the prior inventory is merged into the new spec evidence. With ω_n denoting model sampling and the observed session/tool trace, the honest derivation relation is therefore

$$S_d^{(n)} \sim \mathcal{A}\Big(T_{\mathrm{der},d}^{(n)}; \omega_n\Big), \qquad T_{\mathrm{reg},\ell} \notin \mathrm{dom}(\mathcal{A}). \tag{3.4}$$

Consequently, two derives over the same artifact bytes need not have the same context or the same output: $H_d^{(<n)}$ may have changed, and ω_n is stochastic. In this thesis, $\mathcal{A} : T \to S$ is only shorthand for the projection $\mathcal{A} : T_{\mathrm{der}} \rightsquigarrow S$ in Equation 3.4, not a claim that $\mathcal{A}$ is a pure function.

This split follows the implemented call boundary. The dataset creation and derive paths, `api.create_dataset` and `derivation_service.DerivationService.derive`, admit one baseline, at most one expected artifact, N_d source artifacts, and at most one rules artifact. They do not read `models.DwgTestCase`. Only after S exists does the separate endpoint call `regression.RegressionRunner.run`, which selects every active test case of the spec's lineage and executes them. K_ℓ is therefore the number of active cases at regression time; the implementation imposes no $K_\ell = 3$ limit. Each case c_i contains an input template I_i, its own source artifacts and explicit source-data override u_i, runtime parameters π_i, and an optional expected output $Y_i^?$.

The prompt, tools, and model terms above are concrete inputs rather than abstract nuisance variables. `derivation_agent.OpenCodeAgent.derive_rules` selects the `dwg-derivation` agent and `OpenCodeAgent._run` submits the task payload; the selected agent is configured by `opencode.jsonc`, its prompt is `prompts/derivation.md`, and its permitted CAD MCP tools are defined by the agent configuration and `mcp_server.server`. These files and settings must therefore be versioned with the material definition used in an experiment.

We reason about the resulting specification as

$$S = \big(G, L_s, L_t, F, \Pi, \mathcal{E}_S\big), \tag{3.5}$$

where G contains value-source and transformation semantics, L_s contains source locators, L_t contains independently derived target locators, F contains rendering rules, Π is the runtime-parameter schema, and $\mathcal{E}_S$ is derivation and validation evidence. The persisted `DwgSpec` stores these concerns across `field_map`, rule projections, `runtime_params`, and `evidence`; this tuple is a conceptual separation rather than a claim that each component is a distinct database column.

Information boundary. The operational-rule artifact R_d may identify source fields and state formulas, conditions, constants, and formatting requirements, but it must not contain target cell addresses, bookmark names, content-control tags, or mappings of the form `source_field` $\rightarrow$ `target_cell`. Target references belong to L_t and must be derived from the template by $\mathcal{A}$; otherwise R_d becomes an answer table and the derivation problem is leaked into its input. **Status: normative/aspirational only.** The boundary is stated but not enforced, in two independent ways. First, no admission check exists: neither `api.create_dataset`

nor `derivation_service.DerivationService._load_rules` rejects an answer-bearing rules artifact, so a hand-authored R_d containing target addresses is accepted silently. Second, the persisted projection is not structurally safe: `derivation_service.DerivationServic` copies the field's target-reference slot into `DwgSpec.mapping_rules`. That slot is carrier-agnostic — its interpretation follows the target kind of the template, so it holds bookmark identities for `.docx` and cell addresses for `.xlsx`, which are exactly the two artefact classes this boundary forbids. (The field retains the legacy name `handles` from the drawing-oriented prototype; the naming is historical and does not narrow the scope.) If that projection is exported, presented as the rule table, or fed back as R_d, it exposes target answers, and $\mathcal{A}$ is then measured on its ability to copy them rather than to derive them. It must therefore remain internal, and be stripped of target references or split from the semantic rules, before the boundary may be described as a property of the system rather than a requirement on it.

3.1.2 Compile-Once, Execute-Many

The target problem class consists of recurrent template-instantiation tasks for which the output is deterministically derivable from structured source data and explicit operational rules. "Compile once" means that the expensive, stochastic derivation in Equation 3.4 produces one reusable S; it does not mean that a single fixed output job is reused, because instance values differ. For instance input $V_i = (I_i, \{X_{ij}\}, u_i, \pi_i)$, execution deterministically instantiates and

applies the spec,

$$(J_i, P_i) = \mathcal{C}(S, V_i), \tag{3.6}$$

$$D_i = \mathcal{X}(J_i, I_i), \tag{3.7}$$

$$\mathrm{ok}_i = \mathcal{O}_{\mathrm{plan}}(P_i, I_i, D_i) \wedge \begin{cases} \mathcal{O}_{\mathrm{expected}}(D_i, Y_i), & Y_i^{?} = Y_i, \\ \mathrm{true}, & Y_i^{?} \text{ is absent.} \end{cases} \tag{3.8}$$

$\mathcal{C}$ is the deterministic per-instance compiler, P_i is its authoritative write plan, $\mathcal{X}$ is the format-specific executor, and D_i is the produced document. The problem is to derive an S once such that ok_i holds for every admissible instance, while the cost of derivation is amortized over repeated executions.

The deterministic core is implementation-backed. In `compiler.compile_cad_job`, the spec, source data, runtime parameters, and current template values are explicit inputs; failures are returned before submission, and the canonical job JSON is content-hashed. The DOCX/XLSX path shares this compiler and applies P_i through `docx_target.DocxTargetAdapter.` or `xlsx_target.XlsxTargetAdapter.apply`; `document_production.document_job` records a canonical hash for the document job. Regression is also implementation-backed: `regression.RegressionRunner._run_case` compiles each case, and `regression.Regress` uses `verification.verify_output` plus `verification.compare_expected` when Y_i exists.

This formulation deliberately localizes randomness to $\mathcal{A}$, and the execution path now matches that localization. An earlier revision of this chapter recorded that it did not: `production.ProductionC` invoked `derivation_agent.OpenCodeAgent.repair_cad_job` after a failed gate-way job validation, and the comparison that guarded the repaired job excluded target locations,

so a model-relocated target could be accepted and submitted inside the same run. That branch has since been removed from execution. A compiled job the tenant manifest rejects terminates the run with `CAD_JOB_MANIFEST_REJECTED` and an instruction to re-derive, review, and republish; a cached target reference the current template no longer contains terminates it with `CACHED_HANDLE_NOT_FOUND`. Neither is repaired in place, and `repair_cad_job` has no remaining caller on the execution path.

Status: implementation-backed, and pinned by a negative control rather than by inspection. `test_successful_production_operation_records_zero_llm_calls` replaces both agent repair entry points with a recorder that returns a failure, runs a complete production operation, and then asserts both that the recorder was never invoked and that the `LlmOperation` persisted for the run holds zero model calls. Reading the source would only show that the call site is absent today; the test also fails if a future change reintroduces one. This is the evidence behind claim C4, and it is deliberately the narrowest of the thesis's claims: it covers $\mathcal{X}$, not $\mathcal{A}$, and not the derivation-time repair loop of Chapter 4, which is by construction inside the compile step whose cost repeated execution amortizes.

3.1.3 Required Properties of the Specification IR

The reusable IR must satisfy the following five properties. Each status below refers to the repository state examined for this thesis, not merely to the intended design.

P1—separation of structure and instance values. S must describe reusable source/target locators, transformations, formatting, and parameter types; a particular instance supplies values through V_i. The only values permitted in S are constants explicitly prescribed by R_d, not values copied from a calibration output. **Status: implementation-backed at the representation interface.** `compiler.compile_cad_job` accepts `spec`, `source_data`,

and `runtime_params` separately, while `value_layer.resolve_field_raw` evaluates the value layer at execution time; `regression.RegressionRunner._run_case` reuses the same spec with each case's inputs. This interface supports reuse, although successful regression—not the type shape alone—is what detects a semantically overfit constant.

P2—explainable and verifiable locators. Every source or target locator must expose what it resolved to and must be checkable against the corresponding artifact; an unresolved or ambiguous locator is evidence, not permission to guess. **Status: implementation-backed.** `source_locator.r` returns the resolved sheet, cell, selection, and match count, and `source_resolution.verify_again` checks that the locator reproduces the derivation sample. On the target side, `target_ref.BaseTargetA` and `describe` provide a common existence and explanation interface. `docx_target.DocxTargetAd` and `xlsx_target.XlsxTargetAdapter.inspect` retain labels, physical locations, positional flags, and ambiguous references, while their `apply` methods return every missed reference.

P3—bounded layout resilience and fail-loud behavior. Semantic identities should survive admissible layout changes, while a change outside the declared tolerance must stop the run rather than trigger a guessed write. This includes the zero-LLM execution requirement: an execution-time model must not relocate a target. **Status: implementation-backed for the fail-loud half; the resilience half is bounded and measured rather than general.** Source locators resolve by sheet/header/section semantics; `docx_target.DocxTargetAdapter` distinguishes stable content-control/bookmark identities from positional table cells; `xlsx_target.XlsxTarg` distinguishes stable defined names from positional coordinates; and both reject duplicate or missing references. The execution branch that previously undermined this property — agent repair of a rejected job inside the run — has been removed, as described in Section 3.1.2, so a

target that cannot be resolved now ends the run with a code instead of being relocated.

What is *not* claimed is the resilience half in general form. Layout tolerance is declared per locator mechanism, and a change outside a mechanism's declared tolerance is detected only insofar as the mechanism can see it. Two measured boundaries are carried forward to Section 3.1.4 rather than glossed here: an unused detail-row cell that no column of a repeat block addresses is not cleared by `unused_slots: "blank"`, so a template reused from a previous reporting period can carry a stale value into a fresh run; and the derivation-time semantic oracle compares only references whose text differs between the baseline and the expected document, so a specification that writes into a cell the expected document leaves blank is structurally invisible to it.

P4—diffability, versioning, and lineage. The IR and its evidence must support reviewable changes, immutable publication, and regression over a template lineage. **Status: implementation-backed for the currently produced handle-backed field-map shape.** `models.DwgSpec` carries a lineage-scoped unique version and template fingerprint; `drift.spec_field_drift` reports added, removed, and attribute-level field changes; `models.DwgProductionVersion.save` prevents mutation of the published spec/hash identity; and `regression.RegressionRunner.run` persists results for all active cases of the lineage. The drift function reads the drawing-flavoured spelling of a target rather than the format-neutral list, which was recorded here as a prospective blind spot; Section 3.8.3 reports the measurement that retired that concern and the narrower gap it found instead.

P5—independently auditable rule coverage. S must enumerate which rules and fields it claims to cover, and the denominator must come from the rule artifact independently of the model being scored. **Status: implementation-backed since the coverage-v2 path replaced**

agent self-enumeration. The original implementation — `rule_coverage.merge_inventory` over an agent-supplied `rule_inventory`, unioned across prior derives — did not satisfy P5 and is retained only to read historical specifications. A rule the agent never enumerated was absent from the denominator, so "22 of 22 covered" and "22 of 40 covered" were the same observation.

The active path externalizes the denominator instead. A dataset that carries a rule document must also carry a frozen, versioned rule-applicability manifest; `rule_coverage.parse_rule_manif` verifies that the manifest names the exact SHA-256 of the rule-document bytes and that its declared document roles match the dataset's, and `derivation_service.DerivationService._loa` refuses the derive with `RULE_MANIFEST_MISSING` before any paid model call, so a run cannot discover after the fact that its release denominator is unauditable. The agent supplies a *disposition* for each externally assigned rule identifier and nothing else: `rule_coverage.score_manifest` discards unknown identifiers, refuses duplicate dispositions, and refuses an assignment whose document roles disagree with the manifest, so the scored agent can neither add, remove, rename, nor move a rule between roles. The resulting partition into `covered`, `unresolved`, `rejected`, and `unmentioned` is mutually exclusive and sums to the frozen applicable set, and `gates.canonical_unresolved` turns every non-`covered` disposition into a publish gate rather than partial credit. Section 4.6 develops what this measures and what it still does not.

3.1.4 Measured Boundaries of the Specification IR

The five properties above state what the IR must do. This subsection states what it demonstrably does not, because a boundary that is only discovered by a reader running the system is indistinguishable from a defect the author did not notice. Each item below is either pinned by a

test or was measured on a real artifact; where an alternative shape was considered and rejected, the reason is given, since "we did not think of it" and "we rejected it" are different claims.

B1—`exclude` is a row-content predicate. A range's `exclude` clause inspects the content of a data row. It does not inherit meaning from a preceding section header, so a sheet whose rows alternate between "current month" and "cumulative" groups under one header cannot be partitioned by `exclude`. The rejected alternative was a domain flag such as `exclude_cumulative`; it was refused because it names one agency's form inside a carrier-neutral IR. The general shape adopted instead is a group projection (`records`: a fixed `group_size` with named row roles and per-column `select_offset`), which describes the same layout without naming the domain.

B2—stacked records assume a fixed group size. `records` models "g physical rows form one logical record" for a constant g. A record whose row count varies with its own data is outside the IR. Both the read side and the write side are implemented, and a specification produced by the agent for the XLSX evaluation template declares `records` with `group_size: 2` and named row roles, so the primitive is exercised by a main evaluation dataset rather than by fixtures alone.

B3—the verification basis for the collection primitives is $n = 1$. `artifact_collection`, `records`, and the equality merge of Section 3.5.2 each have exactly one motivating real workflow. Six independently sourced government workbooks were scanned for a second instance of a repeating multi-row record block and none was found; the nearest candidate was a one-off label merge, not a repeating record. The generalization threshold this thesis sets for a new primitive is therefore not met for these three, and they are presented as carrier-neutral shapes with a single validating workflow rather than as generally validated primitives. The risk this creates

is specific and worth naming: it does not appear as a failing test, because the tests were written from the same single example. It appears as a wrong result on a form nobody has tried.

B4—merge is equality-only and inner. Two resolved row sets can be joined on equal key columns. Range containment, interval overlap, and inequality correspondences are not expressible. The join is deliberately inner: supplying a default for an unmatched key would convert a missing record into a normal-looking number without anyone having authorized the default, which is precisely the silent-failure shape this thesis measures. A specification that needs the filled row must produce it on the source side, where the decision is visible. Merges chain, so three tables can be joined as two binary merges; the agent produced a chained pair unprompted.

B5—the aggregation axis is columns only. Aggregation sums named columns. The criterion is who decides the number of columns: if the template does, naming them is legitimate; if *this instance's data* does, the specification has become instance-specific, and the correct modelling is a long table plus a pivot at write time. A row axis is not offered because a wide table's column names would then be data values, which would disable the author-time static check that a formula only names columns the row set actually has.

B6—a repeat block clears only the cells its own columns address. When a block declares `unused_slots: "blank"`, the cells cleared in an unused slot are the cells that block's columns point at. A cell in the same slot that no column addresses is not cleared, because the IR knows a block's targets only through the strings its columns declare, and "every other cell on this row" is not something it can name. For a pristine template this is exactly right, and every instance of both evaluation datasets starts from a pristine template. For the common practice of reusing last period's completed report as this period's template, it means a stale value can sit on a row the system considers written. The behavior is pinned by a test rather than left to inference,

and the rejected workaround—declaring a column whose value is always blank—was refused because it puts layout knowledge into the value layer and makes the column count grow with the template's width.

B7—one instance is one filled target template. A specification may declare several output documents, but their roles must be distinct; a run that would produce M documents of the same role is refused. Relaxing this is a separate decision and is not claimed here.

3.1.5 Design Principle: Fail Loud, Not Silent

A *silent failure* is an output that is complete, well formed, correctly formatted, and wrong. It is the failure mode this thesis is organized around, and every design decision in this chapter that looks unnecessarily strict is a consequence of it.

The asymmetry is economic rather than aesthetic. A loud failure costs a rerun: somebody sees an error code, reads the reason, and fixes an input or a rule. A silent failure costs whatever the wrong number causes downstream, and it costs it *after* the document has been signed, filed, or paid against, because nothing in the delivery path distinguishes it from a correct document. The two are not two points on one scale of quality. A system that refuses too often is annoying and, crucially, *measurable* — its refusals are counted. A system that guesses is neither: its guesses are counted as successes. Optimizing a template-filling system for the number of fields it manages to produce therefore optimizes it in exactly the wrong direction, and the procedures in Chapter 4 are arranged to resist that pressure. The failure-mode taxonomy of Section 2.3 is the vocabulary this creed is stated over.

The same principle at three levels. The two earlier subsections state this creed twice without naming it, and it is worth collapsing them, because the three statements are not analogies —

they are the same rule applied to three different artifacts.

1. **The produced document.** This is the familiar level, and property P3 of Section 3.1.3 is its statement: a target that cannot be resolved ends the run with a code rather than being relocated by a model. The forbidden behavior is “locate approximately, then write”.

2. **The specification.** A specification can be quietly wrong in ways no single execution reveals. The information boundary of Section 3.1.1 is the statement at this level: if the rule artifact R_d were permitted to carry target addresses, a derivation that merely copied them would be indistinguishable from a derivation that found them. The specification would look derived; nothing about it would be. The forbidden behavior is “supply the answer, then measure the search”.

3. **The measurement.** A metric can be quietly wrong while every mechanism under it is correct. Property P5 is the statement at this level: while the coverage denominator was enumerated by the agent being scored, “22 of 22 covered” and “22 of 40 covered” were the same observation, and no test could fail. The forbidden behavior is “let the thing being measured choose the denominator”.

Each level has its own way of being plausible while wrong, and therefore needs its own loud channel; a guarantee at one level does not propagate to the others. This is why the thesis reports semantic correctness, extent diagnostics, presentation, untouched regions and package integrity as separate numbers (Section 5.5.1) instead of one accuracy figure. An aggregate is a fourth place for a silent failure to hide.

What the principle forbids, concretely. The creed is not a slogan in this system; it is a set of refusals that each cost something. The source-side extent gates are the clearest examples, because each one refuses a value that would otherwise look entirely normal: `source_locator`

refuses an extent that selected zero rows (returning 0 for "sum of no rows" is indistinguishable from a genuine zero); refuses an extent that swallowed a subtotal row (the result is exactly twice a plausible number); and refuses a blank or non-numeric cell inside a numeric aggregate rather than coercing it to zero, which is the most common way a spreadsheet quietly underreports. The value layer applies the same rule one stage later: its aggregate helper raises rather than coercing, explicitly so that it cannot become a second, laxer gate behind the first. The equality merge of Section 3.5.2 is inner for the same reason (Section 3.1.4, B4): defaulting an unmatched key would convert a missing record into a normal-looking number that nobody authorized. A duplicated bookmark name or content-control tag identifies nothing and is recorded as ambiguous rather than resolved to whichever copy came first. And the one construct in the IR whose whole effect is to *relax* a publish gate — a declaration that a reference is deliberately left blank — is checked three ways at derivation time, and a declaration that fails any check is not attached to the specification at all (Section 3.3), because a declaration that both fails and inflates the coverage numerator is a silent failure of the measurement level while looking like a caught defect at the specification level.

The price, stated rather than hidden. Fail-loud has a bill and this thesis pays it in a visible place. Both of the best specifications produced for the two main evaluation carriers are, as measured, blocked from publication: on the XLSX template the merged publish gate reports 13 blocking items and on the DOCX pair it reports 8. Neither number is a defect count; both are correctness-preserving refusals, and Section 4.5 shows that reading them as a defect count is itself an error, because the same repository has reported "N unresolved" from two different counters that disagree in both directions.

Status: implementation-backed as a set of named refusals; *not* a proof that no silent failure remains. The principle is realized as error codes and gates that can be enumerated — among them

```
LOCATOR_RANGE_EMPTY, LOCATOR_RANGE_CONTAMINATED,
SPEC_REPEAT_TARGET_OVERLAP,
SPEC_REPEAT_SLOT_ALIGNMENT_UNVERIFIED,
SPEC_FORMULA_INPUT_UNPRODUCED,
CAD_JOB_MANIFEST_REJECTED, CACHED_HANDLE_NOT_FOUND
```

and `benchmark/probe/guard_sweep.py` mechanizes the rule that adding a new refusal must not quietly retire an older one: it disables each guard in turn and requires that guard's own tests to fail, and a row that selects no tests is treated as invalid rather than as passing. What must not be inferred from any of that is that the produced documents contain no silent failures. The thesis has a measured counterexample from its own system: on the XLSX evaluation template the derived specification declared no column for the remarks field, five calibration and development instances were executed, and three of the resulting cells were scored `silent_failure` by the shared evaluator — output that was complete, plausible, and wrong. That case is discussed as a result rather than as an exception in Sections 4.7 and 5.5.1; here it fixes the strength of the claim. The system is built so that the *known* ways of being quietly wrong are loud, and it measures how many of them remain, but "fail loud" is a design discipline with an evidence trail, not a guarantee.

3.1.6 Scope and Applicability Conditions

Section 3.1.2 introduced the target problem class in the course of arguing for compile-once execution. This subsection states it as an admission test, since Chapter 1 refers to it and

Section 6.2 develops what happens when it fails. Three conditions must hold together; each is stated with the question one would actually ask before adopting the method.

C-I — Recurrence. The same template, or a lineage of revisions of it, is instantiated many times. *Operational test:* can one name the expected number of instances per template revision, and is it large enough that a one-off derivation cost is amortized? The derivation is the expensive, stochastic step; nothing about the method is cheap at $N = 1$. Recurrence is what makes the trade in the first place, and it is why Section 5.4 reports a break-even N rather than a cost ratio.

C-II — Deterministic derivability. Every output value is a function of structured source data and the rules, computable without judgement. *Operational test:* for each field, can two competent people, given the same sources and the same rule document, be expected to produce the same character string? If a field requires professional discretion — a risk rating, a recommendation, a summary in someone's words — it fails this condition, and the correct response is to leave it outside the method rather than to widen the method. The evaluation enforces this rather than assuming it: the path that lets a person supply a value at execution time exists in the code and is disabled at derivation time (Section 3.7), so a derived specification cannot classify a field it found hard as one a human will fill in.

C-III — Explicit operational rules. The mapping is written down as rules over the source data, not held as tacit practice. *Operational test:* does an artifact exist which states, for each field, where the value comes from and what is done to it — and would a new employee be handed that artifact? Note what this condition does *not* require. It does not require the rules to be machine-readable, complete, or even correct: both evaluation datasets use prose rule documents, and building one of them found three rule statements that disagreed with the real form, one of

which pointed at a location that does not exist. It does, separately, forbid the rules from carrying write coordinates, which is the constraint of Section 3.1.1 and is a condition on the *experiment* rather than on the task.

The three conditions are independent. A recurring task with tacit rules fails C-III and is a documentation problem before it is an automation problem. A well-documented, deterministic task performed once fails C-I and should simply be done by hand or by an agent. A recurring, well-documented task containing one discretionary field fails C-II for that field only, and the honest decomposition is to carve the field out — which this thesis does not claim to have measured, because it disabled the mechanism that would allow it.

Status: normative, with the selection bias disclosed. These conditions are definitional, not empirical: no experiment in this thesis varies them. Both evaluation datasets were chosen *because* they satisfy all three, which means the thesis measures the method inside its declared scope and says nothing about how it degrades near the boundary — for instance, how a specification behaves when a rule document is twenty per cent incomplete, or when one field in twenty needs judgement. The bias direction is favourable to the method and is recorded as such in Section 5.10. The consequences of each condition failing are discussed in Section 6.2; this subsection deliberately gives only the definitions.

3.2 Domain-Dependent and Domain-Independent Layers

Whether this work is a tool for one file format or a method that happens to have been implemented for three depends on where the format-specific code stops. This section states that seam, because it is a claim about the contribution rather than an implementation detail: everything on the neutral side is shared by all three carriers as the same code, not as three

parallel implementations that agree today.

Table 4: Where format knowledge is allowed to live.

Carrier-dependent	**What it must answer**
Structured extraction	What is in this file, as a dump: the drawing dumper through the CAD gateway; `DocxTargetAdapter.inspect` and `XlsxTargetAdapter.inspect` in process.
Target location, write-back, and the baseline-to-expected diff	The operations of the `target_ref.TargetAdapter` protocol, plus `addressable` and `rebind`.
Package preservation	`xlsx_package`: which bytes must survive a write (Section 3.6).
Carrier-neutral	**Why it can be**
The IR and its normalizer	`spec_ir` speaks of documents, fields, extents, repeat blocks and blank targets; none of those words names a format.
Source locators and row sets	`source_locator`, `row_set`: a source workbook is read the same way whatever the output template is.
The value layer	`value_layer`: formulas, rounding, formatting.
The compiler	`compiler.compile_cad_job` produces a carrier-neutral write plan that `document_production` or the CAD gateway then executes.
Gates, oracles, coverage	`spec_check`, `gates`, `rule_coverage`.
Lineage, fingerprint, drift, regression	`models`, `fingerprint`, `drift`, `regression`.

Two properties of this seam are load-bearing. First, the carrier-dependent operations are the *only* ones a new format must supply: `target_ref.BaseTargetAdapter` implements `diff`, `verify` and `describe` once over a single per-format primitive, so a format states what its addressable targets are and inherits the rest. Writing those three per format is how a `.docx` diff quietly starts reporting something subtly different from a drawing diff, which no reviewer can see because the two are never displayed side by side. Second, the neutral side is neutral in its *vocabulary* as well as its code: a specification's write target is normalized to `{kind, ref}`, so "which places does this specification write" is one function rather than one per carrier.

The seam is not the module namespace, and confusing the two has cost something. The Python package is still called `features.dwg` and the compiler entry point is still `compile_cad_job`; both names are historical, from the drawing-only prototype, and renaming them would couple a cosmetic change to models, migrations, API paths, frontend types and the CAD case-study contracts. The names are therefore left alone — but the *messages* were not neutral, and that was a real defect rather than an aesthetic one. Until it was corrected, a failed specification validation returned "DWG spec validation failed" for every carrier, so an Excel run whose specification was rejected told its reviewer that something about AutoCAD had failed; two XLSX specifications were in fact rejected that way. The sibling drawing error codes were audited at the same time and are correctly confined to the drawing branch. The general lesson is recorded in Section 4.2: a correct check whose message names the wrong repair costs a reviewer as much as a wrong check, and no test covers the wording.

Status: implementation-backed, with one measured way the seam has been crossed accidentally. The table above describes the current module graph, and the `TargetAdapter` protocol makes it checkable rather than aspirational. What it does not guarantee is that every *path* through the neutral code has been exercised by every carrier. The measured counterexample is Section 4.1's: the single-document branch of the derivation service did not carry repeat blocks at all, because the only dataset that had ever used them produced two documents and therefore took the other branch — a correct agent reply was persisted with the detail table missing and no error raised. Neutrality of the code is therefore claimed; uniform coverage of the code by both carriers is not, and Section 5.10 records it as a threat.

3.3 Specification IR

The IR is the artifact the whole method exists to produce: the reviewable, versionable, executable object that a derivation emits once and every later instance consumes. This section describes what it addresses; Section 3.1.4 has already stated what it demonstrably cannot.

Scope: one instance, not one file. A specification describes the whole output set of one instance. Its root is a list of documents, each with a role, a template reference, a list of fields, a list of repeat blocks, and a list of blank-target declarations; declarations more than one document reads — named extents — live in a shared section beside them. The unit is the instance because that is the unit of the task: a customs shipment produces a commercial invoice *and* a packing list from one set of rules, and deriving them separately is neither the same unit of work as the agent baseline it is compared against nor safe, since two independently declared extents can disagree about which source rows are detail rows and nothing at execution time would notice.

An earlier generation of the IR had a single field list at the root, and twenty-one stored specifications still use it. They are not re-derived. `spec_ir.spec_documents` reads either generation as the same list, and every consumer goes through it. This normalizer is the point of the module rather than a convenience: a consumer that reads the root field list directly finds it empty on a new-shape specification and reports a successful run that wrote nothing, which is this thesis's definition of a silent failure.

A field. A field carries a name, a value source (`source`, `computed`, `constant`, `carry_over`, or the disabled `user_input`), the source key or semantic `source_locator` that supplies its value, an optional transform and transform type, an optional render format, an optional aggregate over a named extent, a human-language

description carrying no meaning to the compiler, and a target. Names are global across documents: one shared formula namespace is what makes a cross-document rule expressible at all — "the two documents' totals agree" is exactly that shape — and the cost, that two documents reusing an obvious name collapse into one, is reported as a warning rather than guessed at.

A repeat block. A block declares a detail table once and is expanded deterministically into the template's fixed slots, so the size of a specification stops depending on how many detail lines the calibration instance happened to have. It names the extent it iterates, the number of slots the template offers, where the first slot starts, how many physical rows form one logical record together with the named role of each, what happens to a slot the instance has no data for, and its columns; each column names its target through a template string containing a slot placeholder, and reads either a source column of the extent or a per-row expression over it. The one policy for an unused slot is to blank it, and deliberately only one: leaving a slot alone keeps the previous batch's row visible in the produced document.

Blank targets. A document may declare references it deliberately does not write, so that a rule whose obligation is negative — "this cell must stay blank" — can be implemented rather than left unresolved. This is the only construct in the IR that relaxes a publish gate, so a declaration must pass three derivation-time checks: every reference must be addressable on the template, must be blank in the expected artifact, and must be written by nothing else in the specification. A declaration failing any of them is reported together with the check that failed and is *not* attached to the document, because attaching it would let a false claim into the coverage numerator even while a finding blocked the publish.

The IR is also a published contract, and a declaration nobody runs drifts. A JSON Schema for the specification ships in the contracts package, and it is a real constraint rather than documentation: several of its object definitions close `additionalProperties`, so a key the schema does not know is illegal rather than ignored. That strictness has already caught the failure mode this repository pays for most often — an allow-list that silently drops a key — but in the opposite direction. When the repeat-block definition was checked against a real specification for the first time, it turned out to have been forbidding, for two design revisions, the row-set columns the implementation had been emitting all along, because no test had ever validated a real specification against it.

The lesson was drawn one notch too narrowly. That check was extended for repeat columns, and the test written to enforce it validated a *hand-written* candidate rather than a stored specification — which is to say it was written from the same understanding as the code it was meant to contradict. Validating all 35 stored specifications against the published schema found 11 invalid from eight distinct causes, and among them the field-level aggregate key that detail-table totals require: every specification of the DOCX shape had been contract-invalid since that key was introduced, and nothing in the system said so. The adjudication was made key by key rather than in bulk. Four causes are the contract being wrong: the aggregate key, and the reviewer-facing label, prose and provenance keys the derivation attaches to repeat blocks and their columns, which reach a stored specification verbatim because repeat blocks pass through no allow-list at all; and a null render format, which the value layer treats as identical to an absent one for a field, a repeat column and a row's second cell alike. Those are now declared, with `additionalProperties` still closed on both objects, since what that flag is for on the repeat side is a misspelt slot template — a key nothing reads, addressing one cell for every slot, with every write reporting success. Four causes are the contract being right: three legacy

specifications with no source declaration and one with an unlocatable second cell. Those four are refused by the compiler as well, for the same reason and at the same place.

The measured statement is therefore not a proportion but an agreement: 31 of 35 stored specifications satisfy the contract, and *no* stored specification is one the compiler accepts and the contract rejects, where six were before. The check runs in two places because the two questions differ. A probe validates the stored corpus, which is the only material derived by paid runs rather than authored for a test; and the persisted output of four end-to-end derivations — the two-document DOCX shape, the single-document XLSX shape, a collection read, and a merge — is validated inside the test suite, which is what turns red the next time a derivation emits something the contract forbids. Reverting each relaxation in turn moves both numbers, which is what makes the first one evidence rather than an absence.

Status: implementation-backed, including the contract. `spec_ir` is a pure module with a normalizer both generations pass through, and its shape checks are the same functions the write side and the read side use, so a specification cannot be validated by one vocabulary and executed by another. The contract gap was handled under the first of this thesis's three routes for such a gap — fix the implementation — rather than by weakening the claim. What remains normative is narrower than it was: the contract is enforced over derivation output and audited over the corpus, but it is not enforced at persistence time, so a specification that violated it would be caught by a test run rather than refused on the spot. And the contract constrains shape only. A perfectly valid specification can still write the right value to the wrong cell; that is what the author-time oracle of Section 4.3.1 and the evaluator of Chapter 5 are for, and a key being legal is not evidence that anything reads it.

3.4 Structured Dump as Grounding

Nothing in the derivation opens a template. The agent is shown a *dump*: a structured inventory of what a template contains and where it can be written, produced by deterministic code before any model call. This is the grounding step, and three properties of it matter for the thesis.

One shape for three carriers. A drawing dump is produced by the CAD gateway; a `.docx` or `.xlsx` dump is produced in process, since neither has a gateway, a worker, or a cold start to time out on. Both are then reshaped into the same envelope — a list of entities, each with an opaque reference, its current text, and its kind — with the richer per-target detail (caption, positional flag, physical location) riding alongside. This is not a fudge: because the target reference is already defined as a format-neutral identity, reshaping here means the baseline-to-expected diff, the field-map derivation, the evidence record, the compiler, the verifier and the publish gates all work on a Word template without a second implementation of any of them. Two things are deliberately kept *outside* the entity list. A document's full text lines ride beside it, because a regression case comparing against an approved document must see text the specification never mentions, while a paragraph is not a write target and putting it in the entity list would make target verification accept it as a place a field may write. A workbook's sheet titles are recorded explicitly, because a blank cell is not a target, so a sheet whose write points are all still empty would otherwise be invisible in the dump.

Content addressing, and the version of the dumper. Dumps are cached and keyed by tenant, the artifact's content hash, the inspection level, the target kind and the scope, so re-deriving over unchanged bytes performs no extraction, and the cache disposition per template is reported with the result rather than hidden. The key carries a dump schema version for a reason that is easy to

miss: a dump is a pure function of the file's bytes *and of the dumper*. Content addressing covers only the first, so changing what the extractor emits without touching any file leaves every key matching and serves stale-shaped dumps indefinitely; bumping the constant invalidates them by construction. The target kind is in the key for the analogous reason — a `.docx` dump and a drawing dump are different shapes of answer to the same question, and one must never be served for the other.

Honest disclosure: the carrier libraries lose things. An extractor is only as faithful as the library under it, and this one has measured losses. Across the 29 real `.xlsx` files in the repository, a load/save round trip through the spreadsheet library silently drops the calculation chain and the shared-string table (both harmless, since Excel rebuilds them), the printer settings and the sheet relationships that reference them, cell comments together with their legacy drawing part, chart colour and style parts, and — only when an optional imaging dependency is absent — every embedded image. That last one was found the expensive way: the dependency was present in the development environment, declared nowhere, and absent from a container build, so images disappeared from written workbooks and every run reported success.

Two consequences are worth separating, because they land in different places. As a *carrier* limitation it belongs here: what a spreadsheet library can round-trip bounds what any method built on it can promise, and it is not a limitation of compile-once execution. As a *measurement* problem it belongs in Section 3.6, because for a period the system under test lost exactly the parts that the benchmark's package-integrity channel injects as its fault, which made that channel unable to tell a correct run from a deliberately broken one.

Status: implementation-backed, with the extractor's coverage stated as a bound. The cache key, the version constant and the two carriers' in-process extractors are all in the pro-

duction path and are exercised by the derivations reported in Chapter 5. What is *not* claimed is that a dump enumerates everything a template can express. A spreadsheet dump offers only cells that carry content — measured on the real monthly-report template, 27 of them, while the entire data area the specification writes ships blank — and that measurement is the reason target verification and target addressability are two different questions in Section 3.6 rather than one lenient one.

3.5 Locator Mechanisms

A locator is the durable, re-executable rule for finding something by meaning rather than by address. The distinction is the method's centre of gravity: an absolute cell reference such as `B153` is correct exactly until a row is inserted, a column is moved, or a line of descriptive text is added above the table, and when it stops being correct it does not stop producing a value.

3.5.1 Semantic Locators

A single-cell source locator names the artifact role it reads, the sheet by semantic name with accepted aliases, and the cell by the intersection of a row filter and a column header. Four properties make it a locator rather than a search.

The header row is discovered, not assumed. Leading descriptive text above a table does not defeat the locator, because the row that carries the declared column headers is found rather than fixed at the first row.

Identification is by header text only. No geometric alignment of x and y positions is used, so moving a column leaves the locator valid. This is a deliberate narrowing: alignment is the spreadsheet analogue of matching a drawing entity by its coordinates, which the derivation prompt bans outright.

Stacked blocks are scoped before anything else happens. Many real sheets stack several blocks vertically, each introduced by a standalone label row and each repeating the same column headers. A section clause confines header discovery, row filtering and selection to one block. Without it a column header resolves against the first block's header row and a maximum sweeps every block, so the locator returns the right number from the wrong block — and only breaks on the next customer's workbook.

Sheet names are resolved across a set of files, and collisions fail loudly. A rule document says "the shipment detail sheet", not "the third upload", so sheets from all source artifacts share one namespace. Two customers' workbooks both containing a sheet called `Sheet1` is entirely ordinary; that used to resolve to whichever file came later, with a warning nobody read, so the value came from the wrong workbook while every file involved looked perfectly normal. A locator that resolves to a colliding title now fails, a locator that names its artifact searches only that file, and a collision nothing looks up is not an error.

Resolution returns the resolved sheet, cell, selection and match count, so what a locator did is reviewable rather than inferred; zero matches, an ambiguous match, and a missing sheet or column each return a structured error rather than a guess.

Two shapes of block, not one. The header-driven table above is one of the two shapes real forms use. The other is a key/value block — a label in one column and its value in the next, continuing down the sheet — which has no header row to discover and where the row's own label *is* the key. It is supported as a distinct declaration rather than as a special case of the first, and the distinction reaches surprisingly deep: the "is this row blank" test that bounds an extent must consider the whole row for a header-driven table, because the noise that contaminates an extent lands wherever the form had space, and must consider a single column for a key/value block, because the label to its left belongs to a different field and a whole-row test reads past

the end of the block.

Status: implementation-backed on the source side; the label-proximity target mechanism is unexercised on both main carriers. Everything above is exercised by both evaluation datasets. The related mechanism that locates a *write* target by proximity to its caption is not: it is built on the non-agent derivation path, both main datasets take the agent path, and across the 33 specifications examined all 13 label-geometry matches belong to three drawing specifications from an earlier phase of the project. Sections 4.7 (D3) states this as a boundary of what author-time verification has seen; the consequence here is a restriction on what this chapter may claim, and it is a narrow one: labeled-field target location is a case-study capability demonstrated on drawings, not a demonstrated capability on DOCX or XLSX.

3.5.2 Range Locators and Source-Side Aggregation

Almost every business form contains a subtotal, a maximum, or a count. Expressing one requires a locator that denotes not a cell but *a run of rows reduced to a value*, and that single change turns out to generate most of the interesting design in this thesis. The history is worth telling as a sequence rather than as a finished mechanism, because each step was forced by an observation rather than anticipated, and the sequence is itself the argument: **the expressiveness boundary of this IR was pushed outward by datasets, not designed in advance.**

Step 1 — the extent, and why it is the hard part. A range locator adds to a single-cell locator a declaration of which rows it covers and how they are reduced. A single-cell locator asks "which cell?"; a range also asks "from which row to which row" — does it include the subtotal line, the notes row, the blank separator, and is it still right after next month's insert? An extent that is one row wrong produces a number that is plausible and wrong, which is precisely

the failure this thesis exists to make impossible to miss.

Three commitments follow. The extent is always *declared*, never inferred: there is deliberately no whole-column form, and the one open-ended option must be explicitly marked unbounded so that it surfaces in review rather than at the next revision. The reduction refuses the three shapes that hide an error — an extent that selected nothing, an extent containing a row whose text marks it as a total, and a blank or non-numeric cell inside a numeric aggregate. And the resolution records the rows it included *and* the rows it excluded with a reason, so a reviewer who can see the extent can see the off-by-one row that a correct-looking total would otherwise hide.

Step 2 — a run of rows reduced to a scalar is not enough. The first specification derived for the DOCX pair touched table rows one, two and three of a template with fourteen detail slots, because the calibration instance happened to have three detail lines and the agent wrote them out one by one. That is not compile-once; it is compile-per-instance, and the cause was not the prompt but the IR: a range could be collapsed into a scalar, and there was no way to say "the nth slot takes the nth row of the extent". The repair was a row projection — a repeat block over a named extent, expanded into the template's fixed slots — together with a specification scope of one instance's whole output set, so that two documents reading the same detail table cannot disagree about which rows it contains.

Step 3 — a repeat column that can only read one source column is not enough. The next derivation produced a well-formed repeat block whose columns could each read a source column and nothing else, so a quantity that is "per carton $\times$ cartons", a unit price that must be looked up in a product master, and every total over the resulting values were all inexpressible. The agent diagnosed this itself in the specification it returned. The repair moved the evaluation

stage: an extent is resolved *once, into a table* of per-row namespaces; joins add columns from other artifacts; computed columns evaluate over that table in declaration order; and a field's aggregate reduces one of its columns.

The decision worth recording is where the computed columns live. They belong to the *extent*, not to the repeat block that displays it. Two of the DOCX rules are "the packing list's quantity equals the invoice's quantity on the same line" and "both documents have the same number of lines, and line n is the same part". If each document's repeat computed its own quantity, those rules would be satisfied only by two formulas happening to agree, and a divergence would appear as two plausible numbers on two documents that nothing compares. One column, computed once and read by both, makes them agree structurally — and it is also what allows a total to name `extent.column` at all, since a value computed inside a repeat exists in no table anything could aggregate.

Step 4 — a slot placeholder with neither an origin nor a stride. The XLSX evaluation template put the data area at rows 6 to 19, two physical rows per logical record. The slot placeholder substituted the slot ordinal directly, so slots one, two and seven addressed rows one, two and seven: any sheet whose data area does not begin at the first row was unaddressable. The repair gives a block a *layout* — a slot origin, a group size, and named row roles with per-column offsets — so that a slot's coordinate is computed rather than assumed. This defect survived a long time for a reason that is itself a finding: the only dataset that had ever used repeat blocks was the DOCX pair, whose detail slots begin at table row one by coincidence, so the XLSX repeat write path had never run on a real layout and no test could have been red.

Step 5 — two resolved tables that must be joined by key. The monthly report pairs a project register against per-project audit results, and expressing that required an equality merge between

two already-resolved row sets. Before it existed, the only way to pair two tables was to point two repeat blocks at the same slots — an *implicit positional merge* whose correctness depends on both sides having the same ordering *and* the same membership. Removing the second of those, by changing one project number so that one side has a member the other does not, moved every audit figure down one row while both declared-count gates passed and every written cell remained a legal integer. That is why the merge exists and why a separate gate now refuses two blocks that share a slot range without a declared join.

Status: implementation-backed at every step, each with the observation that forced it. Steps 1 through 5 are all on the production path and all exercised by at least one evaluation dataset; the resolved-table representation is used by the derivation-time oracle as well as by execution, which is what gives a detail table any semantic check at all (Section 4.3). What this section deliberately does not claim is generality for the newest primitives: the boundaries of the resulting representation — equality-only inner joins, a column-only aggregation axis, a fixed group size, and a verification basis of one real workflow each for the collection, record and merge primitives — are stated in Section 3.1.4 as B2 to B5 rather than repeated here.

3.5.3 Failure Modes and What Each Mechanism Blocks

Table 5 maps the failure modes of Section 2.3 onto the mechanisms of this chapter. Its purpose is to show that the mechanisms were selected against a taxonomy rather than accumulated, and equally to show where the taxonomy is covered by something other than a locator — three of the six rows are not locator problems at all, which is itself the reason the locator mechanisms are not the whole method.

Table 5: Failure modes and the mechanism that is supposed to make each one loud.

Failure mode	**Mechanism**	**Residual**
Wrong value, right place	Author-time semantic oracle over the expected artifact; the resolved row set makes detail cells comparable	Only references whose text *changed* between baseline and expected are compared (Section 4.7, D4)
Locator drift after a template revision	Structural fingerprint plus the publish gate; stable identities (defined name, bookmark, content control) preferred and positional references flagged	A change beyond the last content-bearing cell moves no fingerprint (Section 3.8.3)
Coverage shortfall — a field no rule reaches, so the template default survives	Externally frozen rule-applicability manifest; every non-covered disposition is a publish gate	Coverage is a derivation diagnosis, not an execution correctness measure (Section 4.6)
Format and unit error	Render formats in the IR; decimal half-up rounding (Section 3.7); presentation scored as its own channel	Only the rounding function has been audited for decimal semantics
Negative-obligation violation — what should be blank is not	Implied blanks derived from row roles, written rather than merely checked; explicit blank-target declarations	A target template that appears in no column is outside the derivation (Section 3.1.4, B6)
Extent error — an aggregate covering one row too many or too few	Declared extents with the three refusals of Section 3.5.2; recorded included and excluded rows; per-range attribution in the evaluator	A hypothesis is only distinguishable where the dataset made it so (Section 5.5.4)

Where the locators currently stand, measured. An earlier draft of this section carried a locator failure rate from the drawing phase — thirty-one mapped rules of which twenty-one had no confirmable write location. That number is obsolete and it flattered nothing: it predates every mechanism in Section 3.5.2 and both main carriers. The current measurements are the rule dispositions of the two best derivations. On the DOCX pair, 33 of 34 applicable rules are covered, 0 rejected, 1 unresolved. On the XLSX template, 23 of 27 are covered, 0 rejected, 4 unresolved. The four unresolved on the XLSX template are informative because none of them is a locator failure: three are negative obligations — rules whose content is "this must stay blank" — whose declarations the derivation could not verify against the expected artifact, and the fourth is a remarks column for which the specification declared no column anywhere, which the execution probe later confirmed produces silent failures.

Status: implementation-backed for the mapping, normative for its completeness. Every mechanism named in Table 5 exists and is exercised, and every residual in the third column is measured and cross-referenced rather than asserted. What is normative is the claim that the six rows *exhaust* the failure modes: the taxonomy comes from the literature and from this project's own incidents, and a mode nobody has hit is indistinguishable in this table from a mode that cannot occur. The bias direction is favourable to the method, and Section 5.10 records it.

3.6 Target Reference and Write-Back

A target reference answers "where does this field write?". All three carriers offer a stable, author-declared identity and a positional fallback, and the design decision is to support both while making the difference visible rather than to support only the safe one:

- **Drawing:** an entity handle, an opaque identity the drawing itself maintains across edits.

- **Word:** a content-control tag or a bookmark name — stable, author-declared, document-global — and, positionally, a table cell addressed by part, table, row and column, or an underscore blank addressed by paragraph and index.
- **Excel:** a defined name — the exact analogue of a handle, since Excel maintains it across inserts — and, positionally, a sheet and coordinate.

Refusing the positional forms would mean refusing most real templates. Measured on a genuine forwarder commercial invoice, the three stable forms addressed the line-item table and nothing else: all eleven header fields produced zero targets, plainly visible in the text dump and unwritable. So positional references are supported and flagged `positional`, and a reviewer can see which targets will not survive a row insert instead of discovering it at the next revision. The captioned blank is flagged `volatile` in addition, because filling one blank renumbers the others on the same line.

Verification and addressability are two questions. "Does something exist at this reference?" and "can something be written at this reference?" coincide for a handle, a bookmark and even an empty Word table cell, which exists in the XML regardless. They do not coincide for a spreadsheet cell, which denotes a position that exists whenever its sheet does and whose normal state in a template is blank. Collapsing them made a *correct* seven-slot repeat block on the real monthly-report template report 41 of its 42 cells as unwritable, and the only way to satisfy the check would have been to point the block at cells that already had content. The two questions are therefore two methods, with addressability defaulting to verification for every format that does not override it. What the spreadsheet override still catches is the failure that matters at block scale — a wrong sheet name, after which every slot lands nowhere and no value-level check sees it. What it cannot catch is a wrong column or a shifted row; neither can the writer,

which writes them happily, so no check on this dump ever could, and that gap is closed by comparing against the references the expected document changed.

Writing back a volatile reference requires a second reading. Filling the blanks in a Word template destroys the references that named them: the survivors renumber, so the produced document answers the original reference confidently with a different field. Discarding the produced document's own volatile references and re-reading them by paragraph coordinates is therefore not an optimization but a correctness requirement — and the coordinate chosen is the paragraph ordinal plus that paragraph's literal text, because those are the only two things both writers preserve. A produced document whose paragraph numbering differs from the template's leaves every volatile reference unresolved rather than resolvable-but-shifted, which keeps the failure loud.

Locating a target is the first of three layers, not the whole problem. It is tempting to treat "the reference resolves" as the criterion for a correct write. It is the first of three, and this project has been wrong at each of the other two.

1. **Can the coordinate be found?** This is what target location and addressability answer. Section 4.7 (D1, D2) records what the author-time side of this layer can and cannot see — in particular that an expected output supplies candidate coordinates that introspection alone does not, and that the Word blank locator is a pilot-supported volatile mechanism rather than a general one.

2. **Does the write preserve the package?** A correct value in a correct cell is still a failed run if the file that comes out has lost the parts that make it the form. Writing through a spreadsheet library's save re-serializes the library's own model of a workbook, so every part the library has no model for is simply absent from the output. On the real monthly-report

template that is the drawing part holding the ruled boxes and stamps, and the printer-settings part holding the one-page layout a filed form must keep. The repair replaces the writer with a package editor: open the original archive, rewrite only the one worksheet part that has to change, and copy every other part through byte for byte, while the spreadsheet library stays in place answering only *where* to write, since merged-range anchors, defined names, data validations and conflicting references all need a spreadsheet model. Measured on five calibration and development instances, the package-integrity channel moved from $0/5$ to $5/5$, and after one write the only changed part is the worksheet.

3. **Does the produced document actually say what was planned?** The compiler knows exactly which references it will write and with what text, so the produced artifact is checked against that plan — every planned reference holds the planned text, and nothing unplanned changed. The last clause is the one that matters, and it cannot be established from the output alone; it needs the baseline to diff against.

The three layers are independent, and the middle one is worth a sentence of its own because it is the only one whose failure was invisible to the evaluation rather than merely undetected: the benchmark's package-integrity channel injects its fault by performing exactly one spreadsheet-library round trip, so while the system under test wrote through the same library, a correct run and a deliberately broken one produced the same class of file and neither $5/5$ nor $0/5$ was evidence about anything. The acceptance criterion for the repair was therefore written as "the two are distinguishable" — the clean output must score full marks and a deliberately round-tripped copy of the same output must score zero — rather than as "the broken one scores lower".

Status: implementation-backed, with two residuals named. The adapter protocol, the addressability split, the volatile rebinding and the package editor are all in the production path, and each has a negative control that makes its own guard fail when disabled. Two residuals: the package editor covers XLSX only, and the Word layout parts have not been enumerated the same way, which matters because the DOCX pair is the only DOCX evaluation dataset (Section 5.10). And the package editor does not decide what *may* be written — preserving every part does not make a value written into the wrong column any less legal.

3.7 Deterministic Compiler and Zero-LLM Runtime

The compiler turns a published specification plus one instance's inputs into an executable write plan and a content hash of it, and returns failures *before* anything is submitted. Its two design commitments are that errors surface at compile time rather than during execution, and that evaluation is restricted enough to be auditable.

A restricted expression language, not a sandboxed one. Formulas are parsed to an abstract syntax tree and evaluated by an interpreter that accepts a closed list of node types: literals of string, number and boolean; names; the arithmetic and comparison operators; a conditional expression; and calls to a fixed function table (`min`, `max`, `round`, `abs`, `sum`, `avg`, `count`, `count_nonblank`). Everything else — attribute access, subscripting, comprehensions, keyword arguments, assignment — reaches a final clause that raises with the offending node type named. A name the context does not provide is an error rather than an empty value, which is what makes it possible for the derivation's trial compile to catch a formula referencing something nothing produces; that check is the single most productive author-time gate in Chapter 4, and it is only possible because the language is small enough to have no fallback.

Rounding: the rule says one thing and the language does another. Every rule document in both datasets says "round half up". The implementation initially used the host language's built-in rounding, which is round-half-to-even, and the diagnosis "switch the rounding mode" was wrong — following it would not have fixed the defect. The measured case is a volumetric weight, $35 \times 25 \times 20 \times 3 \div 10^6$, which the rule, the spreadsheet, and a human all read as the decimal literal 0.0525 and round to 0.053. As a binary double its exact value is $0.05249999\ldots$, which is not a tie at all, so half-up rounding applied to that value still returns 0.052. The value has to be pulled back to the shortest decimal literal that round-trips to the same double before the digit the rule talks about exists to be rounded; only then does half-up quantization give the answer the rule specifies. The repair therefore has two steps, not one, and it disposes of the whole family of cases usually written off as needing arbitrary-precision arithmetic throughout.

The reason this earns a paragraph in a thesis rather than a line in a changelog is that it is a perfect specimen of the primary risk. Both 0.052 and 0.053 are entirely normal-looking figures — right unit, right precision, right format — and every gate in the system passed them. Sweeping the DOCX dataset's 79 detail rows found three disagreements, all volumetric weight, and one of them is in an *evaluation* instance. Left unfixed, a rounding-mode defect in the host language would have been recorded as an error of the method and would have contaminated the main result table.

Human input at execution time: available, and deliberately disabled. A field may in principle take its value from a person at execution time. This does not call a model and therefore does not violate the zero-LLM claim, and it is a reasonable capability for a real system, so the code remains. It is disabled at derivation time for a reason of *evaluation validity*: derivation defines such a field as the residual — text that changed and that no source value explains — so allowing it would let the arm being scored decide which fields it is not responsible for. That is

the same error shape as letting the scored agent enumerate its own coverage denominator, and it is asymmetric across arms, since a per-instance agent has no such escape hatch and simply has to produce a value. The closure is placed at derivation rather than at execution on purpose: closing it at execution broke 34 tests, because the twenty-one stored drawing specifications are built on runtime parameters, and that would have confused "this thesis's evaluation does not credit human input" with "this system does not support human input". Reopening it is an evaluation change before it is a code change: the input manifest would have to declare per field which values are human-supplied, and all three arms would have to receive the identical strings.

The zero-LLM red line. Execution must contain no model fallback: a target that cannot be located must fail and be reported, never be guessed. That requirement, the removal of the execution-time repair branch, and the negative-control test that pins it are stated in Section 3.1.2 and are not restated here. What belongs in this section is the consequence for the compiler: because there is no fallback, every failure mode has to be expressible as a compile-time refusal with a code, which is why the error vocabulary of Section 3.1.5 is as large as it is. A smaller vocabulary would not mean a simpler system; it would mean a system that continues past conditions it cannot name.

Status: implementation-backed. The compiler is a pure function of specification, source data, runtime parameters and current template values, and the canonical job it emits is content-hashed, so identical inputs are recognisably the same compiled work. The restricted evaluator, the two-step rounding and the derivation-time closure of human input are each pinned by tests, and the rounding repair carries a negative control on the specific measured case. Two limits are worth stating: the decimal audit covers the rounding function only, and no other numeric function has been checked for decimal semantics; and content-hash equality establishes that two

runs compiled the same work, not that the produced files are byte-identical, which is a separate check (Section 5.4.5).

3.8 Lifecycle Management

A specification that is correct once is a demonstration. A specification that stays correct across template revisions, is reviewable when it changes, and can be traced from a delivered file back to the exact program that produced it is an artifact an engineering process can accept. This section describes the four mechanisms that make the difference, and it is where the thesis's governance argument against per-instance agents is grounded — not in a claim about regulation, which this thesis does not make, but in traceability, reproducibility and change control, which are properties one can point at.

3.8.1 Lineage and Versioning

Three identities are tracked and related.

A *template lineage* is the family of revisions of one customer template, and it carries the current structural fingerprint of each document role it covers. A *specification* belongs to a lineage, carries a version number unique within it, and records the fingerprint of each template role it was derived against — one per role, because a specification produces an instance rather than a file, so the packing list's template drifting invalidates it even though the invoice's targets are untouched. A *production version* freezes one approved specification together with the compiled artifact and its content hash; those four fields are immutable after creation and the model refuses to save a change to them, so only activation and rollback may move. An *execution run* then references its production version, the exact compiled artifact it submitted, and a unique correlation identifier.

The result is a three-way trace in both directions: from a delivered file to the run, from the run to the compiled program and its hash, and from the hash to the specification version and the template fingerprints it assumed. The published program's identity hash is computed over the field map, the rule projections and the runtime parameter schema, deliberately excluding runtime values, so "the same program" and "the same inputs" remain separate questions.

3.8.2 Regression Suite

Regression cases belong to the lineage rather than to a specification version, so a new derivation is judged against the same cases as the old one. Each case pins its own input template, source artifacts, source-data override and runtime parameters, and optionally an approved expected output. A run compiles the specification against the case's inputs, executes it, and checks the result twice: against the compiler's *plan* — every planned reference holds the planned text and nothing unplanned changed — and, when the case has one, against the approved expected document as a stricter full-text comparison.

The plan check is the load-bearing half, and it is available for every case including those with no approved output. It is also what pairs with the fingerprint: the fingerprint deliberately excludes text, so a caption renamed in place with the same structure does not move it, and the regression's expected comparison is what catches exactly that. The two compose — structural drift is caught by the fingerprint, semantic drift by regression — and a failing regression blocks publication.

3.8.3 Drift Detection

The fingerprint is a stable hash over a template's *structure*: which references exist and, for each, its type, its layer, and its attribute tag where it has one. Text is deliberately excluded,

because text is the payload the pipeline exists to rewrite — folding it into the identity would churn the fingerprint on every production run and invalidate every specification for no structural reason — and because no reliable rule distinguishes a static caption from a value by content alone. Geometry and dump ordering are excluded for the same class of reason.

Measured on both main carriers, because it had only been tested on synthetic drawing dumps. The fingerprint's unit tests feed it hand-built entity lists in the drawing shape, so whether it discriminates on a real document was an assumption. Measuring it on the four evaluation templates — two spreadsheets carrying 27 and 196 references, two Word documents carrying 92 and 106 — gives Table 6, in which every carrier agrees with itself across both of its templates. The controls matter as much as the positives: a library round trip with no edit at all must not move the fingerprint, or the mechanism would report drift every time anything opened the file.

Table 6: Structural fingerprint on the four evaluation templates, two per carrier.

Perturbation	**XLSX**	**DOCX**
Library round trip, no edit (control)	unchanged	unchanged
Caption text edited in place	unchanged	unchanged
Row / paragraph inserted inside the form	**changed**	**changed**
Row / paragraph inserted past the last populated reference	unchanged	unchanged
Block relocated, occupied columns differ	**changed**	**changed**
Block relocated among equally shaped positions	unchanged	unchanged
Headings of two columns exchanged in place	unchanged	unchanged

The bottom four rows are the honest ones, and the corpus of Section 5.6.2 is what let every cell of this table be filled on both carriers rather than left as a dash. On a spreadsheet the fingerprint is a hash over the cells that carry content, because a blank cell is not a target; a template edit that displaces none of them is therefore invisible to it. On the monthly-report

template that means everything below row 25, and the Word carrier behaves the same way below its last blank-bearing paragraph. This is a genuine boundary rather than a bug: the alternative — hashing the declared sheet dimension — would move the fingerprint whenever a cell that once held a value was cleared, since the dimension stays inflated long after the value is gone.

The last two rows are the more serious pair, and both arrive at the same place from different directions: every reference survives and each one now names different content. The design excludes text on purpose, and the mechanism named as covering a renamed caption is the publish-time regression comparison against an approved document — which does not run on the instance path. Section 5.6 carries the consequence, and both boundaries are pinned by tests that assert the run is *not* refused, so that closing either one turns a test red rather than leaving a sentence stale.

A skeleton-level claim this measurement retired. An earlier note in this chapter's outline asserted that drift detection "goes blind the moment a specification uses generic target references", on the grounds that the diff function reads the drawing-flavoured spelling of a target and not the neutral list. Measured across all 35 stored specifications — 585 fields, 582 target references — the diff function sees 582 of 582, and *no* stored specification uses the generic spelling. The published contract, moreover, closes the target object to additional properties, so a specification using it would be contract-invalid: the branch is unreachable for any specification the contract admits. The claim was therefore not so much wrong as untested and pointed at the wrong risk, and the accurate statement is narrower and duller: drift compares fields and repeat blocks, both of which it covers completely, and the neutral spelling is a capability of the reader rather than a shape anything currently produces.

What the same audit *did* find is one unit the diff does not compare at all: a document's blank-target declarations. Two specification versions that differ only in whether a reference

is claimed as deliberately blank produce an empty structural diff, so a reviewer reading the change sees nothing. The consequence is bounded — such a declaration changes no write, and dropping one sends its rule back to blocking the publish through a different gate — so this is recorded as a stated boundary with an entry naming the rejected alternative, rather than as an implementation task ahead of the writing this chapter blocks.

3.8.4 Auditability

Publication is where every mechanism in this chapter is cashed in, and its structure encodes one distinction: which refusals a human may take responsibility for, and which nobody may.

Not overridable: a specification with no fields; a specification failing schema validation; a specification whose declared document roles do not match its dataset's; a specification that did not pass its derivation-time trial compile; and a specification for a multi-document lineage in which some template role has never been fingerprinted. The reasoning differs per item and is worth separating. "This does not compile" is not a judgement call — every run of it fails at the first step, so publishing can only produce a production version that never executes. "Nobody ever fingerprinted this template" is refused because it is indistinguishable in the data from "no template drifted", and publishing on the second reading means publishing with drift detection switched off for that document.

Overridable, with a price: the merged unresolved set, a stale template fingerprint, a failing regression run, and a missing review. A forced publish requires a written reason, and the reason, the operator, the exact list of gates waived, and the timestamp are recorded permanently on the production version and surfaced as a standing warning. The design intent is that overriding is possible — a process that cannot be overridden is worked around — but never quiet.

The merged unresolved set, and the two counters that disagreed. Everything unresolved is merged into one canonical set: items the agent could not resolve, changes in the expected document nothing accounts for, unresolved rule targets, source values whose locator failed or disagreed with the derivation sample, and specification fields carrying no verified target. One property of that merge is worth stating here because it is a general lesson about evidence rather than a detail. The cells a repeat block writes are routed into the "unexplained changes" list on purpose, because that is where the derivation-time oracle reads expected text from and it is the only way a detail table gets a semantic check at all. The publish gate reads the same list to answer a different question — "did anything account for this change?" — and for those cells the answer is yes, the block owns the address. Until that was separated, a *correct* specification carried dozens of spurious blocking items, and no test turned red because what changed was a count. The ownership set is recomputed from the field map rather than read from a tag in the evidence, so a later version that drops the block sends the same cell back to blocking.

The measured state of the two best specifications is 13 blocking items on the XLSX template and 8 on the DOCX pair, against 13 and 14 respectively in the agent's own unresolved list. The two counters therefore agree on one specification and differ by six on the other, and before the repeat-cell separation above they differed the other way, by 34 and 19. Every narrative of "N unresolved" in this project's records counted the agent's list; the merged set is the one that decides publication. Section 4.5 draws the methodological rule: any unresolved count must be decomposed before it is interpreted, because "the gate is too strict" and "the gate is doing its job" call for opposite repairs.

Status: implementation-backed, with the governance claim scoped. Immutability of a published version is enforced at the model, the gate ordering and the forceable set are one pure function testable in isolation, the force audit is persisted, and the fingerprint behaviour in

Table 6 is measured on the real evaluation templates rather than on fixtures. The scoped part is the argument this section supports: the claim is that this architecture supplies traceability, reproducibility and change control, and that a per-instance agent supplies none of the three because there is no versioned program to trace, no compiled artifact to hash, and no diff to review. The claim is *not* that any standard or regulation requires deterministic output; no such source is cited, and Section 2.4 states the requirement in those three words instead.

3.9 Expressiveness Boundary

This chapter has stated boundaries in three different places, which is a hazard rather than thoroughness: a reader who meets a limitation twice in different words cannot tell whether it is one limitation or two. This section therefore does two things — it says where each kind of boundary lives, and it states the one boundary that belongs to none of the other lists.

Division of labour. Section 3.1.4 (B1–B7) states what the *representation* can express: row-content predicates, fixed group sizes, equality-only inner joins, a column-only aggregation axis, what a repeat block clears, and the $n = 1$ verification basis of the newest primitives. Section 4.7 (D1–D6) states what *author-time verification* can see: which coordinates introspection alone yields, which locators are pilot-supported, which mechanism has never executed on a main carrier, which cells the semantic oracle compares, and what the shared evaluator is not. The two are different questions and must not be merged — "a merge does not do outer joins" and "the oracle cannot see a cell that is blank on both sides" are not the same kind of statement, and a combined list would invite a reader to treat them as interchangeable. The present section holds what is neither: boundaries of *execution and lifecycle*, which is to say things the IR can express and the derivation can verify but the running system still cannot promise.

E1 — variable-length write-side detail. Reading is unbounded in the number of rows; writing is not. An instance with seven detail lines does not cause seven rows to be inserted into the template. The asymmetry is deliberate and the argument is blast radius: the read side is confined to two pure modules that take bytes and return values, while inserting rows on the write side reaches the target adapters (reflowing formats, merged ranges, formula references), the comparison logic (two documents with different row counts), the blanking logic, the structural fingerprint, and drift — and, worse, it shifts every positional reference, which means deliberately breaking the thing the system itself flags as fragile. The accepted alternative shape is a template with a fixed number of detail slots of which an instance uses some, the rest of which must be cleared.

That alternative was recorded as a limitation, and it is worth stating plainly that it is no longer only that. The clause "the unused slots must be cleared" turns a negative obligation into a first-class construct, and both halves of that are now implemented: the blanks implied by a record's row roles are *written* rather than merely checked — checking alone would make compliance a property of the template's pristine state rather than of the specification — and a document may additionally declare references it deliberately does not write, subject to the three checks of Section 3.3. So the residual boundary is narrower than the original statement: what remains unexpressible is a template whose row count is a function of the instance, not the negative obligation that the fixed-slot alternative brings with it.

E2 — fields requiring cross-document reasoning rather than lookup. A value that must be inferred from prose in another document, rather than read from a structured source and transformed, is outside the method by construction — this is condition C-II of Section 3.1.6 viewed from the implementation side rather than from the admission side. Nothing here degrades gracefully into it: there is no execution-time model to fall back on, which is exactly the point of the

zero-LLM claim.

E3 — carrier fidelity bounds what any of this can promise. Two limits established earlier in this chapter are properties of the file formats and their libraries rather than of the method, and they bound its guarantees anyway: a spreadsheet dump enumerates only cells that carry content (Section 3.4), and a structural fingerprint therefore cannot see a change beyond the last such cell (Section 3.8.3). The package-preserving writer closes the corresponding *write*-side limit for XLSX and does not close it for DOCX, whose layout parts have not been enumerated (Section 3.6).

Status: E1 and E3 implementation-backed as boundaries; E2 definitional. E1's asymmetry is a design decision with the coupling argument recorded, and the negative-obligation half of its alternative shape is implemented and measured rather than promised; E3's two halves are each a measurement reported above. E2 is a restatement of an admission condition and carries no evidence, by construction. All three feed Section 6.3, which develops them as future work rather than inventing new ones.

Chapter 4. Methodology

Chapter 3 described the specification S and the deterministic execution that consumes it. This chapter describes the procedure that produces S: how a stochastic model proposal is turned into an artifact a deterministic layer has interrogated, what each interrogation can and cannot see, and where a human decision is required rather than optional. The organizing constraint throughout is the thesis's primary risk measure. A derivation procedure that maximizes the number of fields produced is easy; the procedure below is instead arranged so that every way a specification can be wrong is either detected before publication or is explicitly named as undetectable.

4.1 State Machine

Derivation is a fixed sequence of transitions in which a single stochastic step is surrounded by deterministic admission checks. Writing it as a state machine rather than as "call the agent and validate the result" is not presentation: the order of the transitions is itself a result, because three of them exist only because an earlier ordering wasted a paid model call.

Let Φ denote the transition sequence implemented by `derivation_service.DerivationServi`
Given $T^{(n)}_{\mathrm{der},d}$ it produces a persisted draft specification and its evidence, $(S, \mathcal{E}_S)$. The transitions are:

1. **Ground.** Each declared output template is inspected into a structured dump. Dumps are content-addressed and cached, so re-deriving over unchanged bytes does not re-extract

them; the cache disposition per template is reported with the result rather than hidden.

2. **Admit the denominator.** If the dataset carries a rule document it must also carry a frozen rule-applicability manifest. `DerivationService._load_rule_manifest` verifies the manifest against the exact bytes of the rule document and against the dataset's declared document roles, and fails the derive on any mismatch.

3. **Resolve the carrier.** The target kind is computed from the output artifact before the agent is addressed, because the agent is instructed differently for a drawing than for a document, and a `.docx` derive framed as a drawing task is told to call tools that cannot exist for it.

4. **Warm the drawing gateway,** for the DWG carrier only. Probing a CAD gateway for a `.docx` would fail against a perfectly healthy installation.

5. **Propose.** The agent receives the dumps, the staged original source and rule artifacts, the externally assigned rule identifiers, the field-name hints from prior non-degraded specifications of the same dataset, and the declared document roles. This is the only stochastic transition and the only paid one.

6. **Admit the proposal's addressing.** A reply naming a document role the dataset has no template for is refused with `AGENT_DOCUMENT_ROLE_UNKNOWN`. Silently dropping those fields would leave a specification that is merely short, while rule coverage still counted them as covered.

7. **Bind.** The per-document field map, repeat blocks, and shared extents are assembled, and target references the agent did not supply are recorded as unresolved with a cause that distinguishes an infrastructure outage from a rule that was not understood.

8. **Verify the source side.** Every declared source locator is re-resolved against the derivation's own sample artifacts and must reproduce the value the derivation used; failures and disagreements are recorded, not assumed away.

9. **Resolve the detail tables.** Named extents are resolved into row sets against the calibration instance's own workbooks, and their values are folded into the sample data. This costs no model call—it is the same deterministic pre-pass that execution runs—and it is what gives the semantic oracle of Section 4.3 anything to compare inside a repeat block.

10. **Gate structurally** (Section 4.2).

11. **Gate semantically** (Section 4.3).

12. **Repair, bounded** (Section 4.4).

13. **Make the evidence persistable.** The assembled evidence is checked for encodability before the write.

14. **Persist a draft.** Never a published specification, and never nothing.

Two properties of this ordering are load-bearing.

Admission before expenditure. Transitions 2, 3, and 6 all reject inputs for which the agent could nonetheless have produced a plausible answer. They are placed where they are because the alternative was measured: a dataset assembled without a declared output role was rejected in roughly half a second at zero model cost by `RULE_MANIFEST_ROLE_MISMATCH`, whereas the same defect discovered after transition 5 would have discarded the entire paid run. The general rule this encodes is that a check whose inputs are all available before the model call belongs before the model call, even when placing it later would be simpler to write.

Derivation fails to publish, not to produce. No transition after the agent's reply can cause the derive to return nothing. A specification that will not compile is still persisted, with the reason recorded, because a broken draft a reviewer can read is worth more than an error message. What the procedure must not do is let such a draft look healthy, and that is a separate mechanism: everything that survives unrepaired is written into `evidence.spec_validation`, and `gates.canonical_unresolved` turns it into a publish gate (Section 4.5).

Status: implementation-backed, with one boundary that was learned the expensive way. Transition 13 exists because the guarantee above was once violated by the evidence record itself. Evidence is assembled from a dozen sources and written to a JSON column as the last act of a derive; a spreadsheet cell read as a native date-time value is not JSON-encodable, and the exception it raised inside the database write destroyed a completed paid run, leaving not even a broken draft. The repair is deliberately narrow: `derivation_service._persistable_evidence` attempts the encode and returns the evidence byte-for-byte unchanged when it succeeds, converting only on failure and recording in the evidence itself that a conversion happened. An unconditional stringify would have hidden the underlying defect, which is that something is putting live objects into a record. The honest limit is that this guarantees persistence against encode failures that have been *observed*; it does not prove the evidence assembly has no other way to fail.

4.2 Structural Gates

The structural gate is the question "does this specification run?", asked of the deterministic layer rather than of the model that wrote it. Before this gate existed, derivation persisted whatever the agent returned; a specification declaring a lookup transform with no table to look in was

stored as an ordinary draft and discovered much later by a human running an offline checker. The agent was not being dishonest. Nothing had asked.

`spec_check.check_spec` returns every deterministic defect visible without the drawing gateway, as reviewable items carrying a code, the JSON path or field name, and a detail string. It has four parts, in a deliberate order.

Schema and shape. The IR validator rejects malformed documents, fields, extents, repeat blocks, row sets, merges, and record declarations. The IR shape validator alone accounts for 65 distinct `SPEC_*` codes out of roughly 200 prefixed error codes in the feature, and those codes are the vocabulary that the repair loop and the reviewer both read.

Policy. Runtime human input is disabled at derivation time, not merely at execution time, with `SPEC_USER_INPUT_DISABLED`. A field whose value comes from a person at run time is a residually defined field: it lets the party being scored decide which values it is not responsible for. Rejecting it at derivation rather than at execution also matters for the reviewer's diagnosis, because the execution-time symptom is "a runtime parameter is missing", which names the symptom and not the cause.

Source binding. `SPEC_SOURCE_FIELD_UNPRODUCED` reports a field whose declared source key nothing in the specification produces. It runs after the schema check, because a malformed field has no trustworthy source declaration to inspect, and before the trial compile, because the trial compile is precisely what hides this defect.

Trial compile. The specification is compiled against the derivation's own sample data twice: once with every declared runtime parameter supplied and once with only the required ones, which are the two branches an optional override takes. This is the important half. The worst real defect it caught was a formula referencing a name that nothing provided

— perfectly valid syntax, and invisible to any schema check.

The gate is also defended against itself. A compile that raises rather than returns an error is caught and reported as `SPEC_COMPILE_CRASHED` rather than allowed to propagate, for the reason given in Section 4.1: an exception here occurs after the agent has been paid and before anything is persisted, so a single malformed rule would discard an entire run. The behavior is pinned by `test_the_trial_compile_survives_an_exception_it_cannot_anticipate`

4.2.1 What a Structural Gate Catches That an Instance Run Cannot

The argument for spending effort here is that these checks run before any instance is processed and before any evaluation budget is spent. One concrete case makes the argument sharper than the general claim does. Two repeat blocks may write into the same set of slots, each reading its own row set, so that slot n takes its values from the nth row of two different tables. That is an *implicit positional join*, and its correctness depends on two conditions holding simultaneously by coincidence: that both row sets are ordered identically, and that both contain the same members. Remove the second condition — give one source file a project number the register does not contain, without changing the number of files — and every value shifts by one row. Row-set resolution succeeds. Both independent declared-quantity gates pass, because no quantity changed. Every written cell is a well-formed integer. Nothing downstream can see it.

The gate added for this shape, `SPEC_REPEAT_SLOT_ALIGNMENT_UNVERIFIED`, refuses a pair of blocks that share a slot origin and group size while reading different, unrelated row sets. Two things about it are worth recording as method rather than as engineering. First, the failure it addresses was present in a specification that had already passed every other check. Second, its collateral damage was measured rather than assumed: replayed against all thirty-four previously derived specifications it fires on exactly two block pairs, both true positives, and

zero times on the five specifications of the other evaluation dataset.

Status: implementation-backed, with one known ordering defect. When the schema layer reports an error, `check_spec` returns before running the trial compile, and the source-binding check is likewise skipped. The short circuit has a reason — a structurally broken specification has no reliable source declarations, and a compile error stacked on a schema error only restates it in less specific language — but the cost is that one derivation reveals one layer of problems, and seeing the second layer costs another paid run. This is a real defect against the cost argument of RQ3 rather than against correctness; it is recorded as an open item, and the intended repair is to run the source-binding check over the fields that are structurally sound instead of skipping it for the whole specification.

A methodological rule this section is an instance of. Adding a gate changes what the gates in front of it can still observe, and no test turns red when an older protection becomes redundant. Two protections in this system were measured to have become vacuous in their own test suites exactly this way. The discipline adopted in response is that a new gate is accepted only after each older gate it sits in front of has been disabled in turn and its own test confirmed to fail. This is the negative-control requirement of Section 4.3 applied to guards instead of to measurements.

4.3 Two Complementary Oracles

The structural gate establishes that a specification runs. Whether it is *right* is a different and much harder question, and it is asked twice, by two oracles that see different things. Neither subsumes the other, and the argument that they are complementary is the central empirical claim of this chapter.

4.3.1 The Author-Time Expected-Text Oracle

A derivation dataset that supplies an expected output already states what each target reference should read, and the derivation's own sample data is what produced it. The comparison is therefore available at author time, before any instance is processed and at no model cost: compile the specification against the sample, and compare the planned text at each reference with the expected document's text at that reference. `spec_check._expected_text_warnings` implements this.

What makes it worth a section is the class of defect that nothing else sees. Consider a detail-line quantity that the rule document defines as units per carton *times* the number of cartons, written instead as a sum. The two operands are small integers; the sum is 43 where the product is 120, and the amount that follows from it is 550.40 where the correct figure is $1{,}536.00$. Both results are entirely normal-looking numbers: right currency, right decimal places, right thousands separator. The formula is valid, so the structural gate passes it. Before this oracle existed, the first indication would have been an evaluation run, after the money was spent, and only if the scorer itself had no correlated bug. The example is pinned by a test rather than merely described.

Three exclusions are necessary, and under-excluding is the dangerous direction, because every exclusion missed becomes an accusation against a rule that is in fact correct. A field fed by a runtime parameter is excluded, because the trial compile supplies a placeholder rather than the real input. A two-cell row is excluded, because without current values it collapses into a single string. Anything computing from a field that had to be dropped as invalid is excluded, because it is computing from a value the complete specification would have supplied differently.

Two extensions of this oracle are results rather than details.

The key must include the document role. Under multi-document specifications a target reference identifies a cell in *one* file and is not unique across a specification, so the same reference string exists in both output documents. Keyed by reference alone, the oracle told the agent that the packing list's field was writing the wrong value; the repair loop obligingly rewrote that field to read the invoice's key; and the specification was persisted reporting `ok: True` while planning a letter-of-credit number into a shipping-port field. **The system's own correctness check manufactured a silent failure.** This is the strongest single argument in this thesis for the position that a verification mechanism must be verified with the same suspicion as the thing it verifies. The oracle was not absent, and it was not obviously broken. It was half-keyed, and a half-keyed oracle is worse than no oracle, because it actively drives repairs in the wrong direction.

The detail table was structurally invisible until three separate blockers were removed. The detail table is the largest and most computed part of a specification — seven columns across fourteen slots plus six totals, in the DOCX evaluation dataset — and it was the only part with no author-time semantic check at all. Two blockers were known in advance: the comparable-reference map read only document fields, and a repeat cell had no author-time value, because its value came from resolving a locator against a workbook that the trial compile does not have. Resolving extents into row sets at transition 9 removes the second. The third was not anticipated: every cell of a detail table is written by a repeat block, so no *field* ever claims it, so it never appears in the per-field evidence — it lands among the unexplained changes instead. With both known blockers fixed and the third still present, the measurement was that all 140 repeat cells had a real author-time value and the oracle could compare *zero* of them, because the expected side of the comparison was empty. The number of mechanisms that were individually correct while their composition measured nothing is the point; this is the same shape as the

vacuous-guard problem of Section 4.2.

4.3.2 The Case-Level Oracle

The second oracle compiles the specification against a regression case's own inputs and compares the resulting write plan against what that case says each target must read, yielding four verdicts per reference: `match`, `MISMATCH`, `MISSING`, and `OVERWRITE`. `OVERWRITE` is not redundant with `MISMATCH`: it is the verdict for rewriting a cell the case says must retain its baseline text, which is the negative obligation that a purely value-oriented comparison cannot express. References the specification writes that the case does not model at all are reported separately as out-of-scope rather than as failures, so a specification is not penalized for a cell the annotation does not cover.

The two oracles are complementary because they fail independently. The author-time oracle sees only the single calibration instance the derivation was given, so a rule that is correct for that instance and wrong in general — a constant copied from the sample, an extent that happens to end where the sample's data ends — passes it and is caught only by the case-level oracle. The case-level oracle in turn needs annotated cases, which cost human effort per instance, and it runs after the specification exists, so it cannot prevent a defect from being proposed; it can only refuse it afterwards. Their combination is what allows the expensive oracle to be spent on the defects the cheap one structurally cannot see.

4.3.3 Why Not an LLM Judge

The obvious alternative to both is to ask a model whether the produced document is correct. This thesis refuses it, and the reason is specific to the primary risk measure rather than to a general preference for determinism.

Silent failure is an output that is complete, well-formed, plausible, and wrong. The errors a generating model makes and the errors a judging model makes are drawn from correlated distributions: they share a training distribution, a tokenizer's numeric habits, and — if the judge belongs to the same family — a prior over what a filled form "should" look like. A judge is therefore least reliable exactly where the measurement matters most, because the outputs it is most likely to accept are the plausible-and-wrong ones. A judge would also make the measurement irreproducible, which forecloses the run-to-run identity this thesis claims for execution. The same reasoning governs the evaluator of Chapter 5, which is deterministic for this reason and not merely for convenience.

Status: implementation-backed, with three boundaries that must be stated.

1. **The author-time oracle sees only references whose text differs between the baseline and the expected document.** The comparable map is assembled from per-field evidence and unexplained changes, and both collect changed cells only. The consequence is exact, and belongs in the limitations rather than in a footnote: a specification that writes a value into a cell the expected document leaves *blank* is structurally invisible to this oracle — which is precisely the negative-obligation shape that the detail tables of both evaluation datasets depend on. What currently prevents the failure is that a misaligned block will almost always also collide with some cell whose content did change, and that is an accident of layout rather than a property of the design. The intended repair is a separate warning for "the specification claims a target the expected document leaves empty", not an enlargement of the comparison map, because pouring every blank cell of a template into evidence would both inflate it and lose the distinction between deliberately blank and out of scope.

2. **Expected artifacts and the ground truth used to score them share an origin.** Both are produced by a hand-written per-dataset generator from the rule document, which satisfies the red line that expected outputs must not be produced by the system under test. It does not make them independent of *each other*: a misread rule produces a consistently wrong pair, and the manual recomputation that would catch it has been completed for one calibration instance of the XLSX dataset only. This is disclosed as a threat rather than repaired, and the direction of the resulting bias is toward agreement, not toward detecting error.

3. **Any measurement of zero requires a negative control.** A reported "no mismatches" is not evidence unless a deliberate corruption has been shown to make the count nonzero. This is not a stylistic preference: verification probes in this project returned a serene zero while measuring nothing on four separate occasions — from a wrong key name, a wrong field name, a comparison performed over an empty set after an upstream failure, and a control that changed every addition including a string concatenation, so that row-set resolution failed entirely and the result was $0/0$ rather than a disagreement. A control must also distinguish doing the work from not doing it: an ordering test whose expected order coincided with the arrival order stayed green with the sort disabled, and had therefore measured nothing since the day it was written.

4.4 Case-Repair Loop

Both oracles produce failures that a model could plausibly correct. Allowing it to is a research risk and not merely an engineering one: the agent is being shown the expected answers, and "make case 3 pass" is trivially satisfied by a rule that special-cases case 3's numbers and destroys case 2. Such a specification is *worse* than the broken one it replaced, because it now

passes the tests that were supposed to catch it. The repair mechanism is therefore defined by its refusals.

4.4.1 Two Loops, Deliberately Distinct

The derivation-time loop, `DerivationService._verified_spec`, runs inside transition 12 and closes over the structural gate and the author-time oracle. The case-level loop, `case_check.repair_against_cases`, closes over the case-level oracle. They share their anti-overfitting rules and differ in what a regression means, because the case-level loop has a per-case signal and the derivation-time loop does not.

4.4.2 Four Invariants

I1 — a repair may only touch the value layer. The keys an agent proposal may change are enumerated in one place: which formula, which lookup table, which conditional cases, which format, which constant, which source key, which aggregate. Target references, source locators, runtime parameters, and the set of fields are re-imposed from the original afterwards, so a "repair" can never quietly become a rewrite of where a value comes from or where it goes. Anything the agent attempted outside that set is reported rather than applied, which keeps a rejected proposal auditable instead of invisible.

I2 — a deterministic check owns the verdict; the agent only proposes. Every candidate is re-checked in full before it is taken, and a candidate that increases the error count is rejected outright: a repair that breaks more than it fixes is not a repair. Errors dominate warnings in that comparison, because trading a specification that compiles for one that agrees with the expected document but does not run is never the better deal. A candidate that ties on both is nonetheless accepted, since the agent may have rewritten a rule into a clearer form the checks cannot

distinguish.

I3 — at the case level, regression is judged by *which* references fail, not how many. A count is not enough: swapping a failure on one reference for a failure on another scores identically, so a repair could move the damage elsewhere and still be accepted as long as some other case improved. The guard therefore compares failure sets, and a case that already failed to compile is exempt, because otherwise the moment it started compiling would be scored as a regression.

I4 — the loop is bounded and hands off to a human. The derivation-time loop makes at most three attempts and stops early when the agent proposes nothing admissible, since an identical second round would only cost another call. The case-level loop stops after two consecutive rounds without a net gain. The rationale for stopping rather than continuing is substantive: at that point the ambiguous thing is usually the rule document itself, and no amount of additional model effort resolves an ambiguity in a requirement.

4.4.3 A Refusal That Was Correct

One incident is worth recording because it inverts the usual reading of a failed repair. Across two rounds the agent declined to add a source aggregate that the reported errors clearly called for. The diagnosis was not agent laziness: the repair prompt states that only the value layer may be modified, that key was not on the permitted list, and the resolved row set was not included in the repair payload. **The agent's refusal was the correct behavior for the contract it had been given.** The defect was in the contract. This is why invariant I1's permitted set and the repair prompt are treated as a single artifact: a repair loop constrained by two documents that disagree will spend paid calls producing correct refusals, and the symptom is indistinguishable

from a model that cannot do the task.

Status: mixed, and the mixture must not be blurred. The derivation-time loop is implementation-backed and runs on every derive; the four invariants above are pinned by tests, including tests that a repair may not move a target reference, may not invent or drop fields, and may not be accepted when it worsens any single case. The case-level loop is implemented and tested as pure functions, and is reachable through an offline operator tool that runs a stored specification against its cases without the drawing gateway; **it has no caller in the automated derivation path or in the API.** Following this thesis's rule for a gap between claim and implementation, the explicit route is taken rather than the silent one: the case-level repair loop is presented as a designed and tested mechanism whose automated integration is future work, its absence is recorded as an open item, and Chapter 5 must not attribute any measured result to it.

4.5 Human in the Loop: Evidence Review

The economic claim behind compile-once is not that human effort disappears but that it changes units: from reviewing every produced document to reviewing one specification once. That claim is only honest if the review is actually possible, which imposes requirements both on what derivation records and on what publication refuses.

4.5.1 What Is Recorded

A derived specification persists an evidence record with fourteen top-level keys. On the most recent derivation of the XLSX evaluation dataset they include: the agent's own analysis (its unresolved items with causes, its field inventory, its reconciliations, its rule dispositions, and the observed tool-call health), the per-reference changed text and its count, per-field confidence,

field renames, locator verification, row-set resolution, rule coverage, the specification-validation report, unexplained changes, unresolved labels, residual source keys, and version drift against the previous specification of the same lineage. Two of these exist specifically to keep movement from reading as churn: drift reports added, removed, and attribute-level changes, and the rename report identifies fields that are the same field under a new name. This matters because field names drift between derivations — measured twice — so field identity for review purposes is established by target reference, never by name.

4.5.2 What Publication Refuses

`gates.canonical_unresolved` merges every unresolved signal into one set, de-duplicated by kind and reference: agent could-not-resolve items, unexplained changes in the expected document, unresolved labels, source locators that failed to resolve or disagreed with the sample, rule dispositions that are not `covered`, coverage-validation failures, and the specification-validation errors and warnings the repair loop could not clear. Any nonempty result blocks a normal publish. The design intent is stated in the module itself: nothing may be silently dropped into "ready to use".

Publication is gated on that set together with template-fingerprint staleness, regression results, and human review. A human may override the reviewable gates, but the override is not free: it requires a written reason, and exactly what was bypassed is recorded permanently on the published version. An empty or structurally invalid specification is not overridable at all.

Two design decisions inside this gate are worth stating, because both were mistakes first. *Causes are carried, not collapsed.* An unresolved field says nothing on its own about why it is unresolved, so a gateway outage and a misunderstood rule appear identical in the specification while being fixed in completely different places. The run's observed tool health is therefore

folded into each unresolved item's cause, and the review interface says so first and plainly, because a reviewer who read an outage as a coverage regression went looking for a rule that was never missing. *The headline number is* `covered`*, never* `accounted`*.* Accounted is the total minus the unmentioned, so it counts a rule the agent enumerated and then admitted it could not implement as though the work were done: fifteen rules implemented, sixteen reported unresolved, and zero unmentioned, once rendered as a green "34/34". Both halves of that display were wrong in the same direction — the flattering one — which is why the direction of an error, and not only its size, is treated as a reportable property throughout this thesis.

Status: implementation-backed as a mechanism; normative as an economic claim. The evidence record, the merged unresolved set, the forced-publish audit trail, and the review interface all exist and are exercised by tests. What is *not* measured is the quantity the economic argument needs: how long a review takes, how much of the evidence a reviewer actually uses, and whether a reviewer detects a defect the gates missed. No user study has been conducted. Chapter 5 therefore reports human cost only as the count and kind of items a reviewer is required to adjudicate, and the threats-to-validity discussion discloses that this understates human cost, since adjudicating an item is not free.

4.6 Coverage Is the Real Bottleneck

Once a specification's fields are individually accurate, the residual risk is dominated not by fields that are computed wrongly but by rules that no field implements at all. A rule the specification never addresses produces no field, no unresolved entry, and no gate: it is invisible by construction. Measuring that invisibility is therefore a methodological problem before it is an engineering one.

4.6.1 Why the First Denominator Did Not Work

The original implementation asked the agent to enumerate the rule document and counted its enumeration. This cannot answer the research question, for two independent reasons. A rule the agent never noticed is absent from the denominator, so "22 of 22 covered" and "22 of 40 covered" are the same observation; and omitting a rule *raises* the score, which points the incentive the wrong way. Worse, the visible figure was not comparable between runs: one derivation counted 39 mapped fields and the next 28, reading as a large regression, when nine of the 39 were filler the compiler now handles itself and the truth was a small improvement. Unioning inventories across derivations made omissions monotonic but left the evaluated agent choosing the items being counted.

4.6.2 The External Denominator

The active mechanism externalizes the denominator. A rule document must be accompanied by a frozen, versioned rule-applicability manifest that lists rule identifiers, their prose, and the document roles each applies to — and nothing else, in particular no target coordinates, which would make the manifest an answer key. The manifest is bound to the rule document by the SHA-256 of its exact bytes and to the dataset by its declared document roles, and a derive without it is refused before the agent runs.

The agent supplies a *disposition* for each externally assigned identifier and can do nothing else to the denominator: unknown identifiers are discarded, duplicate dispositions are refused, and an assignment whose document roles disagree with the manifest is refused. The resulting partition into `covered`, `unresolved`, `rejected`, and `unmentioned` is mutually exclusive and sums to the frozen applicable set. A claimed implementation is checked, not accepted: a `covered` disposition must name at least one field that really exists in the specification under

a role where the manifest says the rule applies, and otherwise becomes `unmentioned` plus a deterministic issue. Everything that is not `covered` blocks publication; there is no partial credit.

4.6.3 Metric Plumbing Is Methodology

The most consequential finding of this section is not the manifest. It is that a denominator, an allow-list, or an accepted spelling silently decides a reported number without failing anything, and that this failure mode is separate from — and harder to see than — a wrong mechanism. Three measured instances make the point.

1. **A denominator that admitted the wrong checks.** The scoring policy initially counted the "this slot must remain blank" obligations — 120 of them on one instance — inside the semantic denominator. An arm that submitted the blank template unchanged, writing nothing at all, therefore scored 66.7% on semantic correctness. After separating those obligations into their own channel it scores 0%. Every individual check was correct; what was wrong was which total they were added into.

2. **An accepted spelling that decided the headline.** Rule dispositions may name a field either bare or qualified by its owning block. One derivation used the qualified form throughout; the function computing the numerator accepted only bare names, so eleven correct assignments were reported as naming an unknown target, and coverage was displayed as 8 of 27 when the specification in fact covered 19. The specification and the assignments were both correct. A naming convention had decided the thesis's headline diagnostic number.

3. **Declared data that nothing read.** Each range in the benchmark manifests declares, for every wrong-extent hypothesis, the values that hypothesis would produce. The shared

evaluator did not read that field at all. Once connected, an output that occupies one extra detail slot while leaving all six totals byte-identical passes all 112 semantic checks and is nonetheless *named* as a specific wrong-extent hypothesis. The detection capability had been authored, and was simply not wired up.

The rule adopted from these is stated as a verification obligation rather than as advice: whenever a headline figure moves, confirm that the mechanism changed and not the bookkeeping.

4.6.4 What Coverage Does and Does Not Measure

Coverage is a derivation diagnostic. It states which rules the specification claims to implement, verified against an external list; it does not state that an implemented rule is correct, which is the oracles' question, and it must not be substituted for evaluation correctness.

A single coverage number for a dataset is also an incomplete description of the method, and the honest form is a distribution. Four independent derivations of the same XLSX dataset — with the prompt, the payload, and the input bytes identical — produced coverage of 15, 12, 19, and 23 out of 27 respectively, alongside structural differences of the same magnitude: one run declared three detail slots where the template has seven, another declared none of the cross-file collection its predecessor had used, and the number of hard structural errors ranged from four to zero. This variance is not noise around the method's performance; it *is* the method's variance, and the formulation of Chapter 3 predicts exactly where it lives. Because $\mathcal{A}$ is stochastic while $\mathcal{X}$ is deterministic, all of the run-to-run variance of this approach is concentrated in derivation and none of it is in execution. That is the substance of RQ2, and reporting a single derivation's coverage without the distribution would misstate it.

Status: implementation-backed, with the residual gaps named. The external denominator, the four-state partition with its checksum, the pre-agent admission of the manifest, the numera-

tor validation, and the publish gate are all implemented and tested. Three things remain outside the mechanism. First, the manifest is authored by a human from the rule document; binding it to the document's bytes prevents drift but does not prove the human segmented the prose correctly, and no independent segmentation of the rule documents has been performed. Second, the applicable-set hash proves which rules were counted, not that the set is the right one. Third, coverage says nothing about a rule that is covered *wrongly*; that separation is deliberate, and it is why this chapter needs both this section and Section 4.3.

4.7 Boundaries of Author-Time Verification

Section 3.1.4 listed what the IR cannot express. This section lists what the derivation procedure cannot see, which is a different set. Each item is stated in the form that makes it checkable: what is claimed, what is not, and which direction the resulting bias runs.

D1 — an expected output supplies target coordinates that introspection alone does not. When the dataset includes an expected document, the difference between baseline and expected locates the cells that must be written, including cells the baseline leaves blank. That is what makes blank-target binding work at derivation time. It does not follow that an unmarked blank paragraph or cell has a stable semantic anchor when only the template is available: without the expected document there is frequently nothing to distinguish one blank from its neighbours. Derivation-time capability and introspection-only capability are therefore separate claims, and only the first is demonstrated.

D2 — the DOCX paragraph-blank locator is a pilot-supported volatile locator. The mechanism that addresses a blank run inside a Word paragraph resolves against the literal textual context surrounding it in the template. It is supported for the DOCX evaluation dataset and is not

presented as general Word blank-target addressing; a template edit that changes the surrounding wording can invalidate it, which is why the write-back layer distinguishes stable content-control and bookmark identities from positional ones and fails loudly on the latter rather than guessing.

D3 — the label-proximity mechanism has never executed on either main evaluation carrier. Labeled-field resolution is constructed only on the non-agent derivation path, and both evaluation datasets derive through the agent. Of thirty-three stored specifications, the thirteen fields whose target was matched geometrically all belong to three DWG specifications from an earlier period; the DOCX and XLSX datasets contribute zero. This thesis therefore does not claim that labeled-field location works on DOCX or XLSX, and the measurement that recently tightened that mechanism was likewise taken entirely on DWG specifications and does not transfer.

D4 — the author-time oracle sees only references whose text *changed*, as established in Section 4.3. It compares text against text, so a target that is blank in the baseline and blank in the expected document produces no diff entry and is never compared. Two of the three failures that follow from this have since been closed, and the third has not; separating them matters, because the original single statement of D4 was used to justify a protection that turned out to be a coincidence.

The first failure — the specification writes a value into a reference the expected artifact keeps blank — is now detected by a distinct check rather than by the oracle. Derivation records the set of references at which each expected artifact holds text, and any planned non-empty value at a reference outside that set is reported. On a fixture whose repeat block is offset by one row, it names the two cells carrying a project name in a row required to stay blank; the test that pins that fixture previously recorded, in words, that the oracle said nothing about them. Two

silences in that check are load-bearing and are kept: a planned blank never warns, because an unused slot resolving to nothing is required behaviour and warning on it would accuse every specification whose calibration instance is shorter than the template; and a document for which nothing was recorded never warns, because absent evidence is not evidence of a blank.

The second failure — the specification cannot state that it deliberately writes nowhere — is now expressible, in two forms. Where a repeat block declares column positions for only some of its record's row roles, the uncovered positions are *derived* as implied blanks and written blank, with no new key in the representation: on the XLSX dataset this yields 21 cells that the block owns and blanks. Where the cells lie outside every block — a signature box, for instance — the document declares them, and the declaration is checked at derivation time against three conditions: every reference must be addressable on the template, must be blank in the expected artifact, and must not be written by anything else in the specification. A declaration failing any of the three is refused and does not reach the persisted specification, which keeps it out of the coverage numerator; this construct is the only one in the representation that *relaxes* a publish gate, and without those conditions an agent could dispose of any rule it failed to locate by swearing the cell stays blank.

The third failure remains open and is the residual meaning of D4: a reference that neither the expected artifact nor the specification writes, and that no rule names, is invisible. There is nothing at it to compare and no declaration pointing at it. The bias direction is unchanged and is towards accepting a specification: the oracle can only accuse, and a reference it cannot see is never accused.

D5 — an unpublishable specification is not by itself evidence of a failed derivation, and the count must be decomposed before it is read. Two separate findings sit under this heading, and both were closed as defects rather than being boundaries.

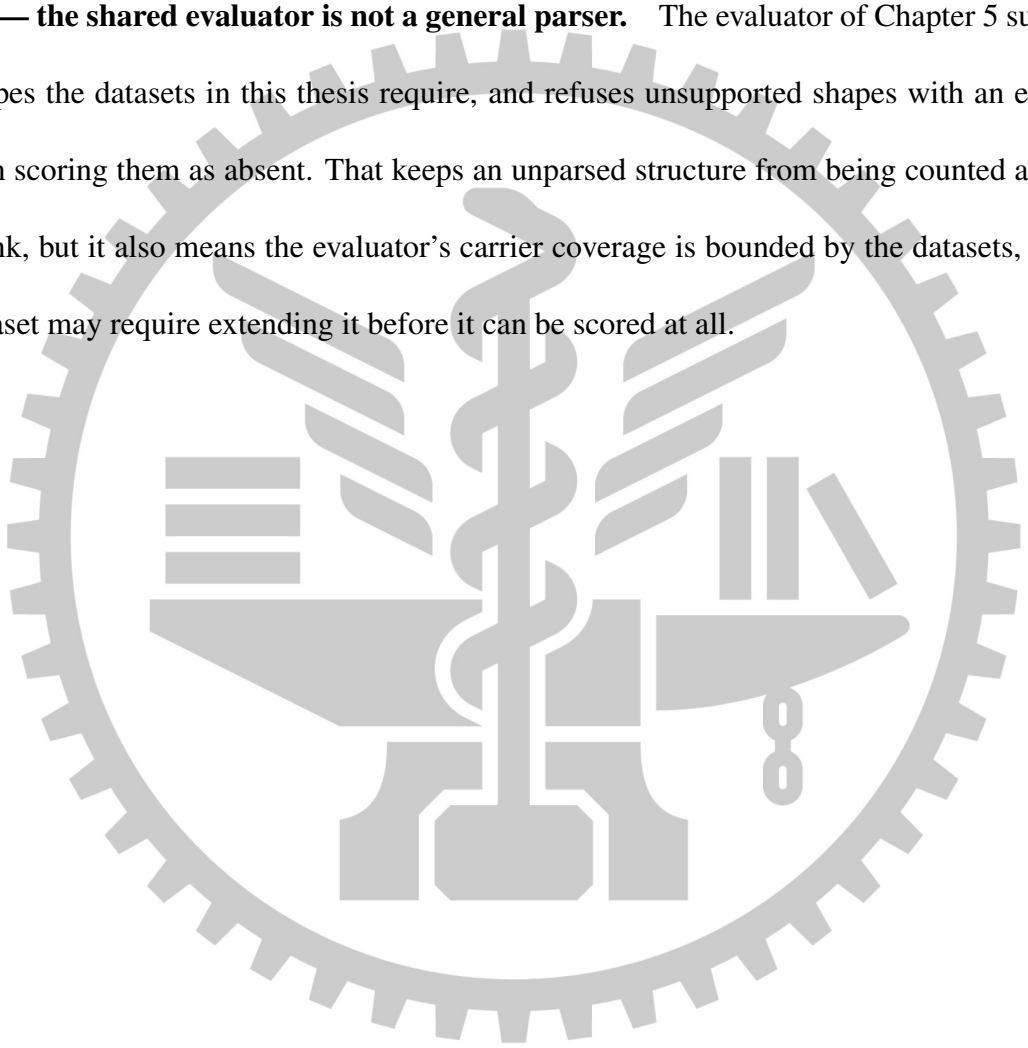
The first is that the unresolved set was computed as "every field without a verified target", which does not distinguish an output field whose target could not be located from an *intermediate* field that legitimately has no target — one carrying a located source value into another field's formula and never written anywhere. The most recent XLSX derivation contained five such fields, each with a working source locator, each read by a header-summary formula, and each recorded as blocking publication. The correction has two halves and the order between them is not optional: intermediate inputs were first admitted to the executable locator set, and only then were they exempted from the unresolved report. The exemption is safe solely because a new hard error now refuses any specification whose formulas read a name that nothing executable can supply. That error was not decoration: replayed across the thirty-five stored specifications it named eleven, and each of the eleven genuinely fails to compile, with no false positive. Before it existed, the two specifications this thesis had recorded as its best both stored a successful validation result and both failed at execution with an unknown formula field.

The second is that two counters reported the same quantity and were never compared. Every narrative of "n unresolved items" in this project's records is the length of the agent's own self-reported list; the publish gate counts a different, canonical set. On the best XLSX specification the two read 13 and 47, and 34 of the 47 were detail cells that the repeat block writes correctly — admitted to the gate's input because the author-time oracle reads expected text from that same list, for a different question. The gate now excludes cells a block owns, recomputed from the specification rather than read from an evidence tag, so that a later version which removes the block puts the same cells back to blocking. No test failed while this was wrong, because what was wrong was a count.

What remains true, and is why D5 stays in this list: after both corrections the XLSX specification is still unpublishable, and decomposing the residue shows the gate is right to block

it. One of the remaining rules requires a value in a column that the specification declares no field for at all, and execution confirms the consequence — three cells scored as silent failures on instances the derivation never saw, because the calibration instance happened to leave that column empty. A count of unresolved items conflates a gate that is too strict with a gate that is doing its job, and the two are repaired in opposite directions.

D6 — the shared evaluator is not a general parser. The evaluator of Chapter 5 supports the shapes the datasets in this thesis require, and refuses unsupported shapes with an error rather than scoring them as absent. That keeps an unparsed structure from being counted as a correct blank, but it also means the evaluator's carrier coverage is bounded by the datasets, and a new dataset may require extending it before it can be scored at all.



Chapter 5. Performance Evaluation

5.1 Research Questions

The claim under test is not that a language model can fill in a form. It is that, for a bounded class of recurring template-instantiation tasks, the model can be removed from the recurring execution path altogether without losing correctness, traceability, or fail-loud behaviour. Four research questions decompose that claim; a fifth asks which components of the derivation procedure carry it.

RQ1 — Correctness. Is a compiled specification's field-level semantic accuracy at least as good as a per-instance agent's on the same instances, measured by the same deterministic evaluator?

RQ2 — Consistency. How much does repeated execution of the same task vary? The question is asymmetric by construction, and the asymmetry is the point: for a per-instance agent the variance is present in every one of the N executions, whereas for the compiled method execution is deterministic and *all* of the variance sits in the single derivation. RQ2 therefore has two halves, and reporting only the first would be a misstatement: run-to-run variation of the baselines, and derivation-to-derivation variation of the compiled specifications.

RQ3 — Cost structure. What is the shape of cost against instance count? A one-off derivation cost plus a near-zero marginal execution cost crosses a per-instance cost at some N; the

question is where that crossing lies, and how sensitive it is to derivation retries.

RQ4 — Behaviour under template drift. When the template changes underneath a frozen artifact, does the method fail loudly and locate the change, and do the baselines fail silently? This is the direct test of the design principle that governs the whole system.

RQ5 — Component contribution. Which of the structural gates, the two oracles, and the repair loop are load-bearing, and which are redundant with each other?

An earlier version of this plan carried a sixth question — whether the method generalizes beyond drawings to office documents. It has been removed rather than answered, because DOCX and XLSX are now the primary evaluation carriers rather than a proof of concept, and a question whose answer is a premise of the study design is not a research question.

A revision to the emphasis, and the measurement that forced it. The original framing put RQ1 first and treated RQ2 and RQ3 as secondary. A pilot measurement on the DOCX dataset changed that ordering. Running a per-instance agent three times over one calibration instance produced semantically correct output every time — the cross-file lookup, the rounding, the gram conversion, and the extent decision were all right — and three different files: three distinct output hashes in three runs, and cell-level agreement of 119 of 172 cells, or 69.2%. All 53 divergent cells were formatting decisions retaken from scratch on each run, a thousands separator present in one run and absent in the next, and none of them was a semantic error.

The consequence is stated here rather than in the results, because it is a claim about what this thesis argues: the primary contribution is not that the compiled method is *more accurate* than a competent agent on tasks a competent agent can do. It is that accuracy is comparable while consistency and cost structure differ structurally. Two tempting responses to that pilot were considered and rejected. Raising task difficulty until the baseline fails would tune the

benchmark towards making the comparator fail; substituting a weaker model would be choosing the opponent. Both are validity threats, and both are recorded as rejected in Section 5.10.

Status: implementation-backed as a measurement, normative as a research plan. The pilot above was executed and its artifacts are retained. The five questions are a plan. The formal experiment has *begun* — nine scored-eligible attempts across three templates and two arms are recorded, and Section 5.8 reports exactly what they do and do not settle — but no template has a complete arm and the compiled arm has not been run at all, so no figure in this chapter should yet be read as an answer to any of the five questions.

5.2 Study Design

5.2.1 Why Not a Few-Shot Held-Out Split

The conventional evaluation for a language-model system is a few-shot held-out split: the model sees K demonstrations and is scored on unseen tasks. That protocol borrows its meaning from in-context learning, and this method is not that shape.

The template is an *input*, not the object of generalization. The system is given the template on purpose; so is every baseline, on every one of its executions. Seeing the template is therefore not leakage but a premise, and it is symmetric across arms. This makes the standard split either vacuous or unfair depending on how it is drawn. If the held-out instances share a template with the demonstrations, nothing was held out that matters: the compiled specification and the agent's demonstrations both describe that template. If the held-out instances come from a different template, the compiled method must derive again — the demonstrations do it no good at all — and the comparison measures the cost of a derivation rather than generalization from examples.

What the split is genuinely needed for is narrower, and is retained: preventing the situation in which one arm's prompts, gates, or intermediate representation were tuned on the very data it is scored on while the baselines were not. The instrument for that is separation at the *template* level, and Section 5.2.2 reports how far that separation actually holds in this study.

Status: normative, and deliberately so. This subsection argues a design choice. It is included because the objection it answers is the first one a reader of a language-model-systems paper will raise, and because declining a standard protocol without stating why reads as avoiding it.

5.2.2 Two-Level Split

The split has two levels, and only the first of them currently holds in full.

Instance level. Every evaluation template carries $K = 3$ calibration instances, shared by all three arms and the only instances any arm may see during construction; two development instances used for debugging; and M evaluation instances that no arm may see at any stage after sealing. S4 contains eight refusal cases of thirty, X1 seven of thirty, and S5 four of thirty. Thus every dataset tests the case in which producing a plausible file is itself the wrong behaviour.

M was set to 30 on 26 August 2026. It had never actually been decided — 5 was a placeholder carried from the protocol's first draft — and the measurement that settled it is in Section 5.10.3: the silent-failure rate is a proportion over cases, so at $M = 5$ a template reporting no silent failures still admits a true rate of 45%, against about 10% at $M = 30$.

All three datasets have reached thirty: X1 on 26 August 2026, S5 on 28 August and S4 on 29 August, for ninety evaluation instances in the main table. This had to be finished before the formal run and could not be done after it, because adding evaluation instances once an arm

has been scored on them is indistinguishable in form from selecting them.

Reaching thirty changed the *genre* of an evaluation set, and that is a methodological change rather than a larger version of the same thing. Thirty hand-designed scenarios would be twenty-five stories invented to reach a number. So each enlarged dataset keeps its five designed instances, each pinning one capability, and draws the other twenty-five from a distribution and a seed written down before the draw; what a reviewer checks is no longer whether each case was a good idea but whether the set on disk is the one that seed produces, which is mechanical. Section 5.2.3 states the conditions this puts on a dataset and what it costs.

Doing this three times showed that the stratification is where the judgement lives, and that it has to be argued per dataset rather than copied: the recipe transferred unchanged — the four auditability conditions, the plan file, the seeded tie-break, the shape-based rerun rule — and the strata transferred not at all. X1's strata are its two refusal codes against success. S5 has one refusal code and a richer failure structure instead: its qualification rule is a conjunction of prerequisites, three core groups, three elective thresholds and a credit total, so an instance can fail for several reasons at once and a failing arm would not say which rule it missed. Four of its five not-qualified strata are therefore required to break exactly one clause and leave the others intact, and the dataset's own oracle — the same function that writes the ground truth — is what enforces the label rather than the sampler asserting it.

The fifth stratum is the interesting one, because it was added after measuring the first draw and it is deliberately *not* isolated. Taking one course in every group except the dropped one fills seven or eight of the form's eight slots, so the first twenty-five instances came out at eight slots on twenty-three of them — leaving the negative obligation that unused slots must be cleared with almost nothing to bite on anywhere in the evaluation set. A student applying partway through the programme breaks several clauses at once, and is the only shape that leaves four

to six slots empty; one such instance uses a single slot, which is the only configuration under which the blank form's two worked example rows survive for an arm that did not clear them. A declared distribution makes this kind of hole visible before the run rather than after it, which is the argument for the genre and not merely a convenience of it.

S4 stratifies on neither codes nor clauses but on *detections*, and the difference is not cosmetic. It declares three refusal codes covering five distinct shapes, two of which share a code: the rule that forbids substituting a default for a missing price fires both when the product master file is absent altogether and when one part number in an otherwise complete master has no row. Those are different things for an arm to notice — a missing file against a lookup that fails on one line — so they are separate strata under one code. Designing them exposed two shapes that had been declared and produced by nothing. The refusal for an undeterminable extent was named in the annotation manifest and carried by no instance in the dataset's history, because the source writer always emitted the terminating total row; and the rule that writes a bare carton number where a line occupies a single carton had never been exercised, in zero of the seventy-nine detail rows the dataset contained. A declared code that no input can reach is a check whose test is green because nothing arrives at it, and neither of these would have turned anything red. The evaluation set now produces both, and a mechanical check requires every code the manifest declares to have a producer.

All three datasets are now sealed — and sealed by mechanism rather than by convention, which they were not before 26 August 2026: their generators rewrote every case unconditionally, evaluation included, while three documents described them as sealed. A shared guard now pins each evaluation file by digest and refuses to regenerate it. X1's and S4's seals were closed first, because their thirty exist and their membership is settled; S5's stayed open until its audit was recorded, because recording one rewrites every ground truth with its audit block. On 30

August 2026 all three were superseded by closed seals carrying a review digest, so no open seal remains anywhere in the repository. The word “sealed” therefore does two jobs that this chapter keeps apart: content frozen, and content checked by a person. *All three datasets now have both*, and the second is weaker than the annotation procedure specifies — Section 5.10.3, T12, states exactly how.

Closing S4’s seal required rewriting bytes that a seal already pinned, and the reason is worth stating because it is the only such case. X1’s enlargement fell entirely in modules that no ground truth pinned, so its five earlier instances survived byte-for-byte and its determinism repair could be forward-only. S4’s oracle, its five writers, its recorded human spot audit and its main loop all live in the single file that every one of its ground truths pins by digest, so a forward-only repair did not exist: the choice was between changing those bytes and permanently carrying four defects, among them a generator that was not byte-idempotent, which is precisely the condition that makes a seal on presence useless. The bytes were changed. What licenses that is not the seal file — which cannot establish an ordering — but the protocol’s actual condition, that instances be added before any arm is scored on them. That condition held when the bytes were changed on 29 August: no arm had been scored on any S4 evaluation instance, and the first that was came a day later, from a commit that postdates the reseal. The ordering is therefore checkable in the history rather than asserted here. Value-neutrality was then proved rather than asserted, by regenerating the ten pre-existing instances in a mirrored tree and comparing every ground-truth key and every decompressed package part: two hundred and thirty declared differences, none value-bearing, with all sixteen expected documents identical in every package part and the dataset’s one human spot-audit record carried through unchanged. The superseded seal is recorded by digest.

One measurement from closing the first seal is worth recording here, because it is about

the guard and not about the data: while every seal was open, the branch that refuses a new evaluation instance was structurally unreachable, and the first time a seal was actually closed it rejected a *calibration* instance — a check that had never once run in the state it was written for. A check that rejects a correct action is worse than no check.

Template level. Three evaluation-template artifacts now exist, all three at thirty evaluation instances, all three with their content sealed, and all three carrying a signed human audit as of 30 August 2026 — a review weaker than the annotation manual asks for, recorded as such in the audit file itself and reported in T12:

- **S4** (DOCX): a Commercial Invoice and a Packing List produced together from a product master file and a shipment detail file. Its principal difficulty is cross-file computation — unit price from one workbook, quantity from another — and it is the only DOCX evaluation dataset, so every DOCX claim in this thesis rests on it. Thirty-four applicable rules. Five designed instances and twenty-five sampled at $M = 30$ as of 29 August 2026, sealed closed: twenty-two successes and eight refusals spread over all three declared codes, with two to fourteen of the fourteen detail slots used across ten distinct detail-row counts. It is the one dataset whose enlargement had to rewrite already-pinned bytes, for the reason given above.

- **X1** (XLSX): the Directorate General of Highways monthly site-audit report chain, in which N real 13020A audit sheets are aggregated into one 13020D monthly report. Its principal difficulties are a cross-file collection of source artifacts and a layout in which one logical record occupies two physical rows. Twenty-seven applicable rules. Five designed instances and twenty-five sampled at the decided $M = 30$, sealed closed, with N ranging from one to sixteen source sheets across the thirty.

- **S5** (XLSX): a faithful spreadsheet transcription of National Taiwan University Public Health's circulating Health Big Data Credit Programme certificate form. It has eight fixed course slots and combines course lookup, repeat resolution, approved credit substitution, category thresholds, prerequisites, course-code history, stale-slot clearing, and a missing-source refusal. Nineteen applicable rules. Five designed instances and twenty-five sampled at $M = 30$ as of 28 August 2026: twelve qualified, fourteen not qualified and four refusals, with one to eight of the eight course slots used. Its three calibration and two development literals, independent oracle, output-only aggregate cross-check and eight-channel mutations are built, and its thirty evaluation instances are frozen by a content seal; all thirty-five of its stored cases were reviewed on 30 August 2026. *Those are two different seals*, and the distinction is load-bearing: freezing the bytes had to happen first, because an audit cannot cover a set that is still being added to, so for two days a dataset in this study carried a seal that did not mean "a person has checked this" — and for those two days all three did. Three mechanisms that had been reading the seal file's existence as though it did were changed to read the recorded review digest instead — the one that mattered would otherwise have gone quiet without turning anything red.

Two further items must be named, because their absence changes how the split reads. **S3** is a synthetic pilot dataset that was built to protocol and then rejected on the grounds that its layout and content were not those of a document that really circulates; it is retained only as a pilot for the evaluator and the range locator, is not in the main results table, and is not offered as evidence of external validity. **S1** and **S2** appear in the protocol document as a proposed assignment of two development templates and *were never built*.

The gap this leaves, and the direction of its bias. Because S1 and S2 do not exist, development did not happen on templates disjoint from the evaluation set. It happened on the DWG

case study, on the rejected pilot, and — decisively — on the calibration instances of S4 and X1 themselves. Three intermediate-representation primitives were added specifically so that the compiled arm could produce X1 at all: artifact collections, stacked records with a slot origin, and equality merge between resolved row sets. X1 is an evaluation template. The template-level separation the protocol describes is therefore, at present, a normative claim rather than a property of this study, and its bias runs *in favour of the proposed method*.

Following the third of the three admissible responses to such a gap, the claim is kept and labelled rather than quietly weakened, and the bias is disclosed again in Section 5.10. Two measurements bound its size without cancelling it. First, each primitive was added only after it had been observed to fail on a real file, and its collateral effect on the already-stored specifications was swept: the slot-alignment gate introduced with the third primitive was replayed against all thirty-four stored specifications and matched exactly the two it was written for, with no false positive on the five specifications of the other dataset. Second, value-level verification of the resulting specification was performed on development instances the derivation never saw: the fourth X1 specification reproduced 255 of 255 scored values across the three calibration and two development instances using the scoring policy's own comparator, and a deliberately corrupted variant of the same specification scored 240. Neither measurement makes the development location correct; they bound how far it could have been exploited.

A second construct-validity item, recorded here and again in Section 5.10. X1's rules require projects to be reported in ascending project-number order. The real 13020D form prescribes no ordering; the ordering is a convention this dataset introduced so that the extent has a checkable manifestation. The mechanism that implements it is built and tested, but the question of *who decided the order* belongs to dataset construct validity rather than to the method.

Status: implementation-backed at the instance level, normative at the template level, with the bias direction stated.

5.2.3 Dataset Construction

This subsection states what an evaluation dataset must satisfy, as conditions that can be checked before the dataset is used rather than inferred from the artifacts afterward. The ordering was deliberate: these conditions were frozen before S5 was built. All three datasets are now checked against them, and all three passed the last of them on 30 August 2026, when their content — frozen since 26–29 August — was reviewed by a person and the review recorded in each seal. The checklist is therefore the exact boundary between "constructed" and "formal-ready", and it is the boundary the datasets crossed last, three days after the mechanical conditions were all green.

That ordering also guards against a specific failure of this kind of writing. A constraint written after the artifacts it governs tends to be satisfied by construction, and a constraint that nothing has ever violated has not been shown to carry any force. Every condition below is therefore reported together with the result of checking it against all three built datasets, and two of the conditions are not satisfied in full.

Three artifacts per case, and the direction in which they are derived. Each case is produced by a hand-written generator that emits an input set, an expected artifact, and a structured ground truth. The expected values are computed from the rule document and the source data. They may never be produced by the system under test: an expected artifact manufactured by the production compiler would make the comparison circular and structurally incapable of failing.

The checkable form of that red line is narrower than "the generator shares no code with the product", and the narrower form is the correct one. Neither built generator imports anything

from the product: both draw only on the standard library, the two document libraries, and their own local modules. But the XLSX validator does import the product's source locator, on purpose, to check that the generated inspection sheets are machine-readable at all — and it labels that import as a readability probe whose output feeds no expected value. The condition is accordingly stated as *no expected value may be produced by product code*, with a product import admissible only where nothing it returns reaches the ground truth.

Independence also has to be substantive rather than nominal, and here the two datasets give a positive demonstration rather than an assurance. The DOCX oracle rewrites paragraph text at the string layer, where the product's adapter splits and rewrites runs; the two agree on nothing except the observation that three or more consecutive underscores mark a blank, which is a property of the template rather than of either implementation. The XLSX oracle edits the one worksheet part directly inside the package, where the product — before the change described in Section 5.5.1 — rewrote the whole workbook through a spreadsheet model. The consequence was measurable, and it is the second reason the oracles are separate: a round trip through that spreadsheet library drops `xl/drawings` and `xl/printerSettings` from every one of six real government workbooks, so an oracle built on it would have failed the package-integrity channel itself, and the channel would then have been measuring the benchmark instead of the arm. A correct answer has to be able to reach every channel that is scored.

Two levels of annotation, and why the split is a correctness measure. Annotation is split into one manifest per template, holding every judgement about what counts as correct, and one ground truth per instance, holding only values and the record of how they were checked. The reason is not tidiness. Twenty-eight instances each repeating the same judgements is twenty-eight opportunities for two instances to be scored against different standards, and an inconsistency of that kind appears on the main results table as a difference between methods. Deciding

once and referring to the decision removes the possibility rather than reducing it.

The scale the split manages is easy to understate. The DOCX manifest declares 28 scalar fields and three slot groups covering 154 slot cells; the XLSX manifest declares four scalars and two slot groups covering, coincidentally, also 154 slot cells. Expanding those into per-cell annotations on every instance would produce several thousand hand-written entries, and a slot-group declaration — a location pattern, a column list, a slot count, and an obligation for unused slots — replaces all of one group's entries with one.

One procedural rule keeps the split from silently absorbing errors: an annotator may not adjudicate anything the rule document does not state. The prescribed response to an ambiguity is to amend the rule document, record the amendment in the manifest's notes, and regenerate. This is not hypothetical bookkeeping. Building the DOCX dataset found three statements in its rule document that did not match the real form, one of them naming a position that does not exist, all found by diffing the rules against a structural dump of the template rather than by reading. Settling such a case inside the annotation instead would make the benchmark a test of guessing what the rule author meant, and the three arms do not have equal access to that knowledge.

Condition 1 — the aggregated set must vary in size, and one case must contain a member the correct extent excludes. If every case aggregates the same number of records, a specification that hard-codes an absolute row range passes the entire suite, and the extent machinery is never exercised. The condition therefore has two halves: the cardinality of the aggregated set differs across cases, and at least one case — not the earliest — carries a member that the correct extent must leave out.

The DOCX dataset satisfies both. Its detail extents run 3, 6, 9, 8, 14, 2, 14 and 5 records across the eight non-refusal cases, and four of those cases carry excluded rows classified as

blank or note, with the three calibration cases deliberately clean so that contamination first appears where an arm cannot have tuned on it.

The XLSX dataset satisfies the first half at a different level and does not satisfy the second at any level. Its declared extent is rows 8 through 60 of an inspection sheet — 53 rows, identical in all ten instances, with zero excluded rows in every one, because the real form fixes the checklist length. What varies is the number of source artifacts aggregated, from four to ten inspection sheets per case. That is a genuine instance of the same constraint one level up, and it is why the condition is stated over "the aggregated set" rather than over rows; but no case in that dataset contains a member the correct extent must exclude. The substitute is described under Condition 3, and it is not equivalent.

Condition 2 — every declared wrong-extent hypothesis must be distinguishable on at least one sealed instance. "The cases must be well designed" is a wish until it is an assertion a program can evaluate. The mechanism is that the manifest declares, for each extent, the specific ways of getting it wrong that could plausibly occur there — a hard-coded row range, stopping at the first blank row, taking the last non-empty value, taking the whole column, swallowing a subtotal row, swallowing a note row — and the generator computes what each of those would have produced on each instance and records whether the result differs from the correct one. A dataset is complete only when every declared hypothesis is distinguishable on at least one sealed instance.

Both datasets satisfy this, and both satisfy it with a margin of exactly one. In the DOCX dataset six hypotheses are declared, and `note_row_included` is distinguishable on one evaluation instance only. In the XLSX dataset five are declared, and `hardcoded_rows` is distinguishable on one evaluation instance only. A margin of one is not comfortable, and it is recorded rather than smoothed: removing a single sealed instance from either dataset would

silently reduce its extent coverage.

The condition has a negative half that binds equally. A hypothesis that can never be distinguished must not be declared, because declaring it claims a coverage the dataset does not have. The XLSX dataset deliberately omits `note_row_included` for that reason: a pure-annotation row on its source form contributes zero to every field the target form reports, so no scored field could move. The omission is recorded in the dataset's own documentation together with the argument, which is the form in which a negative decision stays auditable.

Condition 2 is also where dataset design touches this thesis's primary metric most directly. In the DOCX dataset, four of the six declared hypotheses leave all six totals byte-identical, because the totals row and the note row have empty quantity cells and absorbing them changes no sum. The only signal that anything went wrong is that one extra detail slot is occupied. That is precisely the silent-failure shape the thesis is about: every number in the document is correct and the document is still wrong, and no amount of checking the arithmetic finds it.

Condition 3 — every aggregate needs a detector computed from the output alone, and zero declared checks require an auditable reason. Extent correctness is the one channel an arm could report on itself, and Arm C can: its resolved extents are explicit. Arms A and B produce a document and no provenance. Scoring a channel from the scored system's own declaration would repeat the mistake the coverage denominator once made, so the instrument has to be computed from the output.

The annotation manual and the manifest schema state the condition in its strongest form: every sum-type aggregate field must be covered by at least one cross-check, an identity between two independently scored fields of the same output. The shared dataset lint now enforces aggregate coverage and validates each expression against its declared fields. The DOCX dataset satisfies the condition: seven checks cover all six totals. The XLSX evaluation dataset declares

none because its target form lists one row per project and has no output aggregate. That zero is no longer indistinguishable from omission: a versioned evaluator-policy artifact records the structural reason, shared lint accepts it, and deleting or corrupting the reason makes lint fail. The policy is external to the frozen annotation manifest, so adding this enforcement did not rewrite any sealed truth digest.

The consumer is carrier-neutral and arm-symmetric. It parses only the frozen expression form, resolves scalar values and slot-column totals from the produced artifact, evaluates with decimal arithmetic and the manifest tolerance, and never reads an expected value to fill a missing operand. An unknown identifier, a missing or non-numeric output value, or disagreement between an expression and its declared field list produces a failed check or a lint error rather than a skipped denominator. The resulting `crosscheck/<id>` measurements form an eighth, independent correctness channel; they are not folded into semantic accuracy. A controlled mutation in which a total is changed by one leaves the artifact structurally sound and its detail semantics correct, but changes the case outcome to `silent_failure` through this channel.

The boundary remains important. Cross-checks detect an inconsistent aggregate; they cannot detect compensating detail errors whose sum is unchanged, nor a detail and total that were both computed from the same wrong interpretation. Extent hypotheses remain necessary for the former class of zero-contribution boundary error, and independent annotation audit remains necessary for the latter. On the pilot XLSX dataset, three scalar identities are output-scoreable while three identities over source-only ranges remain explicitly truth-side-only; the policy records that boundary instead of letting the evaluator substitute ground truth.

Condition 4 — the annotation format needs six capabilities, not the four the protocol lists. The evaluation protocol enumerated four things a flat mapping from field to value cannot express: numeric tolerance and format; negative obligation, meaning a position that must be

empty; extent, recorded row by row rather than as a start and an end; and the separation of semantic value from presentation. Annotating the first datasets added a fifth, because an output can satisfy all four and still be wrong — a document that writes every scored field correctly and blanks the rest of the template scores full marks under those four. The fifth is therefore a policy over every addressable position that no field claims, compared against the template.

Checking that count against the frozen schemas rather than against the protocol shows that it is now six. The sixth is refusal: some instances have no correct document at all, because a required source is missing and the rules mandate a hard failure. The ground-truth schema enforces the shape structurally — an instance declared a refusal must name a code drawn from the manifest's declared list, must carry no values and no slots, and must not carry an expected artifact — and scoring evaluates the presence of a file before it evaluates any content. This is not reducible to the second capability: an empty cell inside a produced document and a document that must not exist are different assertions, and only the second gives fail-loud behaviour any evidence in the main results table.

A seventh candidate is deliberately excluded, and the exclusion is a decision rather than an omission. Package integrity is scored, but the list of package parts that matter is a constant of the scoring policy, not a per-template judgement. Whether an embedded drawing matters is not a question each dataset should re-answer, because two authorities for one question drift in the direction of "passes here, fails there".

Condition 5 — at least one sealed instance whose correct behaviour is a refusal. Both datasets carry two of five. Without such an instance the fail-loud design principle has no representation in the main table at all, and could only be inferred indirectly from the drift axis.

What is not satisfied: the human half of the procedure. Two of the annotation manual's requirements are procedural rather than structural. One has now been met in part and the other not at all. The manual requires that a second person independently redo the judgement step for each template, and that each instance receive a spot check of at least three fields recomputed by hand from the sources, including one aggregate and one negative obligation.

The spot check was signed on 30 August 2026. Counted across all one hundred and thirteen stored instances of the four datasets, forty-two now name a reviewer — three DOCX instances, four of the first XLSX template and all thirty-five stored cases of the second, with none of the eight pilot instances — so against the bound relaxed on 26 August from every instance to three evaluation instances per template, the count is three of three, four of three and thirty of three. *By method it is met nowhere.* What the manual asks for is an independent hand recomputation performed without opening the expected artifacts, and what was done was a per-case read-through of each ground truth against the rules with the arithmetic checked, forty-two rows in one hour. The two instances that carry the manual's own procedure — six fields and twenty-five minutes each, one on the DOCX template and one on the first XLSX template — are the same two that carried it beforehand, and that count did not move. The distinction is recorded in the audit file and hashed into the seal rather than left to this paragraph.

The second requirement, the independent second reader, has not been met on any template: *zero of three*. It is not a weaker version of the first and cannot be discharged by doing more of it. A single reader who transcribed the rules and then checks work derived from them is checking their own reading, however carefully; the error class below is precisely the one that survives that.

This is the residual no other mechanism reaches, and it is worth being precise about why. Schema validation catches shape errors. Cross-checks catch a total that disagrees with its own

detail. An independent recompute probe — which exists for the XLSX dataset, reading the written workbooks rather than the case data — catches transcription and addressing errors, and says so in its own documentation. None of them catches a misreading of a rule that the generator and the ground truth share, because every number is then wrong together and every relation between the numbers still holds. The one spot check performed on the DOCX dataset under the manual's full procedure found exactly that class: a per-piece net weight declared in grams where the rules require kilograms, whose omission yields sixty thousand instead of sixty and passes all seven cross-checks. That it was the stronger procedure which found it, on one of the two instances that received it, is the argument for the requirement rather than an anecdote beside it.

Following this thesis's rule for a gap of this kind, the third route is taken: the requirement is kept at its original strength rather than lowered to what was performed, the shortfall is reported as a number rather than described as partial, it is carried as an open item, and its direction is disclosed in Section 5.10. That direction is genuinely undetermined rather than conveniently so. All three arms read the same rule document and all three see the same calibration instances with their expected outputs, so a shared misreading penalizes whichever arm reads the rules correctly, and nothing in the protocol says which arm that would be.

Acceptance criteria for the third template. Restated as a checklist, because it is what S5 is checked against: a template that really circulates, retained with its provenance and either unmodified or faithfully converted with the conversion boundary recorded; a rule document containing no write coordinate of any kind; a generator that imports no product code except in a probe that feeds no expected value; expected artifacts recorded with their digests and able to reach every scored channel including package integrity; three calibration, two development and M sealed evaluation instances; at least one sealed refusal with a declared code; aggregated-

set cardinality varying across cases and at least one non-earliest case containing an excluded member; every declared wrong-extent hypothesis distinguishable on at least one sealed instance, and no hypothesis declared that cannot be; every aggregate covered by at least one output-only detector, with a cross-check wherever the target form makes one expressible; all six annotation capabilities exercised; and the spot check performed and recorded on every instance rather than on one.

S5 satisfies every mechanical and artifact-existence item before the final semicolon. Its sources are four publicly downloadable files from the same programme, pinned by SHA-256 and retained with the public-access-but-no-open-licence boundary; the circulating DOCX form is transcribed without adding a business field. Its rules contain no write address. Nine success oracles preserve the XLSX package while the refusal produces no artifact. The shared linter reports zero problems, all ten oracle cases score cleanly, every success has one actual-output cross-check, all three wrong-extent hypotheses discriminate on at least one evaluation case, and one material mutation turns each of the eight scoring channels red from a green baseline. The last item was procedural and deliberately could not be discharged by any of those results: a signed human audit and the seal made from it. Both were recorded on 30 August 2026, over all thirty-five of the dataset's stored cases, by the weaker of the two review procedures the annotation manual describes — T12 states which one and why the difference is load-bearing.

A sampled evaluation set, and the four things that have to be checkable about it. The last item of that checklist — the spot check on every instance — was relaxed on the same day M was raised, and the paragraph above already reports it as unmet. Raising M also changes an earlier item. "Three calibration, two development and M sealed evaluation instances" is a statement about counting when the instances are designed one at a time; once twenty-five of thirty are drawn, the construction has to be auditable as a *sampling procedure*, and four things

replace “somebody chose what this case tests”:

1. the declared size and membership of the evaluation set match what is on disk;

2. the distributional conditions the dataset claims hold over the draw, with every threshold stored in the declaration rather than in the checking code, because a threshold that lives in the checker is one the checker can quietly relax;

3. the instances on disk are the ones the declared seed produces — verified against the ground-truth files and not against the sampler’s own output, since a sampler agreeing with itself establishes nothing;

4. the instances designated for consistency reruns are selected by a rule over instance *shape* and can be recomputed from the sealed dataset.

Each is implemented as a validator check with a negative control that turns it red. Three consequences of doing it this way are worth stating because they are not improvements.

The declaration cannot live in the annotation manifest. Every ground truth pins that manifest’s digest, and most of them are sealed, so adding one key to it would require rewriting sealed instances — which is the one thing a seal exists to prevent. It therefore lives in a sibling file that the seal pins separately. For the same reason the ground-truth schema does not record that an instance was drawn or from which seed: a reviewer holding a single ground truth cannot see its provenance, and only the dataset as a whole carries it.

The fourth condition needed a detail that looks like fussiness and is not. Ties in the selection rule are broken by a seeded permutation rather than by instance identifier, because the five designed instances hold the five lowest identifiers and an ascending tie-break would resolve every tie in favour of the hand-written sixth of the set.

And sampling makes the diagnostic value of a single failure lower, not higher. When every case was chosen to test something, a failure named the thing; among twenty-five draws it does not, and recovering "which rule did this one tread on" costs more work. That is the price of the confidence interval, and it is paid in the currency of debugging rather than of statistics.

Status: implementation-backed for the mechanisms; all three datasets sealed, and reviewed more weakly than the manual asks. The generators, the two schemas, the per-dataset validators with their negative controls, the hypothesis computation and the refusal shape are implemented and are in the repository; every count in this subsection was read from the stored manifests and ground truths rather than estimated. S5 closes the excluded-member and output-aggregate gaps: it contains blank, note and course-shaped pollution rows, and its written total is checked against the sum of written course-credit slots. X1's zero cross-check count remains a different, structural case with a versioned policy reason. The residual shortfall is the human check itself, and it is now two shortfalls rather than one, because two review procedures of different strength are recorded in the same field. Through four enlargements the count of instances carrying the manual's own procedure — an independent hand recomputation of listed fields, without opening the expected artifacts, measured at about twenty-five minutes each — did not move: two of twenty-eight before S5, two of thirty-eight, two of sixty-three, two of eighty-eight, two of one hundred and thirteen. The denominator was the only thing that grew when a dataset did. On 30 August 2026 a reviewer signed forty-two of those one hundred and thirteen, which is the first time the numerator moved at all; but what was signed was a per-case read-through of each ground truth against the rule workbook with the arithmetic checked, one hour for all forty-two rows, and *the count carrying the stronger procedure is still two*. Both figures are reported wherever either is, because quoting the first alone would describe an audit that was not performed. Automated workbook-side recomputation is 35/35 on each enlarged

dataset but is not counted as human review, and it cannot be — it reads the workbooks that the same reading of the rules produced. The bias direction remains undetermined.

Doing this three times showed which parts of the recipe transfer and which do not. The four conditions above transferred unchanged, and so did the sibling-file placement, the seeded tie-break and the shape-based rerun rule. The stratification did not, in any of the three directions: X1 stratifies on its two refusal codes; S5's failure structure is a conjunction of four clauses, so its strata break one clause each and are checked against the dataset's own oracle; S4 has three codes covering five distinct detections, two of which share a code, so its strata are detections rather than codes. A recipe that transfers and a judgement that does not is the honest summary, and the part that does not transfer is the part that decides what the evaluation set can discriminate.

Designing S4's strata produced the one result in this subsection that concerns what the dataset cannot see rather than what it covers. Two behaviours the annotation manifest and the rule document declared had never been produced by any instance: the refusal for an undeterminable extent, and the conditional form of the packing number for a line occupying a single carton, which zero of seventy-nine detail rows had exercised. Neither would have turned a check red, because the checks that would catch a wrong answer for those shapes were never reached by any input. Enumerating what an evaluation set can discriminate is therefore not the same as enumerating what its rule document says, and only the second was ever written down; a mechanical check now requires every refusal code the manifest declares to have a producer, which closes the half of this that can be closed mechanically. The other half — a conditional branch inside a rule that no instance takes — has no machine-readable list to check against and remains open.

5.3 Baselines and Fairness

Three arms are compared. They differ only in what persists between instances.

Arm A — per-instance agent. For each instance, an agent is given the template, the rule document, and the source data, and produces the output document. Nothing persists: the next instance starts from the same prompt with no artifact carried over. This is the status quo the thesis argues against, and it is deliberately a strong version of it — a current frontier model with the same structured inspection tools the proposed method uses.

Arm B — agent-built frozen skill. The agent is first allowed to generalize from the calibration instances into a reusable artifact of its own design — a script, a checklist, a set of instructions — which is then frozen. Every evaluation instance is executed by an agent that may read that artifact and may not modify it.

Arm C — the proposed method. One derivation produces a specification; every instance is executed by the deterministic compiler with no model call at all.

Why three arms and not two. Arm B exists to answer the strongest available objection to the result. If only A and C were compared, any advantage of C could be attributed to the mere existence of a persistent artifact rather than to anything about how that artifact is produced and verified. Arm B holds persistence fixed and varies only the production procedure: an agent generalizing freely, versus a derivation constrained by an intermediate representation, structural gates, and two oracles. The question Arm B answers is therefore the one that actually distinguishes the contribution — is the verified derivation better than an agent's own attempt at the same idea?

Fairness has two layers, and the second is the one usually omitted. The first layer is *tooling parity*: the same structured dump utilities, the same model, the same retry ceiling, the same wall-clock ceiling. The second is *material parity*: Arms A and B receive an input list that is byte-identical to the one Arm C's derivation receives, because an advantage produced by handing one arm a cleaner or richer set of inputs is not an advantage of the method. The frozen input manifest is truth-free by construction and its envelope is hashable, so what each arm was given is recorded rather than asserted. That parity holds as an authorisation and, for one phase, not as a description of consumption: Arm C's derivation reads one of the K calibration instances its envelope lists, because a derivation dataset carries one source instance and one reference output per document role. The discrepancy, its bias direction — against the proposed method — and why declaring less for Arm C would fail a correct run are T18.

Material parity has a specific consequence that must be stated because it works against the proposed method. The rule material given to any arm may not contain target coordinates — no cell addresses, no bookmarks, no content-control tags, no drawing handles. Arm C's derivation therefore has to locate its own write targets, and cannot be given them; the rule table is not permitted to become an answer key for anyone.

Arm B's freeze must be enforced by the filesystem. An instruction not to modify the skill is not a control. The frozen artifact is hashed and mounted read-only for the evaluation phase; an attempt to write to it is recorded as a protocol violation and reported rather than silently tolerated. This matters because the failure it prevents — an agent quietly improving its own skill between instances — would convert Arm B into a per-instance arm while still being reported as a frozen one.

Status: implementation-backed, including one executed paid run. The three-arm input and evidence contract is frozen and tested: the truth-free input manifest, the hashable envelope, the Arm B freeze, namespace isolation for Arm C, retention of raw attempt evidence, and fake-capture negative controls are all implemented. As of 2026-08-24 so are the adapters that drive the arms: one shared live adapter for Arm A's execution and both of Arm B's phases, and a wrapper for each of Arm C's two phases. All five arm/phase seams have been driven end to end on the calibration split of both main datasets, with every attempt record validated against the frozen evidence schema and the cross-arm parity audit passing.

On 2026-08-25 the harness additionally drove a real provider. Three OpenCode sessions — Arm A's execution, and both of Arm B's phases — ran on one S4 calibration instance against `gpt-5.6-sol`, taking 8 minutes 38 seconds and 101,426 input, 13,868 output, 5,197 reasoning and 445,440 cached-read tokens across 27 assistant messages and 46 tool calls, with no retries. Every attempt validated against the frozen evidence schema, the parity audit passed, each workspace contained exactly the artifacts its envelope authorized and nothing else, and the frozen skill was unchanged afterwards in both the canonical file and the copy mounted for the agent.

One runtime-level input gap found by that run has since been closed. The OpenCode server had declared a product-specific common prompt as a server-level instruction, so its bytes reached every benchmark session without appearing in the envelope's prompt hash. The server now has no global instruction resource: that common prompt is composed only into the three named product agents, none of which the benchmark uses. This is a scoping fix rather than an envelope-schema expansion. In OpenCode 1.17.18, a message-level `system` value is appended after the configured instructions rather than replacing them, so it could not have removed the hidden input. The resolved runtime configuration was checked after the change: no global in-

struction remained, while each product agent retained exactly one copy of the common prompt.

What that run establishes is narrower than it looks, and one of its limits is the more interesting result. First, it says nothing about behaviour on sealed instances, and nothing about retry distribution: the retry policy allows a single attempt, so the observed zero is structural rather than measured. Second, a dry run is by construction incapable of producing a comparable accuracy number, because it must run on calibration instances — which every arm can see. Envelopes therefore carry an explicit `eligibility` field, calibration-split execution envelopes cannot carry an evaluation artifact at all, and a skill or specification frozen during such a run is refused by a scored one.

Third, and this was not anticipated: the same construction also prevents the run from measuring Arm A's per-instance *cost*. Because the fairness clause gives Arm A every calibration expected document, and a dry run's target instance is necessarily a calibration instance, the correct answer sits in Arm A's working directory. It was used: Arm A's two documents were byte-identical to that instance's expected output, and its own reply stated exact agreement with the matching calibration case. Arm B's execution phase is not exposed this way — its envelope authorizes only the template, the instance inputs and the frozen skill — so its figures are usable and Arm A's are not. The consequence is recorded as a protocol rule in Section 5.4.3: an Arm A per-instance cost may only come from a `formal` run.

The pilot reported in Section 5.1 was run outside this harness and is labelled as a pilot for exactly that reason; it must not be read as an Arm A result. That pilot also supplied this chapter's only prior token figures, and the live run corrected the argument built on them. The pilot was treated as a lower bound because the protocol's Arm A sees strictly more material — measured, 3.5 times the files. Its input tokens were none the less two to four times *higher* than the harness run's, because staged material is not context: an arm pays for what it opens, not for what it is

authorized to see.

5.4 Metrics

Section 4.6.3 established the finding this section has to be written against: a denominator, an allow-list, or an accepted spelling decides a reported number without failing anything. Every metric below is therefore defined as a denominator plus a statement of *who decides what may enter it*, because that second half is the part that has silently moved a headline figure four times in this project's records.

5.4.1 Correctness

Correctness is partitioned into the eight channels of Section 5.5.1, each with its own denominator and never averaged into one another. The denominator of each channel is the set of checks the dataset's annotation manifest declares with that channel's prefix; the manifest is authored per template, frozen, and shared by all three arms, and a check whose prefix was never declared raises rather than defaulting into the primary metric.

Field-level semantic accuracy is the primary metric. Its denominator is the values the manifest says the output must carry — prefixes `scalar/`, `slot/`, and `absent/`. What it deliberately excludes is formatting, which is reported as its own channel, and the "this slot must stay blank" obligations, which are reported as extent. Carriers are never merged: a single accuracy figure spanning DOCX and XLSX would average two different difficulties and two different adapters, so the main table has one row per evaluation template.

Extent is reported in its own column, and it is a correctness channel, not a diagnostic. This distinction is worth stating explicitly because the informal shorthand for the extent channel

in this project's own planning notes was "diagnostic", and that shorthand is wrong in a way that would matter. On the DOCX dataset, four of the six wrong-extent hypotheses leave all six totals byte-identical with the correct output; the only trace they leave is one extra detail slot consumed, because the rows they wrongly include have empty quantity and amount cells. An output produced under one of those hypotheses passes every total, every scalar field, and every cross-check. If extent were moved out of correctness, the class of error this thesis is centrally about would appear in the main table as full marks. It is reported separately *and* counted in the case-level correctness predicate.

Case-level rates, and why there are two. `passed` is a case in which every correctness channel — semantic, extent, refusal, structure, crosscheck — is entirely correct. `clean` is `passed` with no preservation violation, that is, with untouched and integrity also intact. The split is not decoration: compressing "answered wrongly" and "damaged the file" into one rate makes them indistinguishable, and their repairs have nothing in common. Presentation enters neither rate. Where a rule document is ambiguous about format, a literal comparison measures whether an arm guessed what the annotator had in mind, which is not a property of the method.

The silent-failure rate is the risk measure the whole design serves. Each case yields one outcome. `correct` is every correctness channel intact. `loud_failure` is a case whose output is missing or unparseable, or a success case that produced no scorable semantic evidence at all — an empty denominator must never be scored as zero errors. `silent_failure` is the remaining case: an output that is complete, opens cleanly, and is wrong. A case whose correct behaviour was refusal but which produced a document is a silent failure rather than a fourth category, because what it produced is precisely a normal-looking document. The two failure rates are reported separately throughout, since a system that fails loudly is operationally

a different system from one that fails quietly, and this thesis's entire argument is about the second.

5.4.2 Reliability, and the Instrument That Measures It

For the baseline arms, reliability is measured over $M \times R$ executions: pairwise agreement between runs of the same instance, the number of distinct output digests, and the proportion of cases correct on *all* R runs. For the compiled arm, execution reliability is a different claim — identity, not agreement — and the instrument has to be specified rather than assumed, because measuring it naively gives the wrong answer.

Identity is claimed at three levels, and the third one needs a normalizer. The first level is that no model is called. The test named in Section 4.3.3's neighbourhood replaces both repair entry points with recorders that fail if invoked, drives a complete production run, and asserts both that neither recorder was called and that the operation's recorded model-call count is zero. The second level is that the compiled write plan is a pure function of the specification and the instance. The third is byte identity of the produced file, which is what the protocol's $R = 5$ subset was intended to demonstrate — and here the naive instrument misreports.

Writing the same content twice into the two evaluation templates, three seconds apart, gives Table 7. The XLSX output is byte-identical, because the package-preserving editor copies every archive entry it does not have to change, including its recorded timestamp. The DOCX output is *not*: the library used to save it stamps every one of the twelve archive entries with the wall-clock time of the save, so two runs differ in twelve date fields while every part's content is byte-identical. Digesting the container would therefore report the deterministic arm as non-deterministic on one of the two main carriers, for a reason that has nothing to do with the method. The instrument of record is accordingly a *normalized* digest over the sorted `(part`

`name, part bytes)` pairs, with the raw container digest reported alongside and its divergence attributed. This is the same failure shape as the 66.7% denominator of Section 5.5.1: the mechanism was right and the bookkeeping would have decided the reported number.

Table 7: Two identical writes, three seconds apart, on the real evaluation templates.

	XLSX (13020D)	**DOCX (invoice)**
Container bytes	identical	**differ**
Archive entries whose date changed	0	12 of 12
Parts present after the write	unchanged	$10 \rightarrow 12$
Every part's content identical	yes	yes
Normalized digest	identical	identical

Two rows of that table are findings rather than calibration. The DOCX writer *adds* two parts, a default header and footer the template did not carry; the integrity channel does not see this, because those names fall outside its critical-part prefixes, so the addition is a bounded blind spot of that channel and is recorded as one. And the contrast between the two columns is a direct consequence of the package-preserving write adopted for XLSX: the DOCX carrier still re-serializes its whole archive on every save.

The compiled arm's variance is real; it is in the derivation. Reporting execution identity without derivation variance would misstate RQ2, and Section 5.4.5 defines the measurement that prevents it.

5.4.3 Cost

The unit of record is tokens; money is a derived and labelled quantity. This has to be settled explicitly, because the obvious choice fails on the available data. Every model call this project has made is recorded with five token counts and a reported cost, and the cost field is deliberately nullable so that "the provider reported zero" is distinguishable from "usage was

unavailable". Across the 26 recorded calls, the token fields are complete — no nulls — and the reported cost is a true, non-null 0 on every one, because the account is a subscription rather than metered. A money-denominated break-even would therefore be not merely unknown but structurally undefined: the numerator is zero by construction. Cost is consequently reported as input, output, reasoning, cached-read and cached-write tokens, with wall-clock as a separate column; a monetary figure, if given at all, is computed from a named public price list at a named date and labelled as derived, with both the compiled arm's derivation and the baselines' per-instance calls priced on the same list. The harness already records the token fields and the null-versus-zero distinction, so this ruling is a decision about presentation rather than about instrumentation.

Break-even, and the retries it must include. The break-even instance count N^* is the point at which one derivation plus N deterministic executions costs less than N per-instance agent executions. Because the compiled arm's marginal execution cost is zero model tokens by construction, N^* reduces to the derivation's total tokens divided by a baseline instance's tokens. The word "total" is the part that is easy to get wrong. A derivation is not one call: of the 15 derivations recorded in this project's model-call log, 9 invoked at least one repair call and two invoked a second. Quoting only the successful derivation call would understate the cost of the method, and in the direction that flatters it. The recorded aggregates, which are development runs rather than harness runs, are 1,402,174 input and 306,446 output tokens across the 15 derivation calls, and 142,361 input and 1,453 output tokens across the 11 repair calls; that asymmetry is itself informative, since a repair re-sends a large payload to produce a very small proposal. Derivation wall-clock ranged from 323 to 986 seconds with a median of 607.

The denominator that was empty, and the half of it that now is not. None of the numbers above is a baseline cost; they are all derivation-side. Until 2026-08-25 no per-instance cost for either baseline had been measured at all, which made N^* undefined and was the largest empty denominator in this chapter. One live harness run closed half of it. Arm B's execution phase, working from its frozen skill on a single instance, used $31{,}564$ input, $7{,}815$ output, $1{,}907$ reasoning and $144{,}384$ cached-read tokens over 9 assistant messages in 219 seconds, with a one-off skill build costing $35{,}977$ input and $3{,}891$ output tokens over 7 messages. Those are usable figures: Arm B's execution envelope authorizes only the template, that instance's source files and the frozen skill, so nothing resembling an answer is reachable from its workspace.

Why the other half cannot be closed by a dry run, ever. Arm A's figures from the same run are not a per-instance cost, and no amount of repeating the exercise would make them one. A dry run may only touch calibration instances, because the evaluation instances are sealed; the fairness clause requires Arm A to see every calibration expected document; therefore the target instance's correct output is necessarily sitting in Arm A's working directory. It was used — the two documents Arm A produced were byte-identical to that instance's expected output, and its own reply reported exact agreement with the matching calibration case. Its $33{,}885$ input tokens measure the cost of recognising an answer that was already present. The rule this fixes is stated in the protocol rather than worked around in code: an Arm A per-instance cost may enter N^* only from a run whose eligibility is `formal`. This is the same boundary that forbids reporting a dry run's accuracy, noticed one argument later — the accuracy consequence had been written down in advance, the cost consequence had not, and three documents had promised that this run would produce a baseline per-instance cost.

What the same run says about estimating cost at all. The pre-run estimate for these three sessions was 175–650 thousand input tokens; the measurement was 101,426, below the lower bound, as were output, reasoning, cached reads and wall-clock. The estimate was not merely imprecise, its argument was inverted: it scaled the earlier pilot's per-instance tokens up because the protocol's Arm A is authorized to see 3.5 times as many files. Authorized material is not context. An agent pays for what it opens, and this one opened little — six globs, five shell commands, two patches. Any cost projection in this chapter is therefore a projection over agent behaviour, not over dataset size, and the two do not move together.

The baseline per-instance cost, now measured, and the crossing it implies. The formal campaign closes the other half. Nineteen Arm A executions carry `eligibility=formal` and ran on *evaluation* instances, so the objection of the previous paragraph does not apply to them: fifteen on the trade-documents dataset, two on each of the two spreadsheet datasets. Their per-instance wall-clock means are 188.0, 136.8 and 178.6 seconds. One compiled-arm derivation, the fixed cost being amortized, took 1,282.7, 456.1 and 1,011.2 seconds on the same three datasets, each averaged over three replicates. The compiled arm's marginal execution, which spends no model tokens by construction, took 0.053 and 0.052 seconds per instance on the two datasets that produced documents at all. Three consequences follow, and the third is the one that matters.

First, at $N = 1$ the per-instance agent is faster, by factors of 6.8, 3.3 and 5.7; the slowest recorded baseline execution is quicker than the fastest recorded derivation on every dataset. Nothing in this thesis claims otherwise, but the figure had not previously been written down. Second, the resource crossing N^* — one derivation divided by the per-instance saving — falls at seven and four instances by wall-clock, and between two and six by token channel; the third dataset produced no completed compiled execution, so its marginal cost is unmeasured and its

wall-clock crossing is reported as undefined rather than estimated. One composition detail belongs with those numbers rather than in a footnote: the trade-documents mean includes four designed refusal cases, on which the baseline correctly produced nothing in fifty to seventy seconds. Including them lowers the baseline's per-instance cost and therefore *raises* the crossing, which is the direction unfavourable to this method; over the eleven producing cases alone the mean is 233.5 seconds and the crossing is six rather than seven. Both are reported.

Third, and this is the reason the number above is called a crossing and not a break-even: none of the nine recorded derivations is publishable; every one is refused by the publication gate. A crossing computed from them prices a specification that this system declines to deploy, which is arithmetic about a deployment that would not happen. The quantity that is still missing is therefore named precisely, and it is k of condition C-ii in Section 5.8.1: the number of derivation attempts per publishable specification, which multiplies the fixed cost. What changed is where the uncertainty sits. It is no longer spread across an unmeasured baseline and an unmeasured attempt count; the baseline is measured and small, and the whole of the remaining uncertainty is concentrated in k.

5.4.4 Rule Coverage Is an Arm C Diagnostic

Coverage is reported as `covered`, `unresolved`, `rejected` and `unmentioned` against an external denominator, and two properties of it must be carried into this chapter unchanged from Section 4.6. First, the denominator is a frozen rule-applicability manifest *authored by a human* from the rule document and bound to it by the SHA-256 of its bytes. Binding prevents drift; it does not establish that the prose was segmented correctly, and no independent segmentation has been performed. Describing the denominator as a deterministic segmentation of the rule document would overstate it. Second, only the compiled arm produces dispositions at all,

so coverage cannot appear in any cross-arm correctness figure; it is a derivation diagnostic and is reported with the derivation results, never in the main table.

Status: implementation-backed for the channel definitions and the identity instrument; normative for the cost metrics. The eight channels, their fail-closed classification, the two case-level rates and the outcome classification are implemented by the shared scoring module under a frozen policy version, with negative controls per channel. The identity instrument is backed by the measurement in Table 7, taken on the two real evaluation templates. The cost metrics are half a plan: the ruling on units is made here, the harness records the fields, and the baseline per-instance cost, once the largest empty denominator in this chapter, is now measured on nineteen formal executions. What remains normative is the attempt count k, and the bias direction of leaving it unmeasured favours the proposed method, since it is now the only quantity that could move N^* against it.

5.4.5 Asymmetric Rerun Budget

The asymmetry is a consequence of the claim, not a saving. The symmetric design — every case, every arm, R times — allocates most of its budget to copying files. If the compiled arm's execution is deterministic and calls no model, five runs of one specification over one instance produce five identical outputs, and the fifth teaches nothing the first did not. The budget is therefore $R = 1$ for Arm C on the main table, with a designated subset run $R = 5$ to *demonstrate* the identity rather than to average over it, using the normalized digest of Table 7. Skipping the demonstration would be assuming what is being claimed; repeating it on every case would be measuring the file system.

The baselines' reruns are decoupled from M, because correctness and consistency do not want the same thing. The original design multiplied them: every evaluation instance of Arms A and B was to be run $R = 5$ times. That spends four fifths of the budget on repeats which serve only the consistency question, and the consistency effect is not subtle — the pilot produced three distinct file digests in three runs and a cell-level agreement of 69.2%. Correctness wants *many instances*; consistency wants *many repeats of one instance*. They are therefore separated: Arms A and B run every evaluation instance once for the main table, and a set of five instances per template, *declared and frozen in advance*, is run five times for the consistency figures. Freezing the choice in advance is not ceremony — choosing afterwards which instances to rerun is a researcher degree of freedom, and a large one. The cost of the decoupling is stated rather than hidden: each main-table case now rests on a single observation, so "Arm A was correct on this case" no longer carries "and would be every time". That second claim is what the consistency column reports, over five instances per template rather than over all M.

Arm C's execution is free per instance, which creates a temptation the protocol forbids explicitly: running the compiled arm over all M instances while running the baselines over fewer would break the parity condition of Section 5.3 in the method's favour. Every arm sees the same instances.

The variance did not disappear. It moved, and it must be measured where it went. Reporting Arm C as having zero variance without qualification invites, and deserves, the one-line rebuttal that the randomness was merely relocated. It was. The formulation of Chapter 3 places all of it in the derivation: $\mathcal{A}$ is stochastic and $\mathcal{X}$ is deterministic. The measurement that keeps the claim honest is therefore $D = 3$ independent derivations per evaluation template, each producing a specification, each specification run over every evaluation instance, and the reported quantity a *distribution* — accuracy, coverage, and the size of the diff between the specifications

— rather than a single derivation's numbers.

"Independent" has an operational definition, and getting it wrong would contaminate the measurement in the method's favour. A product derivation reads the field names of the dataset's existing healthy specifications and feeds them back to the agent. That accumulation is a genuine capability of the method — a per-instance agent structurally cannot have it — but a $D = 3$ measured with it is a warm-start measurement, and reporting it as the method's variance would compare an arm with memory against two arms without any. The three replicates therefore run in separate dataset namespaces, which is what severs the history, and the harness enforces the condition rather than trusting it: each replicate's envelope must carry empty prior-specification, diagnostic-run and repair-run lists, the Arm C derivation may not be handed a specification resource, and the store refuses a dataset namespace that has already been used. The three replicates share one frozen rule manifest, and each must record the same version and SHA-256, so the denominator cannot move between them. Convergence under repeated re-derivation within one dataset is a separate and legitimate experiment; what is prohibited is running warm and reporting cold.

What is already known about that variance, and what it is not. Four derivations of the XLSX template have been run with identical prompt, identical payload and identical input bytes. They produced rule coverage of 15, 12, 19 and 23 out of 27; one declared three detail slots where the template has seven; another declared none of the cross-file collection its predecessor had used; the count of hard structural errors ranged from four to zero. Two derivations of the DOCX template with an identical prompt differed in whether the computed extent columns survived at all. This establishes that derivation variance exists and is large, which is the qualitative half of RQ2. It is *not* the planned experiment: those runs were not namespace-isolated, the

implementation changed between several of them, and their purpose was debugging. They are reported as observations with that provenance attached, and the distribution promised above has not been measured.

The shape in which that distribution will be reported. Table 8 fixes the columns now, before any of them can be chosen after seeing the numbers. Each row is one evaluation template, and the entries are read over its $D = 3$ isolated replicates: the field-level accuracy of each replicate's specification executed over all M instances, reported as a range rather than a mean because three points do not support one; the rule coverage of each replicate against the frozen manifest, whose denominator is fixed across replicates by Section 5.4.4; the number of replicates whose specification passes every publish gate, which is the quantity Section 5.8.1 calls k seen from the other side; the size of the specification diff between replicates, as the count of fields on which any two disagree; and the number of *cases* on which the three replicates do not produce the same verdict, which is the closest thing this design has to the dispersion measures of Section 2.13. The last column is the one that matters most and is the easiest to omit: three specifications can differ substantially and still agree on every case, and a reader cannot infer one from the other.

Table 8: Derivation-time dispersion, $D = 3$ isolated replicates per evaluation template. The columns are fixed in advance; no replicate has been derived under the protocol and no cell carries a number.

Template	**Semantic accuracy, min–max**	**Rule coverage, min–max**	**Publishable replicates**	**Spec diff, disagreeing fields**	**Cases with a split verdict**
S4 (DOCX)			/ 3		/ M
X1 (XLSX)			/ 3		/ M
S5 (XLSX)			/ 3		/ M

No row of that table may be averaged with another, for the reason Section 5.5.1 gives, and no column may be summarized as a single dispersion score: the point of reporting five quantities

is that they can move independently, and a composite would hide exactly the case where the specifications differ a great deal and the outputs do not.

Status: normative for the budget and the $D = 3$ design; implementation-backed for the isolation mechanism, the identity instrument, and the existence of derivation variance. The namespace isolation, the empty-history conditions and the manifest-version checks are implemented in the harness and covered by negative controls. The budget itself is a plan, and its bias direction is worth naming: giving the baselines five runs and the compiled arm one is generous to the baselines under any best-of reading and unfavourable to them under any average, so the reporting must state which it uses — this thesis reports per-run and all-runs figures, never a best-of.

5.5 The Evaluator

All three arms are scored by one deterministic program. It parses the produced artifact, extracts the values at the locations the dataset's annotation manifest names, compares them against that instance's ground truth, and emits per-check results. It never asks a language model whether an output looks right, for the reason given in Section 4.3: a judge that can be wrong in the same direction as the system under test cannot bound the system's error.

5.5.1 Separate Channels, Never One Average

Scores are partitioned into channels, each with its own denominator: *semantic* (the primary metric), *extent* (which source rows were included), *refusal* (a case whose correct behaviour is to refuse), *structure* (the artifact opens and has the expected shape), *crosscheck* (output-side identities), *presentation* (formatting, reported separately from meaning), *untouched* (regions no

field declares were not modified), and *integrity* (the package's critical parts survived). A check whose prefix is not one of the declared channels fails closed rather than being scored as absent. The policy is versioned, and every result record carries the evaluator version, the policy version, the source hash of the scoring code, and the repository commit.

The partition is not presentational. The one measurement that settles it: while the "this slot must remain blank" obligations were counted inside the semantic denominator, an arm that submitted the blank template unchanged — writing nothing whatsoever — scored 66.7% semantic correctness on one instance. After the obligations were moved into their own channel the same output scores 0%. Every individual check had been correct; what was wrong was which total they were added into.

5.5.2 Two Self-Validations, Neither Sufficient Alone

Forward. The evaluator must score the generator's own expected artifacts at 100%. This catches an evaluator that reads the wrong location or compares with the wrong tolerance.

Reverse. Controlled corruptions are applied to the expected artifact and the evaluator must catch every one. Without this, an evaluator that always answers "pass" would give every arm a perfect score and the forward test would not notice. The current mutation coverage is 6 of 6 on the pilot dataset, 13 of 13 on S4, and 13 of 13 on X1; X1 was the first dataset in which all seven pre-crosscheck channels had at least one mutation that exercised them, which closed two channels that had previously been structurally vacuous.

A mutation test is not a validity proof, and the gap is specific. The expected artifact and the ground truth are produced by one generator. If the generator computes a value wrongly, both are wrong together and the evaluator faithfully reports 100%. Mutation testing establishes that the

evaluator detects corruption; it says nothing about whether the generator computed correctly. The two do not overlap. The present detection mechanism for that residual is a per-dataset spot audit — at least three evaluation instances per dataset, each with fields recomputed by hand from the rule document, including one aggregate and one negative obligation — which is probabilistic rather than a guarantee, and is reported as such. The audit performed on 30 August 2026 meets that floor by instance count on all three datasets and does not meet it by method on any: it read each ground truth against the rules rather than recomputing the fields independently of the expected artifacts. T12 carries the distinction, and the audit record carries it too, hashed into the seal so that what was done cannot be separated from the claim that something was.

5.5.3 A Negative Control for Every Zero

The evaluator and the probes around it are themselves software, and the failure mode observed repeatedly during construction is that a broken probe reports a serene zero: zero mismatches, zero problems, full marks. Measured instances during this work include a probe that used the wrong key name, one that omitted a worksheet prefix from every reference so that nothing matched anything, one whose trial compilation failed so that the comparison returned an empty list, and one that compared string forms where the annotation declared exact decimal comparison, so that twenty *correct* cells were reported as wrong. Two of those report "looks broken" and the rest report "looks fine"; the second kind is the dangerous one, and it is indistinguishable from success by inspection.

The rule adopted is therefore mechanical rather than advisory. Any reported zero or perfect score must be accompanied by a deliberate control that makes the figure move, and the control must distinguish doing the work from not doing it — a control whose expected value also happens to be the result of skipping the mechanism measures nothing and never turns red. That

last clause is not hypothetical: an ordering test was found whose expected row order equalled the arrival order, so disabling the sort entirely left it green.

5.5.4 The Extent Diagnostic Is Not Symmetric Across Arms

Only Arm C declares which source rows it included; Arms A and B produce a document and no provenance. Scoring the extent channel from an arm's own declaration would repeat exactly the mistake that the coverage denominator made in Section 4.6 — a figure self-reported by the system being scored is not a cross-arm metric.

Two cross-arm instruments are implemented. Hypothesis attribution names a wrong extent from the produced output without asking any arm anything. Cross-check scoring evaluates manifest-declared identities between fields of that same output. The common consumer runs after actual values have been extracted for both XLSX and DOCX, accepts only scalar identifiers and sums of declared slot columns joined by addition or subtraction, uses decimal tolerance, and fails closed on unknown, missing, or non-numeric operands. It neither reuses a generator's truth-side result nor fills an operand from the expected artifact. S4 has seven output-side checks; the S3 pilot has three and marks three source-range identities as truth-side-only; X1 has none because its target totals nothing, with that structural reason enforced by a separate versioned policy artifact. The effect of retaining the independent extent instrument remains measurable: an output that occupies one extra detail slot while leaving all six totals byte-identical passes all 112 semantic checks on one instance and is still identified as a specific wrong-extent hypothesis. Arm C's own resolved anchors are reported as supplementary evidence and are explicitly marked as available for one arm only; they are never placed in a column alongside A and B.

Status: implementation-backed. The shared evaluator lints, self-tests, and scores all three datasets; the scoring policy is frozen at version 1.2.0 and the evaluator at version 1.1.0; the mu-

tation suites and their per-channel negative controls are in the repository and run in continuous integration. What is not established is the generator's own arithmetic, as stated above.

5.6 Template Drift

Every split in Section 5.2.2 holds the template fixed and varies the instance. Template revision is an orthogonal axis, and a held-out design cannot absorb it: an arm that has never seen a template is in exactly the position of an arm whose template just changed only if the change is total, which is the least interesting case. This section is the direct test of the design principle of Section 3.1.5, because a drifted template is where a silent failure is cheapest to produce and hardest to notice.

One variable per variant. Each evaluation template gets a set of controlled variants, each differing from the original in exactly one respect: a block moved, a field's caption reworded, a block substituted, or a row inserted. A variant that changes two things at once cannot attribute the failure it provokes, and the earlier benchmark review made the same point about the pilot dataset. Section 5.6.2 reports how that requirement is enforced on the artifacts themselves rather than trusted to whoever chose the edit. The observations are then symmetrical across arms: does the baseline produce a wrong file without saying anything; does the compiled arm refuse, does it name the location of the change, and what does re-deriving cost.

5.6.1 What Exists, and at Which Moment It Runs

Drift detection in this system is a structural fingerprint — a hash over which references exist and what kind each is, with text and geometry excluded for the reasons in Section 3.8.3. Its sensitivity has been measured on both real evaluation templates rather than on fixtures, and

Table 6 reports what it does and does not move on. The essential entry for this section is that inserting a row inside the form changes the fingerprint on both carriers, while a library round trip with no edit does not.

At review time the detector is wired, and its gate distinguishes two kinds of silence. Re-inspecting a template recomputes its fingerprint, updates the lineage, and marks every specification keyed to the old structure as stale. Publication then refuses in two different ways. A stale fingerprint is a reviewable refusal: an operator may override it, but only with a written reason that is recorded permanently on the production version. A template role with *no* recorded fingerprint at all is not overridable, because "nothing drifted" and "nobody ever fingerprinted this" produce the identical empty list, and publishing on the second reading means publishing with the detector switched off for that document. That asymmetry is the part of the mechanism this thesis actually wants to claim, and it is implemented and gated.

At execution time the detector was not wired, and that was a defect rather than a boundary. The claim RQ4 needs is stronger than the one above: a frozen artifact meeting a drifted template on the run that uses it must fail loudly. Auditing the production path while writing this section found that it did not, in two different ways on the two paths, and both are worth recording because the second is the more instructive.

On the DOCX and XLSX path — the two main evaluation carriers — no fingerprint comparison occurred at all. What does protect that path is a different check: every reference the compiled plan writes must exist on the template, and a plan naming a reference the template does not have is refused with `TARGET_NOT_FOUND` before anything is written. That is a real fail-loud behaviour and it covers a real class of drift, namely any change that invalidates a reference. It cannot cover the complementary class, and the complement is precisely what the

fingerprint was built for: a row inserted above a detail area leaves every reference a perfectly valid address and changes what that address means.

On the drawing path a fingerprint comparison was present in the code and returned `TEMPLATE_DRIFT_` before any inspection. It compared the specification's fingerprint against a `structural_fingerprint` entry in the input artifact's metadata — and a search of the repository found exactly two occurrences of that key: the line that read it, and one test that constructed the metadata by hand. Nothing in the system ever wrote it. The comparison was therefore unreachable on any real artifact, the guard had never fired, and its test was green and would have stayed green. This is the failure mode Section 4.2 names as the vacuous guard, and the reason it survived is the one recorded in this thesis's verification rule: the only thing exercising the declaration was an input written by the same hand as the code.

It is now wired, and one measurement had to be corrected to wire it correctly. Following the first of the three admissible responses, the comparison was moved ahead of the carrier branch so both paths share a single comparison site, and the production run now computes the fingerprint of each template it is about to write and records it on that artifact — which is what finally gives the key a producer. The office carriers can be re-fingerprinted in process; the drawing path still reads a recorded value, because a drawing's structure needs the CAD host, and that asymmetry is inherent rather than a duplicated rule.

The repair is verified on the real evaluation templates rather than on fixtures: a run over the unmodified spreadsheet template records the expected fingerprint on its artifact, a variant with one row inserted inside the form is refused with `TEMPLATE_DRIFT_DETECTED`, a plain library round trip of the same file is *not* refused, and a specification pointing at a sheet the template lacks is still refused with `TARGET_NOT_FOUND`. The last two are what make the first two evidence rather than assertion: without the round trip a moved fingerprint would only show

that the spreadsheet library rewrote the file, and without the missing-sheet case the drift code would merely be the new name for every failure.

One thing the repair had to get right is worth stating, because the obvious design is the wrong one. If the run-time verdict were read from the persisted metadata rather than computed from the bytes in hand, then a metadata write silently failing would silently switch drift detection off again — the same defect in a new place. It is computed, the persisted value is provenance, and the write is pinned by a test on the real controller. That test earned its place immediately: the first version of the write named a timestamp column the artifact model does not have, the exception was swallowed by the deliberately broad error handler around a best-effort write, and the key went unwritten exactly as before.

What this does *not* settle is the scope of the claim. Fixing detection does not run the drift experiment; it gives RQ4 a working control. The variants that control now has are the subject of the next subsection, and the classes the fingerprint cannot see are unchanged.

5.6.2 The Variant Corpus, and What It Cannot Reach

Controlled variants of the two evaluation templates now exist. Each of the four templates — the two spreadsheets of the highway-bureau chain and the two Word documents of the trade pair — receives seven variants, twenty-eight in all, and each variant differs from its template in exactly one respect: a row or paragraph inserted, a block relocated, a caption reworded, a block of headings substituted, or nothing at all.

The last of those is the control, and it is the reason the others are interpretable. A variant produced by opening the file in a document library and saving it again differs from its template in hundreds of bytes and, on the spreadsheet carrier, in eleven package parts: the library does not preserve the drawing or the printer settings. Its fingerprint does not move. Without that

measurement, “the fingerprint moved” on the other variants would establish only that a file had been rewritten.

One variable is a property of the artifacts, not a promise from their author. The generator refuses to write a variant it cannot attribute. Each variant declares whether it moves *structure* or *content*, and three views of the file — the reference set, the text at each position, and a text-free layout signature — are compared before and after. A structural variant must preserve the multiset of content, because a permutation cannot invent a value; a content variant must leave both the reference set and the layout untouched; and every variant must move at least one of the three views, or it is a file with different bytes and the same meaning. Two situations are refused outright rather than annotated: a relocation whose row permutation would tear a merged range across the boundary of the moved block, and a worksheet containing any construct whose row references the editor has not been checked against. A refusal that still writes the file is not a refusal.

The generator computes no expected output and imports nothing from the system under test. It produces the question; the answer is measured elsewhere.

Three of the seven classes are out of scope for the detector, and the corpus says so in writing. Each variant carries a declaration, written from the class’s semantics before anything was measured, of what the detector should do: fire, stay correctly silent, or be structurally unable to see the change. All twenty-eight declarations matched the measured behaviour of the structural fingerprint on the first run. Eight variants must be refused, eight are variants on which silence is the right answer and the compiled output is still correct, and twelve are drift the fingerprint cannot reach. Those twelve are recorded with a reason and must be reported as out of scope rather than counted as detector misses; a validator refuses a corpus in which one of

them is unlabelled, because an unlabelled blind spot is one a results table would silently score as a failure.

The three unreachable classes are worth naming, because two of them were carried in this thesis as prose and the third was not previously written down anywhere.

Layout past the last populated reference. A spreadsheet fingerprint hashes the cells that carry content, so a row inserted below the last of them — below row 25 on the monthly-report template — cannot reach it. The corpus makes this sharper than the earlier statement did by pairing it: the *same* insertion one anchor higher does move the fingerprint, on both carriers, so the difference in verdict is a property of where the row went rather than of the edit. The Word analogue is measured here for the first time; the earlier record noted that only three perturbations had ever been tried on that carrier.

A permutation among equally shaped positions. Two rows occupying the same columns, or two paragraphs carrying the same number of fillable blanks, exchange places. Every reference survives, every one of them now names different content, and the fingerprint's key set is identical. This is the silent failure of Section 3.1.5 in its purest form — a compiled specification writes real values into the wrong slots and the file parses perfectly — and at execution time nothing detects it. The design excludes text from the fingerprint deliberately, because every production run rewrites text, and names the publish-time regression comparison as the mechanism that covers a renamed caption; that mechanism does not run on the instance path.

A block of headings substituted in place. Two columns of a form exchange meaning while keeping their slots. Structurally identical to the previous class and reached by a different edit, which is why the corpus keeps them apart: they differ in what an operator would see, not in what the detector can.

A fourth limit is unchanged and is about review rather than execution: a specification's blank-target declarations are not compared by the structural diff at all, so two specification versions differing only in what they claim to leave blank present a reviewer with an empty change list.

What the corpus is not. The classes come from the drift protocol and from what these particular forms plausibly do between editions; they are not drawn from an observed corpus of real revision histories, and no claim is made that their frequencies resemble those of real revisions. The edits are byte-level patches to individual package parts, which keeps them attributable at the cost of being tidier than a revision made by hand in an office application — row-anchored drawings are moved with their rows for exactly this reason, but the general point stands and bounds the external validity of every number the drift axis will report. And the two carriers are served by two implementations that share no code, so a class covering both is one name over two mechanisms.

5.6.3 Metrics for This Axis

Three quantities are reported per variant, and each is defined so that it can be computed identically for an arm that declares nothing.

Silent-failure rate under drift. The proportion of variants on which an arm produces a complete, parseable, wrong document. This is the same outcome classification as Section 5.4 and needs no arm-specific instrumentation, which is what makes it comparable.

Localization rate. Given a failure, whether the arm's output identifies *where* the template changed. For the compiled arm this is the refusal's named reference or role; for a baseline it is whatever the agent's transcript states, judged against the variant's known single

variable. An arm that fails loudly without localizing scores on the first metric and not on the second, and separating them matters because a refusal that says only "something is wrong" costs an operator the same search as no refusal at all.

Repair cost. For the compiled arm, the tokens and wall-clock of re-deriving against the variant, including repair calls, measured in the units settled in Section 5.4. For the baselines this quantity is structurally zero and reporting it as an advantage would be an error: they pay per instance forever, which is the trade Section 5.8.1 discusses.

Status: detection and the variant corpus implementation-backed on both carriers; the experiment normative. The fingerprint's sensitivity is measured on both real evaluation templates; the publish gate's two-way refusal is implemented and tested; and as of 2026-08-24 so is the execution-time comparison, which now runs ahead of the carrier branch, is shared by both paths, and is verified on the real spreadsheet template against a row-inserted variant with a library round trip as its control. As of 2026-08-26 the production side exists as well: the twenty-eight variants of Section 5.6.2 are generated, validated by ten checks each of which has been shown to turn from green to red under an injected fault, and consumed by the production controller's own tests — including two tests that assert a variant is *not* detected, so that the two blind spots turn red the day somebody closes them.

What remains normative is the experiment, and only the experiment. **No drift experiment has been run and this section reports no result.** Building the generator produces variants, not measurements: every *number* the three metrics above describe is still a plan, the classes the fingerprint cannot see bound what those numbers could show, and the corpus's own external validity — generated revisions standing in for observed ones — bounds them again. Where the design is claimed without evidence the bias runs towards the proposed method, and Section 5.10 repeats it there.

5.7 Ablations

RQ5 asks which parts of the derivation procedure are load-bearing. The question is not rhetorical: this chapter argues that a verified derivation beats an agent's own attempt at the same idea, and if the gates and oracles turn out to be mutually redundant then the honest version of that argument is much smaller. Four ablations are planned, and before they are listed it is worth separating two different things the word can mean here, because this project already runs one of them continuously and has never run the other.

5.7.1 Two Kinds of Ablation, and Which One Is Cheap

Observability ablation, already mechanized. Disabling one guard and asking whether its own tests still fail measures whether that guard is still *observable* — whether anything in the suite depends on it. This project runs it as a sweep over fourteen enumerated guards, each disabled by a one-line substitution, each required to turn its selected tests red; a row that selects no tests is treated as invalid rather than as passing. The sweep exists because a guard becoming redundant turns no test red, and it has caught two guards that had become vacuous inside their own suites. It costs nothing and runs offline.

What it does not measure is contribution to accuracy. A guard can be fully observable in the test suite and still never have prevented a defect on real material, and conversely. Conflating the two would let a green sweep stand in for RQ5, which it cannot.

Retrospective replay, also cheap, and the instrument this project actually uses. The second cheap form is to replay a gate over the corpus of stored specifications that were produced *before* that gate existed, and count how many it would have refused. This answers "would it have fired", on real derivation output, at no model cost. It has been used four times in this

project and the numbers are informative in both directions: a gate on formula inputs that nothing produces flagged 11 of 35 stored specifications with no false positive; a slot-alignment gate replayed over the corpus matched exactly the two specifications it was written for and left the other dataset's five untouched; an audit of which authority decided each of 610 target references found 10 to be inferred rather than authored, all in one dataset; and the contract audit of Section 3.3 moved from 11 invalid to 4, all four of which the compiler independently refuses.

The limitation of retrospective replay is precise and must be stated wherever it is used: it tells you whether the gate would have fired on the specification that was produced, not what specification would have been produced had the gate been present. A gate that fires changes the repair loop's input, and therefore changes the artifact. Replay bounds a gate's yield from below; it cannot bound the method's accuracy without it.

5.7.2 The Four Planned Ablations

A1 — structural gates removed. Derive with the structural gate set disabled and score the resulting specifications with the same evaluator. The expected effect is not lower accuracy on the fields that exist but a larger count of specifications that do not execute at all, which the outcome classification of Section 5.4 would score as loud failures. That is the interesting prediction: gates are hypothesized to convert silent failures into loud ones rather than to raise accuracy, and an ablation that measured only mean accuracy would find them worthless.

A2 — one oracle instead of two, and the precondition this ablation has. The complementarity argument of Section 4.3 is the central empirical claim of Chapter 4, so testing it matters. The ablation cannot currently be run as stated, and the reason is recorded rather than worked around: the case-level oracle has no caller in the automated derivation path. Removing it from that path is a no-op, so the measurement would return zero difference, and zero would be read as

"the second oracle contributes nothing" when what it actually means is "the second oracle was never in the pipeline". The ablation is therefore stated in two parts. Wiring the case-level loop into the automated path is a precondition, recorded as an open item; until it is done, only the author-time oracle can be ablated, and the case-level oracle's contribution can be reported only offline, by running stored specifications against their annotated cases and counting the defects the author-time oracle had passed.

A3 — repair loop removed. Accept each derivation's first proposal and score it. Of the 15 derivations in the model-call log, 9 invoked at least one repair, so this ablation touches a majority of runs rather than a tail. It also has a cost reading as well as an accuracy reading, since the repair calls are where a derivation's token cost stops being a single number.

A4 — a second model. Every one of the 26 recorded model calls used a single model from a single provider. A conclusion drawn from one model is a conclusion about that model, and the thesis claims a property of an architecture. This ablation is the one with no cheap form: it requires paid runs of the full derivation, and it interacts with the cost accounting of Section 5.4, since a second provider will report metered costs where the first reports a subscription zero — which is another reason the unit of record there is tokens.

Status: normative. Nothing in this section has been run. The observability sweep and the four retrospective replays are implemented and their numbers are real, but neither answers RQ5, and saying so is the point of the distinction drawn above. The four ablations are a plan; A2 additionally has an implementation precondition that is recorded as an open item, and A4 requires model budget. No component contribution is claimed anywhere in this thesis on the strength of the sweep alone.

5.8 Results

The formal experiment began on 30 August 2026 and this section still carries no table, because the quantity every row of that table reports is a proportion over all $M = 30$ evaluation instances and no template has been run to completion. What exists is a first batch, and it is described here rather than tabulated so that a partial run cannot be read as a result.

What has been run, and what it is not. Nine attempts are recorded with `eligibility: formal`, all successful, all validated against the frozen evidence schema: on S4, two Arm A executions, one Arm B skill build and two Arm B executions; on X1 and on S5, two Arm A executions each. Six produced documents were scored by the shared evaluator. Every one of the six is *semantically* perfect — 52 and 172 scored fields on the two S4 instances under both arms, 70 and 114 on X1, 60 and 60 on S5 — and none is a silent failure. That is 6 of the 90 instance-arm cells the main table needs, from a campaign whose declared size is 50 agent runs per arm per template plus $D = 3$ derivations, so it fixes no proportion and no interval. It is reported because the alternative — saying nothing until the campaign completes — would leave the three findings below undiscoverable, and each of them changes how the completed table must be read.

The XLSX integrity channel already separates the arms, and not for the reason it looks like. All six scored cases pass every correctness channel, and four of them fail overall on the integrity channel's critical-part check. The two X1 outputs lost their drawing and printer-settings parts; the two S5 outputs kept the drawing part and changed it; the two DOCX outputs are clean. This is the failure ADR 0012 removed from the compiled arm's own writer, appearing now on the baseline — and the comparison is not yet fair, for a reason internal to this study

rather than to the baseline. The frozen Arm A prompt requires that the template's layout not be changed and, four lines later, names `openpyxl` as available; a single `openpyxl` round trip is precisely the negative control this channel uses, because it drops those parts. The harness recommended a tool that cannot satisfy the harness's own instruction. The prompt is frozen and its digest is carried in every envelope, so it has not been changed — altering the configuration after seeing a result is what the methodology freeze exists to prevent — and the consequence is recorded as a threat with its bias direction named in Section 5.10. The integrity column may not be reported without it.

The cost currency is not currency. Every one of the nine attempts reports `cost_usd` of exactly zero, because the provider is accessed through a subscription that bills no per-call amount. The aggregator of Section 5.4.3 therefore returns a per-instance cost of $0.00, which is a true statement about this billing arrangement and useless as a cost model: a break-even instance count computed from it is undefined. Section 5.4.5 had already recorded that subscription billing reports zero; what it had not done is connect that to RQ3's denominator. The measurable units are tokens and wall clock, and the first batch fixes their order of magnitude: an Arm A execution took 198–261 seconds and 53,360–94,699 input tokens, an Arm B execution 330–386 seconds and 36,401–61,719, and the one-off Arm B skill build 270 seconds and 59,305. The budget estimate of Section 5.4.5, which assumed 2.9 minutes per session, is corrected by these to roughly 4.8 minutes.

Arm B is not structurally cheaper than Arm A. Its execution envelope authorizes strictly less material — the template, the instance and the frozen skill, with no rule document and no calibration set — and on the second S4 instance it nevertheless consumed more input than Arm A did, 61,719 against 53,360, because it produced more output across a longer exchange

and a multi-turn session reads its own output back. Any sentence of the form "the reused-skill arm is cheaper because it sees less" is unsupported and is not made in this thesis.

The compiled arm's derivations, and the one number that may not be quoted from them. Five batches of $D = 3$ derivations have been paid for on S4, each under a named declaration and each on a different revision of the product, so they are five conditions rather than five repeats of one. Across the last three, field-level accuracy on the one replicate per batch that produced documents was 96.4%, 95.5% and 96.4%, and all-or-nothing accuracy was zero of forty-four every time: the movement is derivation variance, not a trend, and the channel that does not move is the one this thesis measures. What they establish is not a proportion. It is that this system's own release gate would have refused every specification it produced — and that the specifications were frozen and executed anyway.

That was a deliberate choice and the reason is not that it keeps more data. A run that ignores the gate is the gate's negative control. Enforcing it alone leaves the compiled arm's column empty and a serene zero beside it; bypassing it alone produces a number describing a system this thesis does not propose, because the review step *is* the "once" in compile-once. Neither run is the result; the difference between them is what measures C3, and it exists only because both ran at one frozen configuration. Every attempt therefore records the gate's verdict *before* the specification is frozen, and the evaluator reports the two populations apart.

The verdict must not be reported as its boolean. That boolean is one value over twelve heterogeneous item kinds answering different questions: for the gate's own question — should a person look at this — merging them is correct, since every kind means yes; for the question this section asks — what kind of thing blocked it — the same merge is wrong. Recounting the fourth batch's itemised lists gives 18, 14 and 19 items, which resolve into 15, 12 and 16 distinct events in three classes. Eleven, ten and twelve are defects of the specification itself, which is

the compiled arm's review working: two of them are the gate that fires when a rule supplies a name nothing consumes, and it fired on exactly the two fields that had been silently wrong in the previous batch. Three, two and three are artifacts of role attribution and are the subject of T19. One, none and one are residue from a known limitation of the derivation-time oracle, and belong to neither. Three classes inside one integer is why the integer may not be quoted.

One number the itemised list does not give directly is how often the compiled arm read a rule and honestly declined to implement it, since a declined rule whose declared roles do not match the frozen manifest is filed as a rule nobody mentioned. Recovered from the stored derivation records, the fourth and fifth batches declined nine rules across their six replicates while the blocking lists name six. That gap is not noise about which rules are hard; it is the same attribution predicate as T19, seen from the side that matters most to this thesis, because a system that refuses to fabricate a check it cannot express is exhibiting the behaviour the silent-failure metric is meant to reward.

An honest specification produced a worse document than a dishonest one. Between the third and the fourth batch a derivation-time gate was added that refuses a specification supplying a source name no field or formula reads. In the fourth batch it fired, correctly, on the two fields the third batch had filled in wrongly, and the agents responded by declaring both unresolved rather than implementing them. Field-level accuracy fell, from 3814 of 3955 to 3776 of 3955, and the difference of 38 is exactly the number of times the one field it stopped writing is scored. All-or-nothing accuracy was zero of 44 in both.

A specification can diagnose itself correctly, say so in its own record, be correctly refused by the release gate — and, so long as nothing asks that gate, produce a worse document than a specification that lied. This is the single strongest reason the compiled arm's field-level column may not be reported alone.

Expressiveness the derivation was told about and did not use. The interval predicate above, and a second construct added at the same time for stacking two independent lists on one row grid, were both written into the derivation prompt, and a third run declaration was issued for the change because the prompt's digest is sealed into every derivation envelope. Three fresh derivations were then paid for under it. **Neither construct was used by any of them.** All three specifications instead recorded the carton rule as *rejected*, and all three gave the same reason: that the specification language has no primitive for comparing one row's interval with the next. That sentence is now false, the primitive is named and illustrated in the prompt those runs were given, and the prompt's hash in their envelopes is the hash of the file containing it.

The lesson this repository had drawn from an earlier instance was that adding expressiveness is necessary but not sufficient, and that three steps are needed: build it, describe it in the prompt, and add a derivation-time check that fires when it is missed. All three steps existed here, and the outcome was still zero of three. The corrected statement separates three things that had been treated as one: **a check can exist, be reachable, and fire, and these are not the same.** The alignment check exists and its own tests pass, and it fired zero times in these three derivations, because it only speaks when a specification declares two blocks over one row grid and none of them declared even one — they stopped a step earlier, at reading a variable-length labelled continuation list, which the system can in fact do. The interval predicate has no third step at all: a rejected rule blocks the release gate, but that says a person should look, not that the stated reason is untrue.

What this does not establish is that the constructs would never be used. Three derivations, one model, one prompt, one calibration instance. Nor does it separate "the agent did not use it" from "that paragraph is not clear enough" — separating those requires rewriting the prompt and paying again, and rewriting a frozen instruction after seeing a result is precisely what the

methodology freeze forbids without a fresh declaration.

The refusal channel, and a refusal that was right for the wrong reason. S4 declares eight instances whose correct behaviour is to produce nothing. In every executed batch the compiled arm refused six of them and produced documents for the other two, and those two documents are silent failures of the kind this thesis measures: a packing list whose carton intervals overlap or skip, every individual value legal. The specifications were not lying about it. Each recorded the governing rule as *rejected* and gave the reason — that the specification language had no primitive for a predicate over intervals in adjacent rows — and that statement was true when it was written.

What makes this worth a paragraph rather than a footnote is how it stayed invisible. Both instances *were* refused for months, by an unrelated defect that rejected an empty labelled cell; the refusal was right and its reason was wrong, and repairing the reason converted two correct refusals into two silent failures without turning a single test red. The missing primitive has since been built (ADR 0013) and its acceptance measurement is two-ended: over the thirty evaluation instances the acceptance split moves from 24/6 to 22/8, and exactly the two instances move. No derivation has yet been run with it available, so nothing here is claimed about whether a derivation would use it; T4 records the primitive's own evidential weakness.

The second template, and a refusal channel that rewards a broken specification. The compiled arm was then derived three times on the spreadsheet template S5, which until that point had never been derived at all. The result reverses the reporting problem of the preceding paragraph rather than repeating it. One replicate produced a specification the validator passed and executed on every instance, producing fifty documents; it declares no refusal anywhere, so the four instances that must produce nothing produced a document each, and its refusal channel

is zero of four. The other two replicates produced specifications with three fields missing a required transform, so the compiler refused *every* instance, produced nothing at all, and scored **four of four** on the same channel — the four correct outcomes being the four instances whose correct behaviour is refusal.

So a specification that cannot execute earns a perfect refusal channel, and the failing replicate looks better on that axis than the working one. The rule this fixes is a citation rule and not a mechanism: **the refusal channel is never reported without the number of documents that replicate produced.** It is the mirror of the defect above — there, repairing a wrong reason turned two correct refusals into silent failures; here, breaking a specification turns silent failures into correct refusals — and in both directions no test changes colour.

The two templates also fail in shapes that a single accuracy column would flatten into one. On S4 the specification wrote the wrong thing while asserting it could not express the rule, and field-level accuracy stayed above ninety-five percent with every failure inside one field group. On S5 the specification *honestly refused* most of the rule set — seventeen, fourteen and fourteen of nineteen rules recorded as rejected — and field-level accuracy is 282 of 1458. Both templates score full marks on package integrity and on cells that must not be touched, which is the same sentence in both cases: the deterministic compiler did what the specification said, exactly. What differed is what it was told. Those two channels are, for that reason, close to free for a specification that writes almost nothing, and must not be reported alone.

Two further reporting consequences follow from that batch. First, the undercount of honest refusals described under T19 is a property of a manifest with more than one document role, not of the scorer: recomputed over S5's three replicates, forty-five declared refusals are reported as forty-five, with no reclassification, against nine reported as six on S4. Second, the residual category into which unexplained expected-text changes are placed held at most one item per

replicate on S4 and between forty-seven and fifty on S5, and item-by-item it is three distinguishable populations — cells the expected output fills and the specification does not write, cells the template ships with sample content that a correct output must clear, and unused grid slots blank on both sides (five, six and thirty-nine on S5's first replicate). The first two are the arm's own coverage gap, so reporting the whole category as a measurement residue would understate it, in the direction that favours the proposed method.

The difference between the two templates in that category is not that the category grew, and the distinction is worth stating because measuring it the obvious way gives the wrong answer. The count of unexplained changes recorded in a specification's evidence is thirty-three to thirty-five on S4 against the publish gate's nought or one, because S4's specifications declare a fourteen-slot repeat block that owns those addresses, so the gate finds them explained; S5's three declare no repeat block at all, so every one of the fifty is unexplained and blocks. What separates the two templates here is therefore whether a repeat block was declared, and the population split is computed over the subset that actually blocks.

The third template, and a specification that claims almost everything and executes nothing. The compiled arm was finally derived three times on X1, the remaining spreadsheet template, which completes the compiled column of the main table. All three derivations succeeded, all three produced a specification the validator rejected, and **all one hundred and fifty executions failed, producing no document at all**. Field-level accuracy is therefore nought of the frozen denominator of 1566.

Two properties of that batch matter more than the number. The first is that the three replicates are not merely similar but identical: instance by instance, all fifty runs return the same failure code under all three specifications — thirty-five with an unresolvable formula input, eight with a declared-versus-resolved collection count mismatch, seven with an unmatched record in

a row-set merge. Nothing like it appears elsewhere in the campaign, where S4's replicates differ in how many documents they produce and S5's differ in whether the specification validates at all. Three independent cold-start derivations, into three separate and demonstrably empty datasets, behaved the same way on every evaluation instance. That is a dispersion measurement in the direction that favours the proposed method, and it is also the least useful kind of agreement available: the three specifications agree in being unable to do anything.

The second is that this template makes the refusal-channel rule of the preceding paragraph stricter rather than merely confirming it. On S5 the perfect refusal channel was obviously free, because every instance failed identically and a reader could see it at a glance. On X1 it is not. The seven instances whose correct behaviour is refusal each fail with a code that *is* semantically the designed refusal condition: the four whose dispatch register declares one more attached audit sheet than is present fail on the collection count, and the three whose project register lists one more project than received an audit sheet fail on the unmatched record. Checked case by case against the ground-truth refusal codes, the mapping is exact. The refusal channel is seven of seven and every reason is right — and it still establishes nothing, because the same specification also refuses the twenty-three instances that should have produced a document, only with a different code. **A system that fails on every instance can look correct on the refusal cases, and can be correct there for the right reason.** The citation rule therefore grows a clause: the refusal channel is reported with the number of documents produced *and* with the outcome distribution over the instances that should have succeeded.

Producing nothing also collapses the denominators, and that is a measurement hazard of the same family as the field-count denominator of Section 5.4. With no artifact to inspect, the scorer reports nought of nought on semantic, extent, presentation, cross-check, integrity and untouched alike; only the refusal and structure channels have a denominator at all. Confirmed

against S5's second replicate, which also produced nothing and reports the same way, so this is general and not a property of X1. An accuracy quoted from those figures would be an undefined ratio that reads as an absence of errors, so every compiled-arm accuracy in this chapter is quoted against the frozen denominator file instead. The same collapse retires the free full marks noted above: on this batch, package integrity and untouched cells are not full marks but nought of nought.

Read beside the other two templates, X1 settles the shape of the main table. The three specifications fail in three mechanisms that no single column can express: on S4 one lied about what it could express and wrote the wrong thing anyway; on S5 one honestly refused most of the rule set; on X1 all three claimed to cover almost all of it — twenty-four of twenty-seven rules covered, none rejected — and were structurally unable to execute. **And on the metric this thesis treats as central, X1 is the best of the three: it produced no silent failures at all, against forty-four of forty-four on S4 and thirty of thirty on S5.** That is not quality. A specification that writes nothing cannot fail silently, so the silent-failure rate is reported with the number of documents produced, for exactly the reason the refusal channel is.

The cause is a derivation defect and not a gap in the representation, and the distinction was checked rather than assumed. All three specifications compute the report's two header fields from a formula that reads three or four names, and none of them declares anything that produces those names, so the validator raises an unproduced-formula-input error three times per replicate and the compiler refuses at run time. A pilot specification derived on the same template in August wrote the same two fields correctly, using a formula string identical word for word, because it also declared sibling fields that supplied those names. The language expresses it; these three derivations omitted the declaration.

What that leaves is a statement about the derivation loop rather than about the language,

and it sharpens the three-part claim made earlier in this section. The gate in question exists, is reachable, fires — three times per replicate, at the right addresses — and its message names the offending identifier, quotes the whole expression, and states the repair. Whether that repair was *available* to the loop was then measured rather than inferred, and the answer first written down here was wrong. The message offers two repairs: bind the name a source locator in the rules, or stop referencing it. The first is not reachable. The rules live in a mapping the repair application function does not take as an argument at all; the repair request sent to the model does not carry that mapping among its six payload keys, so the round cannot see the half of the specification it is being told to edit; and the function refuses both substitutes outright — a proposal that introduces a field is rejected as one a specification may only grow through a derivation, and a proposal that attaches a locator to an existing field is rejected because where a value is read from stays frozen. That is the same structural position as the rule-assignment case under T20, not a contrast with it. The second repair is reachable, because a transform belongs to the value layer. **So the only repair the loop could have made is the one that removes the reference, and a specification that stops reading a value it needs writes a document that is wrong rather than one that is right.** None of the three rounds made even that one.

What the missing declarations were worth — a diagnostic, not a result. The head-room was priced before deciding whether to pay for a further derivation campaign, by hand-writing the declarations the three derivations did not write and running the frozen specifications over the same thirty evaluation instances, through the same three production stages and the same evaluator. The figures below are a diagnostic and appear in no results table: the declarations were written by someone who had already seen the reference outputs, which is adjacent to the second of this thesis's own red lines, so each figure is stated together with what was hand-written. The reference outputs themselves were not touched, and the specifications were read

from the frozen resources those fifty executions bound, verified against their freeze digests, rather than from the database.

All three replicates behave identically. As frozen they produce no documents and score zero of the 1566 semantic checks. With the two header transforms rewritten to stop naming the unsupplied inputs — the only repair the loop could have taken, applied through the product's own repair function — they produce twenty-three documents and 1507 of 1566, and all twenty-three are silent failures. With three hand-written intermediate inputs instead, one per name the gate reported, the validator returns no errors at all and the same twenty-three documents score 1553 of 1566, nine of them silent failures. The thirteen wrong cells that remain are a single column, and closing it is priced under T18 rather than here. Three things follow. The distance between a specification that writes nothing and one that writes almost everything is three declarations of a construct the language already had. The repair the loop was permitted to make would have produced documents, but would have turned a loud failure into twenty-three silent ones — so on this template the unreachable repair and the reachable one differ in the direction this thesis measures. And neither figure says a derivation would ever write those declarations, which is the standing lesson of this section and is not settled by hand-writing them.

Making the named repair reachable, and what that cost. The gate's first repair was made available to the loop and a further three derivations were run on X1 under a fifth frozen declaration, changing three things and nothing else: the repair application function now admits one addition to the source layer, an input a formula reads that writes nowhere, for a name the deterministic checks themselves reported as unproduced and only where no rule of that name exists; the repair request now carries the source layer, so the round can see the half of the specification it is told to edit; and the repair agent's own prompt describes that repair and states that the alternative — stop reading the value — makes the document wrong rather than right.

The derivation-time result is positive and is worth separating from what follows. All three replicates' repair rounds proposed the missing inputs, all three proposals were admitted, and specification validity went from false in three of three to true in three of three. That is not the pattern this section has reported three times before, where an expressiveness affordance was added and no derivation used it, and the difference is statable as a rule: **adding a capability an agent must think to use is a different investment from making the stated repair of a gate that already fires reachable.** The second needs no invention on the agent's part; the message already names the identifier.

The execution result is that nothing changed, and the reason is a defect of this thesis's own making, which is why it is reported here rather than as an agent failure. The same one hundred and fifty executions produced no documents. Two of the three locators the loop bound resolve on none of the three calibration instances — including the one the derivation itself read — so all twenty-three success instances stop at locator resolution, while the seven designed refusal instances still refuse for their own reasons. The frozen semantic denominator is unchanged and the score is zero of 1566, as before.

But the two batches are not the same result, and reading them as one is the mistake this paragraph exists to prevent. In the earlier batch the specifications were correctly judged unpublishable and the publish gate blocked them; in this one they report themselves valid and fail at run time. Two checks were each silenced, and neither alone would have sufficed. The cross-instance locator gate closed over the source layer as it stood when the gate was built, so it never saw a locator the repair added — it reported checking two while the specification shipped five. And a type-correct placeholder, added so that the trial compile would not report an unproduced name for a value a locator supplies, answered on that name's behalf; removing the placeholder returns the specification to invalid, measured. So a placeholder is an assertion

that execution will supply a value, and it holds only where something has actually resolved the locator — the same family as the exemption discussed under the runtime-parameter gate, and harder to see, because it looks like a convenience rather than a carve-out. Both are now closed, and neither has yet been exercised by a real derivation, so this thesis claims nothing about whether they suffice.

Where the compiled arm's failures actually sit, measured rather than assumed. A reader of the three subsections above may reasonably suspect that the bottleneck is the deterministic compiler rather than the derivation, since half the pipeline is rule-based compilation. The suspicion is checkable and was checked. All four hundred and fifty recorded compiled-arm executions were attributed to the stage that returned them, using the three call sites of the execution adapter — resolve the source data, compile, write the document — rather than the wording of the error: one hundred and eighty-seven stopped *before* the compiler, one hundred and sixty-nine at compilation, none at the write, and ninety-four completed. No execution was left unclassified.

Three further measurements separate "the failure surfaced at compilation" from "the compiler is where the defect is". First, not one of the one hundred and sixty-nine compile-stage refusals came from a specification that had passed its own derivation-time validation; every one refused a specification the publication gate had already judged invalid and which was executed anyway under the ruling of Section 5.8. The four specifications that did pass that validation produced fifty completed executions and one hundred and fifty failures *upstream* of the compiler. Second, in all ninety-four completed executions the write plan the compiler persisted is exactly the write-address set the frozen intermediate representation declares — ninety-four of ninety-four, with no mismatch. Third, and decisively, of the semantic checks that land on an address the representation declared, four thousand one hundred and eight were scored and none

failed; every one of the two thousand and eighty-one semantic failures falls on an address the representation never declared. On the trade-documents batch the count of checks inside the representation is numerically the batch's reported semantic score, which is another way of saying that the score measures what the specification declared and nothing else.

The conclusion is narrow on purpose. It says that repairing the compiler would move none of the numbers in this section, and that the residual defects are omissions from the intermediate representation rather than mis-execution of it. It does not say that the representation is the right intermediate form — a compiler faithful to a specification that declares too little is exactly what produces a silent failure — and it does not extend backwards over the write path, which has carried a defect of its own within this project's history: a guard intended for single-cell rules once made a correct extent form unreachable through the repeat construct, and no test turned red. What it does establish is the direction of the remaining work.

The reporting shape, unchanged. One row per evaluation template with no average across templates, because carriers may not be pooled into a single accuracy; the per-channel denominators of Section 5.4 kept separate rather than aggregated; and any externally sourced template reported in its own subsection rather than inside that average, since it answers a validity question rather than a capability one. The subsection below is written now because it does not depend on the numbers.

5.8.1 Where the Randomness Went

The single most available objection to this thesis is one sentence long: the randomness was not removed, it was moved. That is correct, and this subsection is where it is conceded in full. Section 5.4.5 states where the variance went and defines the measurement that keeps it visible; the questions here are different. What did the relocation buy, and under what conditions is it a

bad bargain?

What is actually claimed. Executing N instances with a per-instance agent draws N times from a stochastic process. Executing them from a compiled specification draws once, at derivation, and then applies a deterministic function N times. Nothing in that exchange reduces the variance of the draw. The distribution of derived specifications is wide, and measured to be so: four derivations of one dataset with identical prompt, payload and input bytes produced rule coverage of 15, 12, 19 and 23 out of 27, with structural differences of the same magnitude. Any sentence in this thesis that reads as "the method is deterministic" is a claim about the execution function only, and Section 1.6's table labels it that way deliberately.

The first thing the exchange buys is a place to put a check. This is the part that is not merely bookkeeping. N stochastic executions have no natural review point: reviewing them means reviewing N documents after they exist, and the cost of review grows with the same N the cost of production does. One stochastic derivation produces one artifact, and that artifact can be examined before any document exists. Everything in Chapter 4 is a consequence of that shape rather than an independent set of features — the structural gates, the author-time oracle, the case-level oracle, the coverage denominator, and the human evidence review all operate on a specification, and none of them has an analogue in a per-instance arm. This is not a claim that the baselines are weaker; it is a claim about where a checkpoint can be placed at all. An agent that writes a document has nothing to show a reviewer until the document is written, and by then the review is per instance again.

The second thing it buys is an asymmetry in how failures surface. A bad derivation is refused before publication and costs a re-derivation. A bad execution ships a document. That asymmetry is independent of N, and it is the reason the exchange is worth taking even where the

cost argument is neutral. The evidence for the first half is direct: the publish gate has blocked derived specifications on this study's own datasets with named codes — overlapping repeat targets, a source field nothing produces, a formula input the execution path cannot supply — and each refusal was later confirmed to correspond to a real defect.

The evidence against overclaiming it is equally direct and is stated in the same breath. When one of those specifications was executed for the first time against the shared evaluator, three scored cells failed and all three were classified as silent failures: the arm produced a complete, well-formed document with three wrong cells, and the gate set did not stop it. The asymmetry is therefore real and partial. It says that a whole class of defect is caught before any document exists, not that every defect is.

What the exchange costs, in the units this chapter settled. Three costs are measured and belong in the numerator rather than in a footnote. First, repair: of the 15 recorded derivations in this project, 9 triggered at least one repair call, and Section 5.4 rules that those calls are part of the derivation cost, because reporting only the successful attempt would understate the numerator in the direction that favours the proposed method. Second, catastrophic loss of a paid attempt: one derivation ran to completion and was then lost in its entirety at persistence, on a defect in the surrounding system rather than in the derivation. Third, human review: the evidence review of Section 4.5 is real work that the per-instance arms do not perform at all, and any cost model that omits it is measuring the wrong thing.

The three conditions under which the exchange is not worth making. Each is stated as a quantity rather than as a caveat, and each is reported honestly as measured or unmeasured.

C-i — too few instances. Below the break-even instance count, a one-off derivation plus near-zero marginal execution costs more than paying per instance. The relevant count is not

the number of instances in existence but the number executed *per specification lifetime*, which template drift shortens: a form revised every quarter resets the amortization every quarter, which is why Section 5.6 is a cost question as much as a correctness one. This quantity is *measured, conditionally*, and the condition is the whole of what is left. Arm B has a per-instance execution cost from a live harness run; Arm A now has one too, from nineteen scored executions on evaluation instances, which is the run the previous version of this paragraph said would be required — a dry run could not have produced it, because the only instances it may touch are ones whose expected output Arm A is entitled to see, and it used it (Section 5.4.3). The resource crossing that follows is small: seven and four instances by wall-clock on the two datasets whose compiled arm produced documents, and two to six by token channel. That number is not the break-even point, because it prices a derivation the publication gate refuses; the gap between the two is condition C-ii below, and it is no longer one gap among several but the only one. One scope note belongs here rather than in the introduction, because this is where the condition is stated as a quantity. C-i is an *economic* condition and not a feasibility one. Section 1.2 gives a second motivation that does not depend on N — that real instance data may not leave the organization, so the per-instance alternative is not among the options at any N — and below break-even those two readings point opposite ways: the exchange can be a bad bargain on cost and still be the only admissible arrangement. Nothing in this thesis measures how often that situation arises, and the threat that records the claim is T16.

C-ii — derivation succeeds too rarely. If the expected number of derivation attempts before a publishable specification is k, the amortized derivation cost is multiplied by k, and a large enough k removes the advantage at any N. This quantity is also *unmeasured*, and it must not be inferred from this project's run history even though that history is

recorded in detail. Fifteen derivations were run, but they span several versions of the system being derived from: nearly every unsuccessful one was blocked by a defect in the intermediate representation, the gates, the tool registration or the persistence layer that was then fixed. The denominator is a sequence of different systems rather than one system run repeatedly, so no rate computed from it would mean anything. What can be stated without inference is the current position, measured after the most recent round of gate changes: the best specification on each of the two main evaluation datasets is still blocked, at thirteen and eight canonical unresolved items respectively, so neither dataset yet has a publishable specification. The bias direction of that fact is genuinely mixed — the history is pessimistic because the defects it records have been fixed, and no optimistic reading is supported either, because a clean end-to-end derivation has not yet been demonstrated on either main carrier.

C-iii — the task is outside the class. If the output is not deterministically derivable from structured sources and explicit rules, no k helps, because the derivation is being asked for a function that does not exist. The applicability conditions in Section 3.1.6 are the operational form of this, and the honest answer where they fail is that the method does not apply rather than that it performs poorly.

What the conditions imply for how the results must be read. One of the three is now measured on one side and unmeasured on the other, and the unmeasured side is the one that would decide the economic argument. This has a direct consequence for Section 5.8: a break-even curve drawn from a single successful derivation without an attempt count would still be a picture of an assumption, even now that the baseline cost underneath it is real. The half that is measured makes the remaining half sharper rather than smaller — the crossing is at a handful of instances, so the whole economic question is now whether a publishable specification costs

a handful of derivations or many. Chapter 6 takes up what would have to be measured to close each of the three, and this subsection deliberately stops at naming them.

Status: normative for the three conditions, implementation-backed for the mechanism and for the costs quoted. The zero-model execution property is pinned by the negative-control test named in Section 3.1.2; the gate refusals, the repair frequency of 9 of 15, the lost paid attempt, and the three silent failures in an executed specification are all recorded measurements from this repository, none of them produced by the evaluation harness. The resource crossing is now measured and reported above; the derivation attempt count is not, and is labelled here rather than estimated, so the break-even instance count remains unavailable because it is the product of the two. The bias direction of leaving the attempt count unmeasured runs *towards the proposed method*, because it is the one quantity remaining that could make the exchange a bad one, and Section 5.10 repeats that disclosure.

5.9 Case Study: Engineering Drawings

The drawing carrier is reported here and is not in the main results table. The reason is worth stating in the same words used in Section 1.1's discussion of carriers and in Section 2.5, because an earlier version of this chapter's plan gave a different one and the difference matters: the exclusion is not a statement about the carrier's difficulty or its prestige. The entire system was built on a drawing template. Every prompt, every gate, every choice of intermediate representation was tuned against it over months. Any figure this carrier produces is contaminated by construction, and reporting it beside figures from templates the system did not grow on would be reporting two different things in one column. That is a validity precaution. Presenting it as a showcase — "the method even works on the hardest representation" — would invert a precau-

tion into a selling point, and would additionally rest on an argument of the form "models cannot read this format", which the next model answers.

5.9.1 What the Case Study Is

The artifact is a fan datasheet drawing. Its material, counted rather than estimated, is one baseline template; five case directories, of which four carry an independently produced expected drawing with its own expected-value record; rule material consisting of a provenance slide deck and a checking list rather than a single prose rule document; and zero drift variants. Seventeen of the thirty-five stored specifications in the development database belong to the two drawing lineages, and four production versions were published from them.

Two properties of that inventory rule out treating it as a small evaluation dataset rather than as a case study, independently of the contamination argument. The rule material is not a rule document in the sense the rest of this thesis uses — there is no frozen rule-applicability manifest for it, so the coverage denominator of Section 5.4 does not exist here. And there is no evaluator adapter for the carrier at all: the shared scoring program raises rather than guessing when asked for a dataset and carrier combination it has no adapter for, and drawings are such a combination. The numbers this case study can produce are therefore not produced by the instrument that produces the main table, and cannot be placed next to it even channel by channel.

5.9.2 What It Contributes Anyway

It is the evidence that the pipeline is carrier-neutral rather than carrier-agnostic by assertion. The compiler is shared across all three carriers, and the plan it emits is a carrier-neutral edit operation rather than anything drawing-shaped. That claim is checkable, and checking it found one leak: the compile entry point returned a drawing-specific message for every carrier,

so an XLSX specification refused by validation was told its “DWG spec validation failed”. It was corrected, and a sweep for the same shape found that the three remaining drawing-specific error codes are each correctly confined behind a target-kind branch with a stated reason. A carrier that is never exercised is exactly where such a leak survives, and having a live drawing path is what exposed it.

It is the origin of one intermediate-representation construct that no office carrier needs. A drawing renders a two-cell row — a value, its annotation, its maximum, and a unit — as four independently positioned entities with no containing structure. The IR models this as a single field owning the whole row, collapsing into one string when either number changes and left untouched when neither does. It has its own gate: a row whose maximum has no locatable handle is refused, because without it the row can be neither blanked nor compared, and the carry-over decision degrades silently into an unconditional rewrite. That gate is not decorative. One stored drawing specification declares a row whose maximum carries a transform and no handle; the compiler refuses it, and the published contract independently rejects the same specification for the same missing key. It is one of only four stored specifications the contract rejects, and the contract and the compiler agree on all four.

It is where the loosest inference in the system was actually observed to misfire. Label-proximity inference — deciding which rule a field belongs to by the caption nearest the target — is the weakest matching mechanism in the derivation, and an audit of it found that on the two office evaluation datasets it contributes nothing, because the geometric path is reachable only on non-agent derivations. The specification it did damage is a drawing one, and the mechanism by which it did so is a good illustration of why the audit was worth doing: a revision note reading `REVISE AVC BARCODE DEFINITION` was matched to a current-rating rule, because that

rule listed the unit `A` and the letter `A` occurs inside `AVC`.

The zero-model-execution claim is carrier-independent, and this is where it was first true. The production path calls no model on any carrier, and the evidence is not a reading of the source but a test that substitutes failing recorders for both repair entry points, drives a complete run, and asserts that neither was reached and that the operation's model-call count is zero. The drawing path is the one where a repair branch used to exist, so this carrier is where the claim has the longest history of not being true; the test is what makes it hold in the future rather than only today.

Status: implementation-backed as a description; explicitly not evidence of generalization. The inventory above is counted from the repository, the carrier-neutral compile path and the row gate are implemented and tested, and the zero-model-call property is pinned by the test named. What this section does *not* claim is that the method generalizes to engineering drawings, and the reason is the one it opens with: the system was developed on this artifact, so a good result here is uninformative and a bad one would be surprising. The bias direction is towards the proposed method in the obvious way, which is exactly why nothing from this section enters the main table, any average, or any of RQ1 through RQ4.

5.10 Threats to Validity

Each item below states what is threatened, what was measured, and which arm the bias favours. Items whose direction favours the proposed method are listed first, because those are the ones a reader has no independent way to discover.

5.10.1 Threats Biased Towards the Proposed Method

T1 — development happened on evaluation templates. The template-level separation described in Section 5.2.2 does not hold: the two development templates the protocol proposes were never built, and the prompts, gates, and intermediate representation were tuned against the calibration instances of the evaluation templates themselves. Three representation primitives exist specifically because the compiled arm could not otherwise produce X1. The bias favours Arm C. The two bounding measurements are given in Section 5.2.2.

An earlier version of this paragraph added that the bias "is not quantified — the only way to quantify it is to build the missing development templates". The second clause is false for the sharpest form of the threat, and the correction is worth more than the original claim. Whether an address named in a prompt is an address a correct output must write is decidable from the template and its expected output alone: it is in the baseline-to-expected diff or it is not. So the leak can be counted without building anything. Counted over the frozen derivation prompt (1184 lines, whose SHA-256 is sealed into every `C/derivation` envelope), which names nine concrete target references and five slot-indexed target templates: six of them are references S4's own output must write, three are X1's, and none is S5's. The prompt additionally names both of the X1 cells that a refusal rule requires to be left *blank* — which by construction can never appear in a baseline-to-expected diff and are therefore not in those counts. The addresses entered the prompt in the same commit that built the X1 dataset, downstream of X1's own pilot derivations.

An earlier form of this paragraph also counted the prompt's statement of X1's slot geometry — slot count, slot origin, record group size and row-role names — as part of the leak. Checked against X1's own rule workbook, three of those four are not leakage at all: cell `C28` of its rule sheet states verbatim that the data area holds seven slots at two rows per project, and cell `C6`

states that the upper row of each pair carries the month's value and the lower the running total. Only the slot *origin* is prompt-only, and even that is one step from the workbook's provenance column, which places the form's header on rows four and five. The related claim that the prompt's worked example was identical to what all four stored X1 specifications wrote is also wrong: two of the four wrote a slot count of three, and across all seven stored X1 specifications the leaked count of seven appears in five. **This correction shrinks a disclosed threat, which is the direction this section warns about, so it is stated rather than quietly applied.** What it does not touch is the load-bearing half. The address counts are unchanged, and they now carry a mechanical control: none of the counted references appears in any of the three rule workbooks, the only address-shaped strings in X1's being two provenance notes that name header *rows* rather than cells.

The open question — what the help was *worth* — has one measured cell, and its value is zero. The prompt names X1's two blank signature cells verbatim, and none of the three specifications derived for the main results declared them: all three left the blank-target list null and recorded the reason, which is that the two cells show no text change in the calibration instance and so no verifiable reference for them appears in the evidence the agent was shown. That is section 0 of the same prompt being obeyed — copy a handle verbatim from where you found it and never construct one — so one part of the prompt withheld what another part had handed over. Counted the other way, over the specifications' own field maps, S4 reproduced six of its six leaked references in all three replicates and X1 one of three in all three; X1's figure understates, because the two references it appears to have declined are written anyway through a slot template whose origin is the leaked row.

Three things follow, and the third is the one that fixes the bias direction. First, this is the same prohibition as the one on rule tables — no target cell address, bookmark, or content

control tag, because a target must be derived and not given — displaced into the other artifact the deriving agent reads; the rule workbooks obey it, and so do all three ground-truth records, in which the count of target references is zero. Second, the count is a lower bound: the slot geometry and the column headers are answers too, and are not address-shaped, so nothing counts them. Third, the asymmetry: the Arm A and Arm B prompts are twenty-eight and twenty-nine lines and name no address whatsoever, so this is help the compiled arm receives and the baselines do not. What remains genuinely unquantified is what the help was *worth* — an agent may have found those addresses unaided — and separating that from the leak needs a derivation under a prompt with the addresses removed, which is a further declaration and a further authorisation. S5's zero is why its Arm C batch was run before X1's: it is the one compiled-arm derivation in this campaign whose prompt carries none of its own answer. It bounds only the address leak. S5 remains subject to the rest of this paragraph and to T3, so it is a partial bound and not a clean control.

T2 — the cell-datatype exemption. The compiler stringifies every value before writing it, so a numeric field lands in a spreadsheet as text. The annotation rules therefore declare `presentation.dat` as `any` for every field. Without that exemption Arm C would lose a point on every numeric field, and the study would be measuring a write-side type policy rather than the method. With it, a real defect — numbers stored as text — is not scored, and a baseline that writes correct types earns nothing for doing so. This is the third admissible response to a gap: the claim is kept, labelled normative, and the direction disclosed. It favours Arm C.

T3 — the rule tables are ours. Even where the template is a genuine third-party artifact, the machine-readable rule applicability manifest that supplies the coverage denominator was written for this study. It weakens the objection that the templates are self-designed; it does not

weaken the objection that the rules were written by someone who knew what the method can express. Any use of an externally sourced template must state this in the same breath, or it reads as padding.

T16 — the deployment argument is made without a deployment. Section 1.2 states a second motivation — derive in the cloud from synthetic calibration instances of a real template, execute on the organization's own machines with no model at all — and it is entirely normative. No on-premises installation exists, no organization has used this system under a confidentiality constraint, and no experiment here varies the deployment topology. Three things about its bias direction have to be separated. It is favourable to this thesis in the plain sense that a motivation requiring no experiment cannot be falsified by one. It *inherits* T12 rather than adding to it: the argument's load-bearing assumption is that synthetic calibration instances resemble real ones closely enough, and where they do not the specification is wrong on real data deterministically and silently, which is the same shared-misreading class T12 already describes, arriving by a different route and covered by the same spot audit, whose method is weaker than the procedure it is named for. It also inherits T8 without improving it, since the arrangement has been exercised on the same three templates as everything else. What T16 adds beyond those two is one item they do not cover: this thesis constructs no threat model and does not claim that a published specification is free of information traceable to source data, so a reader must not take Section 1.2 for a privacy guarantee. That section states the same four boundaries in the paragraph that makes the claim, which is the only place where a reader who skips this chapter would see them.

T17 — the baseline was pointed at a library that cannot obey the instruction it was given. The frozen Arm A prompt says that the template's layout, label text and table structure must not

be changed, and four lines later says that `openpyxl` is available. A single `openpyxl` round trip drops `xl/drawings` and `xl/printerSettings` — that is exactly the operation the integrity channel uses as its negative control, and exactly the defect T5 records the compiled arm's own writer having had. In the first formal batch the baseline used the named library and every XLSX output failed `integrity/critical-parts`: the two X1 outputs lost both parts, the two S5 outputs kept the drawing part and changed it, and the two DOCX outputs were clean. Every one of the six was semantically perfect.

The direction is unambiguous. Any integrity comparison that reports the compiled arm preserving what the baseline destroys is partly measuring which library this study's own prompt suggested, and the suggestion sits inside the material the study controls. The prompt has not been changed in response: its digest is carried in every sealed envelope, and altering the configuration after seeing a result is what the methodology freeze exists to prevent, so the correction would be a re-declared campaign rather than an edit. Until that happens, the integrity column may be reported only alongside this paragraph, or not at all.

What is *not* claimed here is that the baseline could not have preserved the parts. The task is achievable — the compiled arm achieves it by editing one package part and copying the rest — and nothing in the prompt forbids that route. Whether an agent that had not been pointed at `openpyxl` would have found it is unmeasured, and measuring it would require a second campaign under a second declaration.

T4 — three primitives rest on a single example. Artifact collections, stacked records, and equality merge between row sets were each validated on one dataset, and it is the same dataset in all three cases. The key–value extent added for the DOCX dataset has the same property: a survey of six real government forms found no second instance of the shape. The failure mode this predicts is not a red test but a different form that silently does not fit, and no measurement

in this thesis would detect it.

The interval-sequence predicate of ADR 0013 is the fifth, and its case is worse than unexamined. The standing hypothesis for a second instance of the shape was the chainage intervals of the engineering form family, and searching all three frozen rule sets for an interval, overlap or contiguity predicate found one rule in one of them and none in the other two — the engineering family in this study is an audit report, in which chainage does not appear at all. The primitive was built anyway, because the alternative was to leave a measured silent failure in the one carrier that exhibits it; but it rests on a single rule in a single dataset and this paragraph is the record of that.

T21 — the resource crossing is small, and quoting it alone would be quoting a break-even that does not exist. Section 5.4.3 reports a crossing of seven and four instances by wall-clock, and two to six by token channel. Those figures are favourable, they are easy to quote, and quoted by themselves they are misleading in a specific way: they divide one derivation by one baseline execution, and the derivation in the numerator is one that this system's own publication gate refuses. None of the nine recorded derivations is publishable. The crossing therefore answers "how many instances until a derivation pays for itself, *if* a derivation of this cost were deployable", and the conditional is doing the work.

The bias direction is unambiguous, and it has two components rather than one. The obvious one is the missing multiplier k of condition C-ii: if it takes several attempts to reach a publishable specification, the numerator is several derivations and not one. The less obvious one is that the crossing omits the human review that Section 4.5 makes part of compiling once — the recorded audit of this project's three datasets took an hour of a person's time across forty-two rows, and not one second of it appears in the arithmetic above. Both omissions push the same way. The rule this thesis adopts is therefore that the crossing may be reported only with the

count of publishable derivations beside it, in the same way that Section 5.8 reports a refusal channel only beside the number of documents the arm produced; a ratio whose numerator the system refuses to ship is a measurement of an arrangement nobody would deploy.

One thing this paragraph does *not* concede. The crossing being conditional bears on the first of the two motivations only. The confidentiality argument of Section 1.2 does not depend on N at all: where instance data may not leave the organization, the per-instance alternative is not an option at any instance count, and a crossing of four or of four thousand would not change that. Reading a cost result as a verdict on the whole method would conflate the two, and T16 is where the second one is recorded.

5.10.2 Threats Biased Against the Proposed Method

T5 — package preservation was, until recently, unmeasurable for Arm C. The integrity channel treats drawings and printer settings as critical parts of a spreadsheet package, which is why the government form was chosen as a dataset in the first place. Until it was corrected, the production write-back for XLSX serialized the workbook through a spreadsheet model, and that operation drops exactly those two parts — the same two parts the mutation suite's integrity negative control drops, because the negative control *is* a round trip through the same library. Arm C therefore scored 0 of 5 on integrity by construction, and the channel could not distinguish a correct run from a deliberately corrupted one.

The response was the first admissible one: the write-back was replaced by a package editor that edits the one worksheet part that must change and copies every other byte through. The measurement after that change, on the same five instances, is 5 of 5 with no preservation violation, while re-serializing the same outputs through the spreadsheet model still yields 0 of 5. Narrowing the channel to fit the writer was considered and rejected, because it would have been

a reduction of the thesis claim to match the implementation. The threat is recorded rather than deleted, because every integrity figure produced before that correction is an artifact of the writer and not a result.

T19 — the coverage denominator answers one question and is read as another, and what the agent decided is not always recoverable. The frozen applicability manifest gives every rule a set of document roles, and that field is written from the rule document's own category column: a consistency rule and a refusal rule carry every role they *govern*, since a refusal forbids producing any of the documents. The scorer reads the same field as a different statement — that each of those roles must contain a name implementing the rule — and for four of the DOCX dataset's thirty-four rules the two readings differ. Three of the four cannot be satisfied under the scorer's reading at all: the labels those rules name occur in only one of the two templates, verified over every package part of both, so no assignment naming both roles could point anywhere.

The effect is measurable and systematic rather than incidental. Across the fourth batch's three replicates, every rule that fell into the unattributed bucket had in fact been given a well-formed disposition and discarded by this predicate; *no rule was omitted by any agent*. Two rules were discarded in three replicates of three. The consequence runs against the method in both directions at once: the discarded assignments inflate the count of items blocking release, and they deflate the coverage numerator that Section 5.4.4 reports.

The manifest is not corrected. Its bytes and its canonical hash are pinned inside every specification's stored evidence, the arms have already been scored against it, and rewriting a frozen denominator after seeing results is the thing the methodology freeze exists to prevent; a corrected manifest would be a re-declared campaign. What has been corrected is the record: the validator now stores, on each discarded assignment, the disposition and the reason the agent

gave and the category the manifest assigned.

An earlier version of this passage went further and said that for the campaign already run what the agent decided about a discarded rule was not recoverable, so a count of honest refusals could not be given. That was wrong, and the way it was wrong is the more useful half. The validator does discard the disposition; nothing else does. Each derivation persists the agent's entire reply alongside the scored summary, one key below where the check had looked, and it has done so since well before the first of these batches. Recomputing from that record reproduces the stored coverage partition rule by rule and the recorded blocking list item by item, so the classification can be read rather than inferred.

Read that way, the two batches whose specification records survive declared nine honest refusals across six replicates; the blocking lists report six. The other three were reclassified as rules nobody mentioned, purely because the roles the assignment named did not equal the manifest's set — and in each case the roles the agent named were the ones whose template actually contains the label. The same rule is reported as an honest refusal in one replicate and as an omission in another, from an identical disposition. A count of how often this system honestly refused a rule is therefore given, and it is the larger of the two: what the release gate reports understates it by a third. The boundary is narrower than the earlier claim and worth stating exactly — a batch whose specification rows were deleted without first exporting them is genuinely unreadable, and one such batch exists.

T18 — the compiled arm's derivation sees one calibration instance, and the envelope declares three. The fairness condition of Section 5.3 requires that the calibration material visible to Arm A's execution, Arm B's skill build and Arm C's derivation be identical, and the cross-arm audit compares those artifact sets exactly. It holds as an *authorisation*: all three envelopes list the same $K = 3$ inputs and reference outputs. It does not hold as a description of con-

sumption. A derivation dataset in the implemented system carries one source instance and one reference output per document role, so the derivation reads one of the three worked examples while both baselines read all three. The limit is the product's data model, not a setting: there is no second slot to fill.

Declaring only the consumed instance was considered and rejected. The parity audit compares per-role artifact sets, so an Arm C envelope listing one calibration instance against Arm A's three would fail a correct run — and a check that rejects a correct action is worse than no check, a lesson this system has paid for repeatedly. What is enforced instead is the property that a wrong implementation could actually violate: the replicate setup ingests only bytes the envelope authorised, refuses anything else, and records the SHA-256 of every file it did hand to the product, so the instance each derivation consumed is in the evidence rather than in a claim.

The bias favours the baselines. Arms A and B see three input/output pairs; the derivation sees one. Whether a derivation given all three would produce a better specification is unmeasured, and so is the converse — how much the additional two examples are worth to a per-instance agent. Neither can be settled without a change to the data model and a second paid campaign. The disclosure exists because the discrepancy was found before the first Arm C derivation was paid for, by enumerating what the envelope declares against what the product reads, and because it runs in the unusual direction: most gaps of this kind, as Section 5.10 notes elsewhere, lean the other way.

One direction of it is now priced, on one dataset, by hand, and the price is against the method. The thirteen semantic errors left over in the X1 head-room diagnostic reported earlier in this section are a single column, the per-project remark, which all three derivations recorded as unresolved with the cause "target not found" and the reason that the column is blank in every row of the instance they were shown — so its cell never appears in the baseline-to-expected

difference a derivation reads, and the prompt forbids constructing an address it has not seen. That reason is true of the instance consumed and false of the set authorised: the cell is non-blank in exactly one of the three calibration instances, and it is not the one the derivation reads. Hand-writing that one column closes all thirteen remaining semantic errors and takes the nine silent failures to zero. One calibration instance out of three is therefore worth, on this template, thirteen scored cells and nine of twenty-three documents — and the refusal that stood in its place was honest, and correct about what the derivation could see.

T6 — the DOCX write path has not had the same audit. The correction in T5 covers XLSX only. The layout-bearing parts of a Word package have not been enumerated part by part, and S4 is the only DOCX evaluation dataset. Any DOCX integrity figure in this thesis therefore carries an unmeasured residual.

T7 — an unpublishable specification is not necessarily a failed one. The publish gate blocks on any unresolved item, and the unresolved set has repeatedly been found to contain entries that are not defects. Two counters existed for the same quantity — the agent's own self-reported list and the gate's canonical set — and they were never compared: on the best X1 specification they read 13 and 47. Thirty-four of the 47 were detail cells that a repeat block writes correctly, admitted to the gate's input list because the author-time oracle reads expected text from that same list for a different purpose. Any count of unresolved items must be decomposed before it is interpreted, and a headline "unpublishable" figure without that decomposition would understate the method.

T20 — an expressible construct the derivation did not use, and denied existed. The specification language can state the refusal condition of the DOCX dataset's carton-range rule. A row set may declare an interval predicate over two of its own ordinal columns, a violation stops

the run, and injecting that one declaration into an already-derived specification moves exactly the two evaluation instances it should and leaves the other twenty-eight untouched. The construct is documented in the derivation prompt, and the prompt's hash is pinned byte for byte inside every derivation envelope of the batch discussed here.

Three independent cold-start derivations nonetheless declared that rule rejected, and all three gave as their reason that no cross-row validation primitive exists. The reason is false. None of the three declared the construct; none declared the sibling construct for repeat-block row alignment either.

This is recorded as a boundary of the derivation, not of the language, and the distinction is the point. Adding expressiveness, writing it into the prompt, and adding a derivation-time check that the agent answered are three separate things, and this thesis has now measured all three combinations: with the check present and reachable the capability was used on the next batch; with the check present but structurally unreachable for the shapes the agents produce, it never fired; with no such check at all, the capability went unused and its absence was asserted. What generalises is the last clause rather than the particular rule: *a capability the system has, that the prompt describes, and that nothing checks for, is indistinguishable at the specification level from a capability the system lacks* — including to the agent writing the specification.

The direction runs against the proposed method twice over. The refusal channel for that dataset is reported at six of twelve when the language could express eight, and the two undetected instances are counted as silent failures, which is this thesis's primary metric. No claim is made that a different prompt, model, or number of replicates would behave the same way; that separation needs a re-declared campaign and was not bought. One observation constrains the obvious next attempt: the paragraph introducing the construct offers, two sentences later, an explicit instruction to record a refusal rule as rejected when the language cannot serve it. The

agents took that exit while denying the three lines above it, so “the description was unclear” is not the only hypothesis the wording supports.

5.10.3 Threats of Undetermined or Mixed Direction

T8 — dataset scale, at both levels, and they are not interchangeable. Three evaluation templates exist, their content is sealed, and all three passed the human-audit boundary on 30 August 2026, against an original intention of three to five per carrier. By instance count the relaxed lower bound of three audited evaluation instances per template is met on all three — three, four and thirty — and by method it is met on none, for the reason T12 gives. Passing it does not enlarge what three templates support: they support descriptive conclusions only, no statistical significance may be claimed, and no single average may be taken across carriers.

The instance level is a separate limit and it binds a different claim. Field-level accuracy rests on a large denominator — 2287, 1566 and 1458 scored fields on the three templates, 5311 in all — but the metric this thesis leads with, the silent-failure rate, is a proportion over *cases*, and each template contributes M evaluation cases of which one or two are refusals. A result of zero silent failures on a template carries a one-sided 95% upper bound of roughly 45% on the true rate at $M = 5$, falling to about 10% at $M = 30$; and because Section 5.5.1 forbids averaging a rate across carriers, it is the per-template bound that the main table must report, not the pooled one. That measurement is why M was raised to 30. **All three datasets have now reached it** — X1 on 26 August 2026, S5 on 28 August and S4 on 29 August — so the 10% bound applies to every template in the main table and the 45% bound applies to none of them.

Those three figures replaced a smaller set, and the substitution is recorded here rather than made silently, because earlier drafts of this thesis cited the smaller set. Two different counts of “the field-level denominator” were in use and neither had been written down: a hand count

of what a ground truth *contains* — the lengths of its value list plus the cells of its slot rows, obligations excluded — which gave 304, 226 and 225 over the five instances per template then available; and the total of the evaluator's frozen `semantic` channel, which gave 306, 232 and 234 over the same five. Both are now executable code, and the older one reproduces to the digit. The difference between them is obligations and the channel the frozen scoring policy routes each one to: X1's six all land in `semantic`, so $226 + 6$ is exactly 232, while S4 and S5 route some of theirs elsewhere. The second count is the one reported above, for a reason that is not a matter of taste: it counts the checks an arm is scored on, and any accuracy this thesis reports is a fraction of that, whereas the first counts entries in a file. The figures are taken from a frozen record regenerated by a single command and compared against a live recount by a test, so that Section 5.8 cites one artifact rather than a number in a log. Aligning the two counts does not make either correct: which positions *ought* to be scored is the annotation manifest's judgement, and that is T12.

Raising M does nothing for external validity, which is bounded by the template count above and by the fact that instances of one template are repeated draws from one rule document and one generator. The two limits are often conflated and must not be: more instances make "on these three templates" precise, not "on template-filling tasks". Raising M also makes the threat in T12 worse rather than better, and the same decision deliberately weakened its countermeasure: hand-recomputing at least three fields of *every* instance was the original lower bound, and at ninety instances and twenty-five measured minutes each it is about thirty-one hours, so the bound was relaxed to three instances per template. A shared misreading of a rule document is therefore now spread over more cases and sampled by a smaller audit. That trade is recorded here rather than in the protocol alone, because it is the kind of relaxation that would otherwise be invisible in the results.

T9 — the inclusion criterion is itself skewed. The datasets admit only tasks whose target row count is fixed, because the write side deliberately does not insert rows, and they exclude formula results. Both exclusions coincide with boundaries of the proposed method, so the excluded region may contain tasks the baselines can do and the compiled method cannot. This is the second admissible response — an explicit expressiveness boundary — and it is disclosed rather than argued away.

T10 — what the untouched channel does not reach. Untouched-region comparison covers cells and paragraphs. It does not reach footnotes, endnotes, comments, or other independent package parts, and styles and formula results are outside the first version of the scoring policy by design, to avoid confounding correctness with whether a spreadsheet application refreshed its cached values.

T11 — model dependence. Every derivation reported here used one model family. The ablation in Section 5.7 is the instrument for this and has not been run; until it is, no claim in this thesis is established as vendor-independent.

T12 — the correctness of the reference outputs. Discussed in Section 5.5.2. Mutation testing bounds the evaluator, not the generator; the residual is covered only by a spot audit. That audit was signed on 30 August 2026, and *what was signed is weaker than the procedure it discharges*, so this paragraph states both.

The annotation manual asks for an independent hand recomputation of the listed fields from the rule workbook, without opening the expected artifacts, measured at roughly twenty-five minutes for one instance. What was performed was a per-case read-through of each ground truth against the rules with the arithmetic checked: forty-two rows across the three datasets in one hour, about a minute and a half a row. Counted across the one hundred and thirteen

stored instances of the four datasets, forty-two now record a named reviewer, and *two* carry the stronger recomputation — the same two that carried it before the audit, one each on two of the three main datasets. The relaxed lower bound decided alongside M asks for three evaluation instances per template rather than all of them; by instance count it is met everywhere (three for S4, four for X1, thirty for S5) and by method it is met nowhere. The difference is written into each audit record as a `review_method` field and hashed into the seal digest alongside the verdicts, so a reader who obtains the record obtains the method with it and cannot recover one without the other. Recording the weaker procedure accurately is not a formality: an audit record that overstated what was done would defeat the only thing this gate exists to establish.

Enlarging all three datasets to thirty put a number on what the relaxation costs. Four audited instances would have covered four fifths of a five-instance set; they cover an eighth of a thirty-instance one. As signed, the reviewed fraction of each evaluation set is a tenth for S4, a seventh for X1 and all of S5 — and the last of those three is not the reassurance it looks like, because thirty cases reviewed at a minute and a half each is a different instrument from three reviewed at twenty-five. Raising M and weakening its countermeasure were two halves of one decision, and this is the exchange rate between them. Seventy-five of the one hundred and thirteen instances now in the repository were produced by a draw rather than written by hand, so the residual is also less evenly distributed than the fraction suggests: nobody chose what any individual sampled instance would test, and the property a reviewer can still check is the declared distribution rather than the case. The class of error this leaves open is a rule misread identically by the rule document and the generator, in which case every number is wrong together and every cross-check between the numbers still holds; the single audit performed on the DOCX dataset found one instance of exactly that class. The direction is undetermined rather than favourable: all three arms read the same rule document and see the same calibration instances with their expected

outputs, so a shared misreading penalizes whichever arm reads the rules correctly, and nothing in the protocol determines which.

T14 — Arm C now enters through the production controller, and the residual gap is per–carrier evidence rather than per-carrier capability. For most of this work Arm C's execution leg did not enter through the production controller, and that was not a stylistic choice. The controller required every reference a plan writes to already *hold something* on the template, which is the right question for a document and the wrong one for a spreadsheet, where a repeat slot is by definition an empty cell. Measured on the spreadsheet dataset, the controller rejected 138 of the 142 references its own specification writes, while the addressability test the derivation-time tools had already been corrected to use accepts all 142; the document dataset passed with none missing, which is exactly why it went unnoticed for so long — the only dataset ever driven through that path was the DOCX one.

That defect is now fixed, and the fix was verified against the stored specifications rather than against purpose-built inputs: the spreadsheet specification goes from 138 rejected references to none, the document specification stays at none out of 70 and none out of 98, and on the document carrier the two tests return not merely equal counts but the identical set of references. The guard remains reachable in each shape a reference can be wrong — a sheet that is absent, a defined name that is absent, a bookmark that is absent — because a check that refuses nothing is worse than no check at all. Arm C consequently runs through the shipping entry point for the first time, and its output on five calibration and development instances of the spreadsheet dataset is byte-for-byte identical to the diagnostic path that produced every Arm C figure recorded so far, across 380 compared cells, with both an input-level and a cell-level negative control.

Two residues remain and are stated rather than resolved. First, the end-to-end comparison exists only on the spreadsheet carrier: the document dataset has no equivalent diagnostic path

to compare against, so the evidence that the arm under test is the shipping system rests on unit tests there rather than on a cell-level identity. Second, relaxing that guard promoted the writer's own refusal to sole responsibility for a narrow class — a coordinate covered by a merged range — which changes how such a case is reported without changing whether it fails; nothing is stored either way. Neither residue affects any figure in this chapter, because Arm C's numbers are produced by the compiler and the write adapter, the components the zero-execution-LLM claim is actually about. The bias direction of what remains still favours the proposed method: a reader would otherwise credit the deterministic arm with cell-level end-to-end evidence on both carriers when it exists on one.

T15 — the drift variants are generated, not observed. The twenty-eight variants of Section 5.6.2 are constructed from the drift protocol's four named classes and from what these particular forms plausibly do between editions. No corpus of real revision histories for these documents was consulted, so nothing is claimed about how often real revisions fall into each class, and the drift results will be descriptive of the constructed variants rather than of a population of revisions. The construction is also tidier than a revision made by hand: each edit is a byte-level patch to individual package parts, which is what makes it attributable, and the compensations for that tidiness are partial — row-anchored drawings are moved with their rows, but no claim is made that the result is indistinguishable from an edit made in an office application. The direction of the bias is not determined: a cleaner variant may be easier for a baseline to cope with, and a variant class that real revisions rarely produce may flatter either side. This is the third admissible response — the design is stated, the experiment will be reported as descriptive, and the gap is disclosed rather than argued away.

T13 — two responses to the pilot that were rejected, and why. After the pilot in Section 5.1 showed the per-instance baseline to be semantically accurate, two adjustments were available and both were declined. Increasing task difficulty until the baseline begins to fail would tune the benchmark towards producing the desired comparison, and substituting a weaker model would amount to selecting the opponent. Recording them here is part of the claim: the emphasis of this thesis moved to consistency and cost structure *because* the baseline was accurate, not because the tasks were made harder.

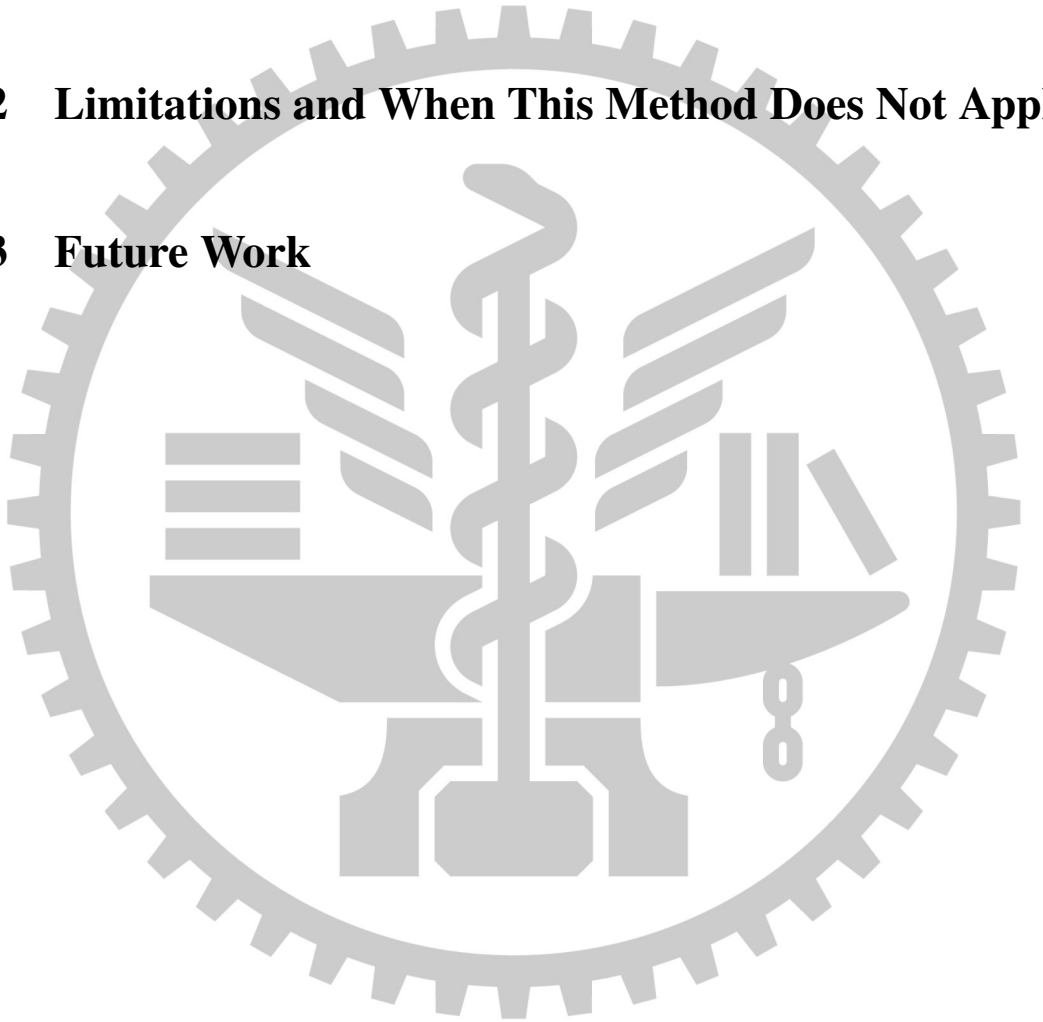
Status: implementation-backed for T1, T2, T5, T6, T7, T9, T10, T12, T14, T15, T17, T18, T19, T20; normative for T3, T4, T8, T11, T13, T16. The items marked implementation-backed each name a measurement in this repository. The others are stated positions or unrun experiments, and are labelled so that no reader takes an intention for a result.

Chapter 6. Conclusions

6.1 Conclusion

6.2 Limitations and When This Method Does Not Apply

6.3 Future Work



References

[1] S. Gulwani, "Automating string processing in spreadsheets using input-output examples," in *Proc. ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, 2011, pp. 317–330.

[2] J. Cambronero, S. Gulwani, V. Le, D. Perelman, A. Radhakrishna, C. Simon, and A. Tiwari, "Flashfill++: Scaling programming by example by cutting to the chase," *Proceedings of the ACM on Programming Languages*, vol. 7, no. POPL, pp. 952–981, 2023.

[3] T. Cai, X. Wang, T. Ma, X. Chen, and D. Zhou, "Large language models as tool makers," in *Proc. International Conference on Learning Representations (ICLR)*, 2024.

[4] G. Wang, Y. Xie, Y. Jiang, A. Mandlekar, C. Xiao, Y. Zhu, L. Fan, and A. Anandkumar, "Voyager: An open-ended embodied agent with large language models," *Transactions on Machine Learning Research*, 2024.

[5] Z. Wang, D. Fried, and G. Neubig, "Trove: Inducing verifiable and efficient toolboxes for solving programmatic tasks," in *Proc. International Conference on Machine Learning (ICML)*, 2024, pp. 51 177–51 191.

[6] L. Yuan, Y. Chen, X. Wang, Y. R. Fung, H. Peng, and H. Ji, "Craft: Customizing llms by creating and retrieving from specialized toolsets," in *Proc. International Conference on Learning Representations (ICLR)*, 2024.

[7] E. Stengel-Eskin, A. Prasad, and M. Bansal, "Regal: Refactoring programs to discover generalizable abstractions," in *Proc. International Conference on Machine Learning (ICML)*, 2024.

[8] K. Kujanpää, N. Liu, S. Alam, Y. R. Sura, T. Yang, K. Klinkner, and S. Malmasi, "Tool-making and self-evolving llm agents in low-latency systems," *arXiv preprint arXiv:2607.08010*, 2026.

[9] N. Shinn, F. Cassano, E. Berman, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language agents with verbal reinforcement learning," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[10] X. Chen, M. Lin, N. Schärli, and D. Zhou, "Teaching large language models to self-debug," in *Proc. International Conference on Learning Representations (ICLR)*, 2024.

[11] L. Zhong, Z. Wang, and J. Shang, "Debug like a human: A large language model debugger via verifying runtime execution step-by-step," in *Proc. Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024.

[12] T. Ridnik, D. Kredo, and I. Friedman, "Code generation with alphacodium: From prompt engineering to flow engineering," *arXiv preprint arXiv:2401.08500*, 2024.

[13] A. Prasad, E. Stengel-Eskin, J. C.-Y. Chen, Z. Khan, and M. Bansal, "Learning to generate unit tests for automated debugging," *arXiv preprint arXiv:2502.01619*, 2025.

[14] X. Chen, Z. Tao, K. Zhang, C. Zhou, W. Gu, Y. He, M. Zhang, X. Cai, H. Zhao, and Z. Jin, "Revisit self-debugging with self-generated tests for code generation," in *Proc. Annual Meeting of the Association for Computational Linguistics (ACL)*, 2025.

[15] M. H. Erol, B. El, M. Suzgun, M. Yuksekgonul, and J. Zou, "Cost-of-pass: An economic framework for evaluating language models," *arXiv preprint arXiv:2504.13359*, 2025.

[16] J. Kim, B. Shin, J. Chung, and M. Rhu, "The cost of dynamic reasoning: Demystifying ai agents and test-time scaling from an ai infrastructure perspective," in *Proc. IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, 2026.

[17] S. Yao, N. Shinn, P. Razavi, and K. Narasimhan, "τ-bench: A benchmark for tool-agent-user interaction in real-world domains," in *Proc. International Conference on Learning Representations (ICLR)*, 2025.

[18] T. Ma, Y. Chen, V. Anand, A. Cornacchia, A. R. Faustino, G. Liu, S. Zhang, H. Luo, S. A. Fahmy, Z. A. Qazi, and M. Canini, "Maestro: Multi-agent evaluation suite for testing, reliability, and observability," *arXiv preprint arXiv:2601.00481*, 2026.

[19] S. Rabanser, S. Kapoor, P. Kirgis, K. Liu, S. Utpala, and A. Narayanan, "Towards a science of ai agent reliability," *arXiv preprint arXiv:2602.16666*, 2026, accepted at ICML 2026.

[20] E. Musumeci, M. Brienza, V. Suriani, D. Nardi, and D. D. Bloisi, "Llm based multi-agent generation of semi-structured documents from semantic templates in the public administration domain," in *Proc. International Conference on Human-Computer Interaction (HCII)*, 2024.

[21] Z. Ma, B. Zhang, J. Zhang, J. Yu, X. Zhang, X. Zhang, S. Luo, X. Wang, and J. Tang, "Spreadsheetbench: Towards challenging real world spreadsheet manipulation," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[22] T. Lv, D. Zhang, J. Ding, Y. Jia, Y. Zhao, Y. Huang, W. Wu, X. Zhou, S. Huang, N. Yang, L. Dong, L. Cui, and F. Wei, "Mind the gap: Can frontier llms pass a standardized office proficiency exam?" *arXiv preprint arXiv:2606.10956*, 2026.

[23] H. Li, J. Su, Y. Chen, Q. Li, and Z. Zhang, "Sheetcopilot: Bringing software productivity to the next level through large language models," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[24] T. Xie, D. Zhang, J. Chen, X. Li, S. Zhao, R. Cao, T. J. Hua, Z. Cheng, D. Shin, F. Lei *et al.*, "Osworld: Benchmarking multimodal agents for open-ended tasks in real computer environments," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[25] H. H. Zhao, K. Yang, W. Yu, D. Gao, and M. Z. Shou, "Worldgui: An interactive benchmark for desktop gui automation from any starting point," *arXiv preprint arXiv:2502.08047*, 2025.

[26] H. Dong, J. Zhao, Y. Tian, J. Xiong, S. Xia, M. Zhou, Y. Lin, J. Cambronero, Y. He, S. Han, and D. Zhang, "Encoding spreadsheets for large language models," in *Proc. Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2024.

[27] Y. Xu, M. Li, L. Cui, S. Huang, F. Wei, and M. Zhou, "Layoutlm: Pre-training of text and layout for document image understanding," in *Proc. ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD)*, 2020, pp. 1192–1200.

[28] G. Kim, T. Hong, M. Yim, J. Nam, J. Park, J. Yim, W. Hwang, S. Yun, D. Han, and S. Park, "Ocr-free document understanding transformer," in *Proc. European Conference on Computer Vision (ECCV)*, 2022, pp. 498–517.

[29] D. Wang, N. Raman, M. Sibue, Z. Ma, P. Babkin, S. Kaur, Y. Pei, A. Nourbakhsh, and X. Liu, "Docllm: A layout-aware generative language model for multimodal document understanding," in *Proc. Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024.

[30] I. Cachola, S. Cucerzan, A. Herring, V. Mijovic, E. Oveson, and S. K. Jauhar, "Knowledge-centric templatic views of documents," *arXiv preprint arXiv:2401.06945*, 2024.

[31] Q. Yang, X. Ma, X. Wang, H. Wang, and R. Wang, "Babeldoc: Better layout-preserving pdf translation via intermediate representation," *arXiv preprint arXiv:2605.10845*, 2026, aCL 2026 System Demonstration paper.

[32] P. Laban, T. Schnabel, and J. Neville, "Llms corrupt your documents when you delegate," *arXiv preprint arXiv:2604.15597*, 2026.

[33] M. Suri, P. Mathur, F. Dernoncourt, R. Jain, V. I. Morariu, R. Sawhney, P. Nakov, and D. Manocha, "Docedit-v2: Document structure editing via multimodal llm grounding," in *Proc. Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2024.

[34] X. Zhang, S. Luo, B. Zhang, Z. Ma, J. Zhang, Y. Li, G. Li, Z. Yao, K. Xu, J. Zhou *et al.*, "Tablellm: Enabling tabular data manipulation by llms in real office usage scenarios," in *Findings of the Association for Computational Linguistics: ACL*, 2025.

[35] X. Cheng, R. Zhu, K. Liu, R. C. Machineni, L. Chen, B. Zhu, D. Jin, Z. Qi, N. Parihar, Z. Xu, and O. Gao, "Sheetmind: An end-to-end llm-powered multi-agent framework for spreadsheet automation," *arXiv preprint arXiv:2506.12339*, 2025.

[36] W. Zhao, Z. Hou, S. Wu, Y. Gao, H. Dong, Y. Wan, H. Zhang, Y. Sui, and H. Zhang, "Nl2formula: Generating spreadsheet formulas from natural language queries," in *Findings of the Association for Computational Linguistics: EACL*, 2024.

[37] J. Wewerka and M. Reichert, "Robotic process automation — a systematic literature review and assessment framework," *arXiv preprint arXiv:2012.11951*, 2020.

[38] Y. Ye, X. Cong, S. Tian, J. Cao, H. Wang, Y. Qin, Y. Lu, H. Yu, H. Wang, Y. Lin, Z. Liu, and M. Sun, "Proagent: From robotic process automation to agentic process automation," *arXiv preprint arXiv:2311.10751*, 2023.

[39] T. Huang, C. Yu, W. Shi, Z. Peng, D. Yang, W. Sun, and Y. Shi, "Prompt2task: Automating ui tasks on smartphones from textual prompts," *ACM Transactions on Computer-Human Interaction*, 2025.

[40] O. Polozov and S. Gulwani, "Flashmeta: A framework for inductive program synthesis," in *Proc. ACM SIGPLAN International Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA)*, 2015, pp. 107–126.

[41] J. Devlin, J. Uesato, S. Bhupatiraju, R. Singh, A.-r. Mohamed, and P. Kohli, "Robustfill: Neural program learning under noisy i/o," in *Proc. International Conference on Machine Learning (ICML)*, 2017, pp. 990–998.

[42] I. Polosukhin and A. Skidanov, "Neural program search: Solving programming tasks from description and examples," in *Proc. International Conference on Learning Representations (ICLR), Workshop Track*, 2018.

[43] W.-D. Li and K. Ellis, "Is programming by example solved by llms?" in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[44] R. Khan, S. Gulwani, V. Le, A. Radhakrishna, A. Tiwari, and G. Verbruggen, "Llm-guided compositional program synthesis," *arXiv preprint arXiv:2503.15540*, 2025.

[45] A. Wei, T. Suresh, J. Cao, N. Kannan, Y. Wu, K. Yan, T. S. F. X. Teixeira, K. Wang, and A. Aiken, "Codearc: Benchmarking reasoning capabilities of llm agents for inductive program synthesis," *arXiv preprint arXiv:2503.23145*, 2025, also in Proc. Conference on Language Modeling (COLM), 2025.

[46] Y. Fu, B. Li, L. Li, W. Zhang, and T. Xie, "The first prompt counts the most! an evaluation of large language models on iterative example-based code generation," *Proceedings of the ACM on Software Engineering*, vol. 2, no. ISSTA, 2025.

[47] S. Zhang and Y. Park, "Pbe meets llm: When few examples aren't few-shot enough," *arXiv preprint arXiv:2507.05403*, 2025.

[48] C. Cummins, V. Seeker, J. Armengol-Estapé, A. H. Markosyan, G. Synnaeve, and H. Leather, "Don't transform the code, code the transforms: Towards precise code rewriting using llms," *arXiv preprint arXiv:2410.08806*, 2024.

[49] T. Scholak, N. Schucher, and D. Bahdanau, "Picard: Parsing incrementally for constrained auto-regressive decoding from language models," in *Proc. Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2021, pp. 9895–9901.

[50] K. Claessen and J. Hughes, "Quickcheck: A lightweight tool for random testing of haskell programs," in *Proc. ACM SIGPLAN International Conference on Functional Programming (ICFP)*, 2000, pp. 268–279.

[51] T. Y. Chen, F.-C. Kuo, H. Liu, P.-L. Poon, D. Towey, T. H. Tse, and Z. Q. Zhou, "Metamorphic testing: A review of challenges and opportunities," *ACM Computing Surveys*, vol. 51, no. 1, pp. 4:1–4:27, 2018.

[52] S. Li, C. Xu, J. Wang, X. Gong, C. Chen, J. Zhang, J. Wang, K.-Y. Lam, and S. Ji, "Llms cannot reliably judge (yet?): A comprehensive assessment on the robustness of llm-as-a-judge," *arXiv preprint arXiv:2506.09443*, 2025.

[53] K. Wataoka, T. Takahashi, and R. Ri, "Self-preference bias in llm-as-a-judge," *arXiv preprint arXiv:2410.21819*, 2024.

[54] S. Tan, S. Zhuang, K. Montgomery, W. Y. Tang, A. Cuadron, C. Wang, R. A. Popa, and I. Stoica, "Judgebench: A benchmark for evaluating llm-based judges," in *Proc. International Conference on Learning Representations (ICLR)*, 2025.

[55] X. Wang, J. Wei, D. Schuurmans, Q. Le, E. Chi, S. Narang, A. Chowdhery, and D. Zhou, "Self-consistency improves chain of thought reasoning in language models," in *Proc. International Conference on Learning Representations (ICLR)*, 2023.

[56] L. Gao, A. Madaan, S. Zhou, U. Alon, P. Liu, Y. Yang, J. Callan, and G. Neubig, "Pal: Program-aided language models," in *Proc. International Conference on Machine Learning (ICML)*, 2023.

[57] W. Chen, X. Ma, X. Wang, and W. W. Cohen, "Program of thoughts prompting: Disentangling computation from reasoning for numerical reasoning tasks," *Transactions on Machine Learning Research*, 2023.

[58] A. Malinin and M. J. F. Gales, "Uncertainty estimation in autoregressive structured prediction," in *Proc. International Conference on Learning Representations (ICLR)*, 2021.

[59] L. Kuhn, Y. Gal, and S. Farquhar, "Semantic uncertainty: Linguistic invariances for uncertainty estimation in natural language generation," in *Proc. International Conference on Learning Representations (ICLR)*, 2023.

[60] S. Farquhar, J. Kossen, L. Kuhn, and Y. Gal, "Detecting hallucinations in large language models using semantic entropy," *Nature*, vol. 630, no. 8017, pp. 625–630, 2024.

[61] K. Tian, E. Mitchell, A. Zhou, A. Sharma, R. Rafailov, H. Yao, C. Finn, and C. D. Manning, "Just ask for calibration: Strategies for eliciting calibrated confidence scores from language models fine-tuned with human feedback," in *Proc. Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023, pp. 5433–5442.

[62] V. Quach, A. Fisch, T. Schuster, A. Yala, J. H. Sohn, T. S. Jaakkola, and R. Barzilay, "Conformal language modeling," in *Proc. International Conference on Learning Representations (ICLR)*, 2024.